Data Mining

Study Text for the Final State Examination

Final State Examination in Artificial Intelligence, MFF UK

August 2026

This text covers the examination topic *Data Mining* of the master's final state examination in Artificial Intelligence at MFF UK. Its chapters correspond **one to one** to the sentences of the official wording of the topic (see the mapping table below). It is based on the slides of the lecture courses **NDBI023 Data Mining** and **NAIL116 Social Networks and their Analysis** (I. Mrázová, MFF UK), supplemented with the standard material of the textbooks Han, Kamber, Pei: *Data Mining — Concepts and Techniques*, Tan, Steinbach, Kumar: *Introduction to Data Mining*, Aggarwal: *Data Mining — The Textbook*, and for networks Easley, Kleinberg: *Networks, Crowds and Markets*, Newman: *Networks* and Zafarani, Abbasi, Liu: *Social Media Mining*. It contains definitions, pseudocode, derivations, numerical examples and comparison tables; every chapter ends with a summary for the oral examination. This is an English translation of the Czech study text *Dobývání znalostí*.

Scope and marking

The length of the chapters reflects the weight of the material: most space goes to the data mining methods (Chapter 3) and to social network analysis (Chapter 6) — that is, to what both lecture courses treat in most detail and what the topic names most concretely.

The marker [♦ beyond the slides] flags material that is **in the slides of neither course** (the slides are available in full) but has been kept in the text — either because *the official wording of the topic names it*, or because the surrounding exposition would not make sense without it. It is an offer for deletion: whatever is not marked is backed by the slides. Where the marker is followed by a colon, it states precisely what the reservation applies to — typically that a summarising table or framework is mine, while the individual items inside it do come from the slides.

The slides cover: data analysis and clustering, association rules, decision trees and ensembles, SVM with classifier evaluation, Bayesian models and ELM (NDBI023); basic concepts, influence modelling, homophily and affiliation, link analysis, and complex networks with community detection and link prediction (NAIL116). The most conspicuous gap is the **KDD process and methodologies** (Chapter 1). The chapter on **characteristic and discriminant rules** (Chapter 4) is built primarily on the slides of the course *Internet and classification methods* (Holeňa, lectures `ikm1`–`ikm6`, above all `ikm5`); only the delimitation of the two kinds of rules with the t-weight/d-weight measures follows Han–Kamber there. The slides `nmr1`–`nmr5` from the same folder belong to a different course and do not cover the subject matter of the chapter (see the note at its start).

Mapping of the examination topic to chapters

Sentence of the official topic	Chapter in this text
Basic paradigms of data mining.	1 Basic paradigms of data mining
Data preparation; attribute selection and methods for the analysis of their relevance.	2 Data preparation, attribute selection and relevance analysis
Methods for data mining; association rules, approaches based on the principle of supervised learning, and cluster analysis.	3 Methods for data mining (3.1 association rules, 3.2 supervised learning, 3.3 cluster analysis)
Methods for the extraction of characteristic discriminant rules and the measurement of their interestingness.	4 Characteristic and discriminant rules and their interestingness
Representation, evaluation and visualisation of the acquired knowledge.	5 Representation, evaluation and visualisation of knowledge
Models for social network analysis; centrality measures, community detection.	6 Models for social network analysis, centrality and community detection
Practical use of techniques for data mining and social network analysis.	7 Practical use of the techniques

A note on terminology. The Czech original gives the English equivalent in italics in parentheses at the first occurrence; in this English version the italicised terms are kept where they mark a technical term being introduced.

Contents

1 Basic paradigms of data mining

This chapter delimits the field, its process, and the basic classification of tasks and data. It is the frame into which all the other chapters fit: without it one cannot explain why data preparation is a chapter of its own, and why the methods are divided precisely into association rules, supervised learning and clustering.

1.1 Motivation and delimitation of the field

Organisations accumulate data in volumes that far exceed a human's capacity to inspect them. Classical querying (SQL, OLAP) can answer a question we already know how to ask; *data mining* answers the questions we did not know how to ask — it looks in the data for structures whose existence we did not know about in advance.

An illustrative example: a system went through 20 million procedures billed to a health insurance company, flagged 214 thousand of them as odd, and passed the 14 thousand most suspicious ones on for manual review. The reviewing physicians, who are able to inspect on the order of hundreds of cases, immediately marked 8 thousand of the submitted cases as fraudulent. The machine did not replace the expert here — it *focused* their attention.

Knowledge discovery in databases

Knowledge discovery in databases (KDD) is the non-trivial process of extracting implicit, previously unknown and potentially useful information from data.
All four words matter:

- **non-trivial** — it is not a mere query or aggregation;
- **implicit** — the knowledge is not written down explicitly in the data;
- **previously unknown** — a result everybody already knows is worthless;
- **potentially useful** — it must lead to an action or a decision.

The term *data mining* (DM) refers, in the narrow sense, only to one step of the KDD process (the actual application of the analytical algorithms); in common practice, however, the two terms are used interchangeably. The field took shape in the 1990s at the intersection of three disciplines:

- **artificial intelligence** — in particular machine learning methods (rule induction, trees, neural networks);
- **database systems** — storage of and efficient access to large data sets, data warehouses, information retrieval;
- **statistics** — modelling and analysis of the dependencies found in the data, hypothesis testing.

The fourth — and in practice decisive — component is the need to use the processed data to support (strategic) decision-making in an enterprise. Without it, data mining reduces to an academic exercise.

1.2 The KDD process

[♦ beyond the slides: the five-phase framework as such; the activities inside the phases are in the slides] KDD is an **interactive and iterative** process: the results of one step routinely lead back to the preceding steps. The classical scheme (Fayyad et al.) has five phases:

Phase	Input → output	Content
Selection	original data → selected data	selection of the relevant records and attributes from the data warehouse
Preprocessing	selected data → preprocessed data	cleaning, missing values, noise, outliers, integration of sources
Transformation	preprocessed data → transformed data	normalisation, discretisation, derived attributes, dimensionality reduction
Data mining proper	transformed data → patterns	application of the algorithms (trees, associations, clusters, ...)
Interpretation	patterns → knowledge	evaluation from the end user's point of view, visualisation, deployment

The goal of the first three phases (collectively *data preparation*, Chapter 2) is to build, out of data stored in a complex structure, **one table** containing the relevant information about the objects under study (a bank's clients, customers, ...). This step typically consumes 60–80 % of the total project time.

Seven steps through a manager's eyes. [♦ beyond the slides] An organisational view of the same process (the impulse is a problem, the output is the knowledge to solve it):

1. **team** — experts on the domain, on the data, and on KDD methods;
2. **problem specification** — "increase profit" → "predict customer churn";
3. **data collection** — including external data on the environment (season, campaigns, weather);
4. **choice of methods**;
5. **preprocessing**;
6. **mining** — repeatedly; the results tune the parameters of later runs;
7. **interpretation** — an analytical report, possibly an action.

1.3 Types of data mining tasks

Tasks are classified from two independent points of view: *what we want from the result* and *whether a target variable is available*.

[♦ beyond the slides] **By what we want from the result:**

- **Classification / prediction** — classify new cases, resp. estimate future development (weather, share prices, electricity consumption). What matters is *accuracy*, not simplicity; the result tends to be a lot of knowledge, hard to interpret.
- **Description** (characterisation, profile) — capture the dominant structure or relations in the data (the profile of a creditworthy client). What matters is *comprehensibility* with full coverage of the concept; the knowledge is smaller and less accurate.
- **Search for "nuggets"** — find new, *surprising* knowledge (an unusual combination of goods in a basket, a suspicious pattern of transactions); it *need not* cover the whole concept.

Descriptive vs. predictive tasks

Descriptive tasks characterise the general properties of the data: characterisation and discrimination, association and correlation analysis, clustering, outlier detection, evolution analysis.
Predictive tasks perform inference over the data in order to predict: classification (discrete target attribute), regression (continuous target attribute), time series prediction.

By the availability of a target variable: *supervised learning* — classification, regression; *unsupervised learning* — clustering, association rules, outlier detection; *semi-supervised* — a small labelled and a large unlabelled part of the data.

This division is also the key to Chapter 3: the topic names in it exactly one representative of unsupervised learning on transactional data (association rules), supervised learning, and unsupervised learning on metric data (cluster analysis).

1.4 Types of data

By attribute type — this determines which operations are meaningful and which methods can be used:

Scale	Operations	Example	Statistics
Nominal	$=, \neq$	colour, kind of fruit	mode, frequencies
Ordinal	$=, <$	grade, {low, middle, high}	median, quantiles
Interval-scaled	$+, -$	temperature in °C, date	mean, standard deviation
Ratio-scaled	$\times, /$	income, weight, frequency	geometric mean

For the interval scale the key point is that *intervals keep the same importance throughout the scale*: the difference in age between 10 and 20 is the same as that between 40 and 50. The ratio scale additionally has an absolute zero, so ratios are meaningful; ratio-scaled attributes with exponential behaviour (e.g. the total amount of microorganisms Ae^{Bt}) are first log-transformed and then treated like interval-scaled ones. A *nominal* attribute with v values is converted, for numeric methods, into v binary attributes (value 1 for the corresponding category, 0 otherwise); an *ordinal* attribute is commonly treated as interval-scaled.

By the structure of the data set:

- **Tabular / relational data** — objects × attributes; the standard input of most methods.
- **Transactional data** — records of variable length (the shopping basket); the input of association rules.
- **Temporal data** — e.g. time series of stock prices. A typical task is to predict future values; the series is transformed into a table by a sliding window: inputs $y(t_0), y(t_1), y(t_2), y(t_3) \rightarrow$ output $y(t_4)$; $y(t_1), \ldots, y(t_4) \rightarrow y(t_5)$ and so on.
- **Spatial data** — geographic information systems; an implicit neighbourhood relation (the attribute values of adjacent objects do not differ too much), useful especially for interpretation.
- **Structural data** — e.g. chemical compounds written as SMILES strings or as files of facts in Prolog; a task might be the classification of substances by their structure.
- **Text and web data, multimedia, graphs and networks** — these require a specific representation (the vector model, graph models, see Chapter 6).

1.5 Data mining methodologies

[♦ beyond the slides] The purpose of a methodology is to provide a uniform framework and to guide applications regardless of the line of business — to allow the sharing and transfer of experience from successful projects. Methodologies arise either at the large BI tool vendors (SEMMA by SAS Institute, 5A by SPSS), or as neutral, software-independent standards (CRISP-DM).

SEMMA (SAS Enterprise Miner), five phases: *Sample* — selection of data for modelling (sampling, imputation, partitioning into training, test and validation sets); *Explore* — visual exploration and reduction of the data (outliers, multidimensional histograms, clustering, Kohonen self-organising maps); *Modify* — preparation of objects, values and variables (selection, creation, transformation); *Model* — application of the methods (trees, regression, neural networks); *Assess* — evaluation of reliability and usefulness; *the user must understand them.*

CRISP-DM (CRoss-Industry Standard Process for Data Mining) — an open process model neutral with respect to industry, tool and application; the development was funded by the ESPRIT initiative, and the consortium consisted of ISL (later SPSS, today IBM), Teradata, NCR, Daimler AG and the insurance company OHRA. Six phases whose **order is not strict** — the results of one phase determine the choice of the next steps and some steps must be repeated (the process is drawn as a cycle with an outer cycle of deployment and monitoring):

1. **Business Understanding** — business objectives and success criteria; an inventory of resources (people, information, software, computing capacity), assumptions and constraints, risks, costs

and benefits; the data mining goals; the project plan.
2. **Data Understanding** — collection of the initial data; its description (quantity, properties, format); exploration and visualisation (distributions of the key attributes, relations of pairs of attributes, important subpopulations, simple statistics); verification of data quality.
3. **Data Preparation** — selection, cleaning, construction of derived attributes, integration, aggregation, formatting; performed iteratively and in varying order.
4. **Modeling** — choice of technique (with regard to its assumptions, which may force further adjustments of the data), test design, building the model with various parameters, assessing the model.
5. **Evaluation** — the results against the *business* success criteria; approval of the models; review of the process; determination of the next steps.
6. **Deployment** — presentation in a form easily interpretable and deployable by the user; deployment plan, monitoring and maintenance; final report and project review (documentation of the difficulties and blind alleys).

ASUM-DM (IBM, 2015) responds to the main criticism of CRISP-DM — its weak support for project management. Its phases are *Analyze*, *Design*, *Configure & Build*, *Deploy*, *Operate & Optimize*, with *Project Management* running across all of them.

	SEMMA	**CRISP-DM**	**ASUM-DM**
Origin	SAS Institute	ESPRIT / consortium	IBM (2015)
Neutrality	tied to SAS EM	tool-neutral	tied to IBM
Business phase	missing	yes (phases 1 and 5)	yes
Project management	no	weak	explicit, cross-cutting
Number of phases	5	6	5 + PM

1.6 Summary for the examination

- **KDD** = the non-trivial process of extracting implicit, previously unknown and potentially useful information from data; interactive and iterative.
- **Five KDD phases**: selection → preprocessing → transformation → mining → interpretation. Data preparation consumes 60–80 % of the time.
- **Roots of the field**: artificial intelligence (machine learning), databases, statistics + the need for decision support. **OLAP** answers questions posed in advance, KDD looks for patterns we do not know about.
- **Three types of task**: classification/prediction (accuracy > simplicity), description (comprehensibility, full coverage), search for nuggets (surprise, need not cover).
- **Descriptive vs. predictive** tasks; **supervised vs. unsupervised** vs. **semi-supervised**.
- **Attribute scales**: nominal, ordinal, interval, ratio — they determine the admissible operations and the methods. Nominal → v binary attributes; a ratio-scaled attribute with exponential behaviour → take logarithms.
- **Methodologies**: SEMMA (Sample, Explore, Modify, Model, Assess), CRISP-DM (6 phases, order not strict, cyclic), ASUM-DM (CRISP-DM + project management).
- The most frequent follow-up question: *why is the order of the CRISP-DM phases not strict?* — because the results of a phase determine the next steps; e.g. the data quality found leads back to a reformulation of the goal.

2 Data preparation, attribute selection and relevance analysis

The sentence of the topic joins two things into one chapter: *data preparation* (what has to be done so that the data is processable at all) and *attribute selection with relevance analysis* (how to pick, out of many attributes, those that carry information). The second part is really a special case of the first — dimensionality reduction — but the topic names it separately, because knowledge of concrete criteria and the ability to compute them are expected.

2.1 Why data preparation is crucial

Preprocessing is *of key importance for the success of the application*, it is difficult and time-consuming, and collaboration with a domain expert is an advantage. The goals are:

- to select (or create), from the available data, the data relevant for the task at hand,
- to represent it in a form suitable for the chosen algorithm,
- the final state being *(one)* data table containing the attribute values of the objects.

[♦ beyond the slides] Data quality is judged along several dimensions: *accuracy*, *completeness*, *consistency*, *timeliness*, *believability*, *interpretability*. The principle "garbage in, garbage out" holds without exception: the best algorithm on bad data produces only confident nonsense.

2.2 Data cleaning: missing values

The basic strategies (in order from the simplest to the most accurate):

1. **Ignore objects** with any missing value. Simple, but with many attributes we lose most of the data; moreover it introduces bias if the missing values are not random.
2. **Replace with a new value** "I don't know". The missing value becomes a full-fledged category — appropriate when the absence itself carries information (an unfilled income field in a loan application).
3. **Replace with an existing attribute value**: the most frequent value (the mode; for numeric attributes the mean or median), proportionally to the ratio of all values (randomly according to the empirical distribution), or any value.
4. **Impute the value using a model** — train a predictive model (regression, decision tree, k-NN) that predicts the missing attribute from the others. The most accurate, the most computationally demanding, and it risks introducing an artificial dependence between attributes.

Some methods can handle missing values themselves: C4.5 ignores them when constructing the tree (the splitting criterion is computed only from the known values) and estimates them at classification time from other patterns; CART does not count missing values towards the decision criterion; naive Bayes can classify even incompletely described patterns (Chapter 3).

2.3 Data cleaning: noise and outliers

Noise is a random error or variance in a measured quantity. Smoothing techniques: **binning** (values are divided into bins and replaced by the bin mean, median or boundary), **regression** (values are replaced by the prediction of a regression function), **clustering** (values outside clusters are marked as outliers) and a **combination of machine and human** (suspicious values are submitted for review).

Range, quartiles, interquartile range, outlier

The *range* of a set of data is the difference between the highest and the lowest value.
The *quartiles* Q_1, Q_2, Q_3 split the data into four parts: 25 % of the observations have a value less than the lower quartile Q_1, 50 % below the median Q_2, and 75 % below the upper quartile Q_3.

The *interquartile range*: $\mathrm{IQR} = Q_3 - Q_1$.
An *outlier* is an extreme value that lies outside the overall pattern of the data. The commonly used rule: an outlier is any value with

$$x > Q_3 + 1.5 \cdot \mathrm{IQR} \qquad \text{or} \qquad x < Q_1 - 1.5 \cdot \mathrm{IQR}.$$

Computing the quartiles for n values sorted in ascending order (discrete data): for Q_1 divide n by 4 — when $n/4$ is a whole number, take the midpoint of the corresponding term and the term above; when it is not, round up and pick the corresponding term. For Q_3 proceed analogously with $3n/4$. For continuous data, interpolate instead of rounding.

Example — box plot and outliers

The blood glucose of 30 females (mmol/l), stem-and-leaf diagram (key: 2 | 1 means 2.1):

```
2 | 2 2 3 3 5 7                                   (6)
3 | 1 2 6 7 7 7 8 8 8 8 9 9 9                     (13)
4 | 0 0 0 0 4 5 6 7 8                             (9)
5 | 1 5                                           (2)
```

$n = 30$, $30/4 = 7.5 \rightarrow$ round up to 8, the 8th number is 3.2, hence $Q_1 = 3.2$. The median is $Q_2 = 3.8$. For Q_3: $3 \cdot 30/4 = 22.5 \rightarrow 23$, the 23rd number is 4.0, hence $Q_3 = 4.0$.
$\mathrm{IQR} = 4.0 - 3.2 = 0.8$; lower limit $= 3.2 - 1.5 \cdot 0.8 = 2.0$; upper limit $= 4.0 + 1.5 \cdot 0.8 = 5.2$.
The outlier is the value 5.5. The highest value which is not an outlier is 5.1 — in the box plot a whisker leads to it, and the outlier is marked with a cross.

The multiplier 1.5 is a convention; with a multiplier of 1 we get a stricter criterion. For male turtles with $Q_1 = 400$ g, $Q_3 = 580$ g we have $\mathrm{IQR} = 180$ and the limits $400 - 180 = 220$ and $580 + 180 = 760$ — none of the values 400, 260, 550, 640 g is an outlier, and the largest weight without being an outlier is 760 g. For females with $Q_1 = 260$, $Q_3 = 340$, $\mathrm{IQR} = 80$ the limits are 180 and 420, so the outliers are the values 170 g and 440 g.

2.4 Data integration

[♦ beyond the slides] In *data integration* we merge several sources into one table. The main problems: the **entity identification problem** (the attribute `cust_id` in one database corresponds to `customer_number` in another; metadata and record linkage are needed); **redundancy** (an attribute derivable from others — annual income = 12×monthly; detected by correlation analysis for numeric and by the χ^2 test for categorical attributes, see Section 2.8); **value conflicts** (different units, scales, encodings); **duplicates** (the same object several times with slightly different values).

2.5 Transformation and standardisation of attributes

In Euclidean space, standardisation of attributes is recommended so that all attributes can have an *equal impact* on the computation of distances. For the points $x_i = (0.1; 20)$ and $x_j = (0.9; 720)$ we get

$$\mathrm{dist}(x_i, x_j) = \sqrt{(0.9 - 0.1)^2 + (720 - 20)^2} = 700.000457,$$

so the distance is entirely dominated by the difference $720 - 20 = 700$ in the second attribute; the first attribute has practically no influence. Standardisation forces the attributes into a common value range.

Basic standardisation transformations

Let f be an attribute and x_{if} its value for the i-th object.
Decimal scaling to $[-1, 1]$: divide the values by the smallest power of 10 that keeps all the transformed values within $[-1, 1]$.

Min-max normalisation (range standardisation) to $[0, 1]$:

$$\text{range}(x_{if}) = \frac{x_{if} - \min(f)}{\max(f) - \min(f)},$$

and to $[-1, 1]$ then $\text{range}_{[-1,1]}(x_{if}) = 2 \cdot \text{range}(x_{if}) - 1$.
Standardisation according to the mean absolute difference — zero mean and unit mean absolute difference:

$$m_f = \tfrac{1}{n}\big(x_{1f} + \cdots + x_{nf}\big), \qquad s_f = \tfrac{1}{n}\big(|x_{1f} - m_f| + \cdots + |x_{nf} - m_f|\big), \qquad z(x_{if}) = \frac{x_{if} - m_f}{s_f}.$$

The mean absolute difference is *robust* to outliers (the deviations are not squared), so outliers compress the remaining data less.

2.6 Discretisation of numeric attributes

A number of methods (association rules, ID3, Bayesian classification with discrete attributes) require categorical input. *Discretisation* divides the range of a numeric attribute into intervals. The methods differ in four respects: the *strategy* (top-down division of intervals, or bottom-up joining of intervals), the *number* of intervals, the *type* of intervals, and the *criterion* of interval quality (minimum classification error, entropy, the χ^2 test).

- **Equi-width** — the range $[\min(f), \max(f)]$ is divided into a preset number k of intervals of the same length $(\max(f) - \min(f))/k$. The simplest (it needs neither the classes nor sorting), but sensitive to outliers and to a skewed distribution: a single outlier stretches the range and most objects then end up in a single interval.
- **Equi-depth** [♦ beyond the slides] — the values are sorted in ascending order and a boundary is placed after every n/k objects, so each interval contains the same number of objects. Robust to outliers, but the boundaries need not respect natural clusters of the values.
- **Using the class information** — the boundaries are chosen so that the intervals are as pure as possible with respect to the target class. *Top-down* (splitting): a candidate boundary is every place between adjacent sorted values where the class changes; the boundary with the smallest weighted sum of entropies of the resulting parts is selected and the splitting recurses as long as it brings an improvement (the same way C4.5 chooses the threshold of a numeric attribute, Chapter 3). *Bottom-up* (merging): initially every value has its own interval and the adjacent pair with the most similar class distributions (the smallest χ^2) is merged repeatedly, until the χ^2 of every adjacent pair exceeds a chosen significance threshold.

2.6.1 Fuzzy discretisation

Sharp boundaries have an unpleasant consequence: two nearly identical values just on the two sides of a boundary fall into different categories. *Fuzzy discretisation* blurs the boundaries — the characteristic function μ of an interval does not drop from 1 to 0 in a step at the boundary, but linearly over a transition zone. Suitable intervals are found by the procedure **Interval**: the attribute values are sorted in ascending order and maximal sequences of values with the same class are concatenated into intervals with $\mu = 1$; stretches with mixed classes (and the gaps between the intervals) receive the class "?". Every interval Int_i of class "?" is then resolved according to its neighbours: if Int_{i-1} and Int_{i+1} belong to the same class, all three are joined into one crisp interval with $\mu = 1$; otherwise *two overlapping fuzzy intervals* $Int_{i-1} \cup Int_i$ and $Int_i \cup Int_{i+1}$ arise — each has $\mu = 1$ on its original crisp part, and over the "?" zone (between the upper limit x_{i-1} of Int_{i-1} and the lower limit x_{i+1} of Int_{i+1}) the characteristic functions decrease and increase linearly, respectively:

$$\mu_{i-1}(x) = \frac{x_{i+1} - x}{x_{i+1} - x_{i-1}}, \qquad \mu_{i+1}(x) = \frac{x - x_{i-1}}{x_{i+1} - x_{i-1}}, \qquad x \in [x_{i-1}, x_{i+1}].$$

One numeric value may thus correspond to *two* adjacent categories whose characteristic functions sum to 1. An object with a value in the transition zone enters the data as a "fuzzy object" with a weight given by the product of the characteristic functions of all attributes,

$$w = \prod_j \mu_j < 1.$$

Discretisation in general may lose information and introduce *contradictions* — objects with the same discretised input values that belong to different classes.

2.6.2 Grouping the values of categorical attributes (KEX)

A categorical attribute with too many values is handled by grouping its values; the goal is to form groups Grp for which $P(\text{Class} \mid A(\text{Grp}))$ differs significantly from the a-priori $P(\text{Class})$. The algorithm: *(1)* build an ordered list of the attribute values; *(2)* for each value, compute the class frequencies among the objects with this value and let the procedure **Assign** give it a code — if all such objects belong to the same class, the code of this class; if the class distribution for this value differs *significantly* (by the χ^2 test) from the a-priori one, the code of the most frequent class; otherwise the code "?"; *(3)* the procedure **Group** joins the values with the same code into groups — at most as many groups arise as there are classes, plus one ("?").

2.7 Data reduction

2.7.1 Too many objects

Batch processing becomes problematic. The remedies: *(a)* work only with a **sample** of objects; *(b)* store the data in a way that allows access without having to keep it all in main memory; *(c)* build **several models over subsets** and then combine them (ensembles, Chapter 3).

The key issue is the **representativeness** of the selected data — the sample should capture the nature of the data as well as possible, in particular as regards the correspondence of the attribute value distributions on the sample and on the entire data.

Unbalanced classes in the original data lead to a tendency to prefer the majority class. The remedies: different *weights* for different types of misclassification (cost-sensitive learning) or selecting objects from different classes with *different probabilities* (oversampling the minority, undersampling the majority).

2.7.2 Too many attributes

Two fundamentally different routes:

- **Reduction through transformation** — from the existing attributes we create *fewer new* ones, typically as linear combinations of the original ones (principal component analysis, PCA). Drawbacks: the values of *all* the original attributes have to be known, and the new attributes may lack a clear interpretation.
- **Reduction through selection** (*feature selection*) — from the existing attributes we choose only the most important ones. The interpretation is preserved and the unnecessary attributes need not be measured at all.

[♦ beyond the slides] PCA seeks orthogonal directions of maximal variance (the eigenvectors of the covariance matrix with the largest eigenvalues); it is an *unsupervised* transformation, so it may discard a direction that is crucial for prediction but has small variance.

2.8 Attribute relevance analysis: filters and wrappers

We are looking for those attributes that best contribute to a proper classification of objects.

Filters vs. wrappers

- **Filter** — for each attribute compute a characteristic of its contribution to classification, order, select the best. *Independent* of the model, fast.
- **Wrapper** — build a model over a *subset* of the attributes and evaluate it; use the best subset. Expensive, but takes the interaction of the attributes *with the concrete* model into account.
- Searching the subsets: *bottom-up* (forward selection — adding attributes) vs. *top-down* (backward elimination — removing them).

Limitation of filters: each attribute is evaluated *separately* ⇒ attribute interactions are hard to capture (a criterion for whole sets runs into combinatorial explosion). [♦ beyond the slides] A filter thus lets through a pair of strongly correlated attributes and discards an attribute useful only *in combination* with another (an XOR relation).

2.8.1 Criteria based on contingency tables

The relation of an input attribute A (values $v_1, \ldots, v_R$) and a target attribute C (classes $v_1, \ldots, v_S$) is described by a contingency table with the frequencies a_{kl} of the combination $A(v_k) \wedge C(v_l)$, with the marginals

$$r_k = \sum_{l=1}^{S} a_{kl}, \qquad s_l = \sum_{k=1}^{R} a_{kl}, \qquad n = \sum_{k=1}^{R}\sum_{l=1}^{S} a_{kl}.$$

Probability estimates: $P(A(v_k) \wedge C(v_l)) = a_{kl}/n$, $P(A(v_k)) = r_k/n$, $P(C(v_l)) = s_l/n$. The expected frequency under independence is $e_{kl} = r_k s_l / n$.

χ^2 as a relevance criterion

$$\chi^2 = \sum_{k=1}^{R}\sum_{l=1}^{S} \frac{(a_{kl} - e_{kl})^2}{e_{kl}} = n \sum_{k=1}^{R}\sum_{l=1}^{S} \frac{\left(a_{kl} - \frac{r_k s_l}{n}\right)^2}{r_k s_l} \qquad \text{(the higher the value, the better).}$$

An alternative form convenient for hand computation: $\chi^2 = \sum_k \sum_l \frac{a_{kl}^2}{e_{kl}} - n$.
As a statistical test: with the validity of the null hypothesis of independence

$$H_0: \; P(A = v_k \wedge C = v_l) = P(A = v_k)\, P(C = v_l) \quad \forall k, l$$

the χ^2 statistic has a distribution with $(R-1)(S-1)$ degrees of freedom. If $\chi^2 \geq \chi^2_{(R-1)(S-1)}(\alpha)$, the null hypothesis is rejected at the chosen level of significance α and an alternative hypothesis of *dependence* is accepted.

Example — χ^2 test on a four-field table

		Provision of credit YES	NO	r_k
Income height	high	50	0	50
	low	30	40	70
s_l		80	40	120

Expected frequencies: $e_{11} = 50 \cdot 80/120 = 33.3$; $e_{12} = 50 \cdot 40/120 = 16.7$; $e_{21} = 70 \cdot 80/120 = 46.7$; $e_{22} = 70 \cdot 40/120 = 23.3$. The differences are therefore $+16.7$, -16.7, -16.7, $+16.7$ and the value of the statistic is $\chi^2 = 42.857$.
At the significance level $\alpha = 0.05$ the critical value is $\chi^2_{(1)}(0.05) = 3.84$. Since $42.857 > 3.84$, **there is a dependence between the income height and the provision of credit**.

Fisher's exact test

The χ^2 test can be used only for sufficiently large frequencies — it must hold that $(r_k s_l)/n \geq 5$ for all k, l. For four-field tables with low frequencies *Fisher's test* is used. The probability that a four-field table has, for the given marginals r_1, r_2, s_1, s_2, exactly the frequencies a_{kl}:

$$p = \frac{r_1!\, r_2!\, s_1!\, s_2!}{n!\, a_{11}!\, a_{12}!\, a_{21}!\, a_{22}!} = \frac{\binom{s_1}{a_{11}}\binom{s_2}{a_{12}}}{\binom{n}{r_1}}.$$

The probabilities are summed over all considered values of the actual frequencies at the given marginals (let $m = \min(r_1, s_1)$):

$$P = \sum_{i=0}^{m} \frac{r_1!\, r_2!\, s_1!\, s_2!}{n!\, (a_{11}-i)!\, (a_{12}+i)!\, (a_{21}+i)!\, (a_{22}-i)!}.$$

If $P \leq \alpha$, the null hypothesis is rejected at the level of significance α.
Example. For a table with the marginals $r = (12, 18)$, $s = (6, 24)$, $n = 30$ we get $P_0 = 0.031$ and $P_1 = 0.173$, hence $P = P_0 + P_1 = 0.204 > 0.05$ — **the null hypothesis of independence is accepted**. (Check: the sum $P_0, \ldots, P_6 = 0.031{+}0.173{+}0.340{+}0.302{+}0.128{+}0.024{+}0.002 = 1.000$.)

2.8.2 Criteria based on entropy

Entropy, mutual information, information measure of dependence

The **entropy** of an attribute A as the weighted average of the entropies of its values (the *smaller*, the better):

$$H(A) = \sum_{k=1}^{R} \frac{r_k}{n} H\big(A(v_k)\big), \qquad H\big(A(v_k)\big) = -\sum_{l=1}^{S} \frac{a_{kl}}{r_k} \log_2 \frac{a_{kl}}{r_k}.$$

Mutual information:

$$\mathrm{MI}(A, C) = \sum_{k=1}^{R} \sum_{l=1}^{S} \frac{a_{kl}}{n} \log_2 \frac{n\, a_{kl}}{r_k s_l}.$$

The **information measure of dependence** — normalised mutual information (the *bigger*, the better):

$$\mathrm{ID}(A, C) = \frac{\mathrm{MI}(A, C)}{H(C)} = \frac{\mathrm{MI}(A, C)}{-\sum_{l=1}^{S} \frac{s_l}{n} \log_2 \frac{s_l}{n}}.$$

The normalisation in ID is essential: MI alone systematically favours attributes with many values (in the limit, the object identifier is the "most informative" attribute). The same bias is addressed by the information gain ratio in C4.5 (Chapter 3).

2.8.3 Criteria for numeric attributes: correlation

The criteria above (χ^2, entropy) work with *categorical* attributes over a contingency table. For a pair of **numeric** attributes, *correlation analysis* is used instead — and it serves two purposes: measuring the **relevance** of an input with respect to a numeric output, and detecting **redundancy** between a pair of inputs (see Sect. 2.4).

Pearson correlation coefficient

A measure of the **linear** relationship between two numeric quantities; it is the sample covariance normalised by the standard deviations:

$$r = \frac{\operatorname{cov}(X,Y)}{s_X\, s_Y} = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2\ \sum_{i=1}^{n}(y_i-\bar{y})^2}} \in [-1,1].$$

$r = \pm 1$ means the points lie *exactly* on a line (with a positive, resp. negative slope); $r = 0$ means there is no *linear* relationship between them. The square r^2 (the *coefficient of determination*) gives the proportion of the variance of Y explained by the linear dependence on X.
Relation to regression (Sect. 2.9): the slope is $\beta = \operatorname{cov}(X,Y)/s_X^2$, hence $r = \beta\, s_X/s_Y$ — the same information, merely scaled to be dimensionless.

Three pitfalls of the Pearson coefficient

(1) It measures only a linear relationship. For data distributed symmetrically around zero and $y = x^2$ we get $r = 0$, even though Y depends on X *functionally*. Zero correlation therefore **does not imply independence** (the converse does hold: independence $\Rightarrow r = 0$).
(2) It is sensitive to outliers. A single extreme point can drive r towards one even where the rest of the data shows no trend at all.
(3) Correlation is not causation. The relationship may be produced by a hidden third variable; distinguishing them requires an experiment or a temporal ordering (cf. influence versus correlation in Chapter 6).

Spearman rank correlation coefficient ρ

It is the **Pearson coefficient computed from the ranks** instead of the values. If there are no tied ranks in the data, it simplifies to

$$\rho = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n\,(n^2-1)} \in [-1,1],$$

where d_i is the difference of the ranks of the i-th object in X and in Y.
What this buys you:

- It captures *any* ***monotone*** relationship, not merely a linear one.
- It is **robust to outliers** — ranking compresses the extremes.
- It is **invariant under monotone transformations** (log, $\sqrt{\ }$, ...); Pearson is not.
- It also works for **ordinal** attributes, whose values can be ordered but not subtracted (grades, satisfaction levels).

A numerical example. Five objects, $x = (1,2,3,4,5)$ and $y = (1,2,4,8,16)$ — the relationship is *perfectly monotone*, yet exponential, i.e. non-linear. The ranks of both variables coincide, so all $d_i = 0$ and $\rho = 1$ (a perfect match). Pearson, by contrast, gives $r \doteq 0{,}93$: it *penalises* the non-linearity. Had we taken the logarithm of y ($\log_2 y = 0,1,2,3,4$), Pearson would give $r = 1$ as well — which is exactly the invariance at work: Spearman returns 1 straight away, Pearson only after the right transformation.

Which measure to choose. Pearson where the relationship is approximately linear and the data are free of outliers (and where we want to read r^2 as the explained variance); Spearman where only the *strength and direction of the trend* matter, where the data contain extremes, or where the attributes are merely ordinal. [♦ beyond the slides] A third common variant is *Kendall's* τ, which counts the proportion of concordant and discordant pairs instead of rank differences.

Use in data reduction. A *correlation matrix* of all pairs of input attributes is built; pairs with $|r|$ close to 1 carry practically the same information and one of them can be removed. Mind, however, the caveat from Sect. 2.8: pairwise correlation alone cannot recognise an attribute that is useful only

in combination with another.

2.9 Relations between attributes: regression

Whereas contingency tables describe the relation of *categorical* attributes, for numeric ones regression analysis is used: *linear regression* determines the *parameters* of a linear relationship between two numerical quantities.

Reasons for regression analysis: *(1)* it is very costly to measure the output ⇒ predict it from inputs that are easily available; *(2)* the input values are known earlier than the output ⇒ estimate the output; *(3)* controlled input values might help find the correct behaviour of the output; *(4)* there can be a causal relationship between input and output which we want to find.

Linear regression for one input variable by least squares

Patterns $[x_1, y_1], \ldots, [x_p, y_p]$, regression equation $y = \beta x + \alpha$, quadratic error

$$E = \sum_{i=1}^{p} \varepsilon_i^2 = \sum_{i=1}^{p} \big(y_i - \hat{y}_i\big)^2 = \sum_{i=1}^{p} \big(y_i - \beta x_i - \alpha\big)^2.$$

Setting the partial derivatives to zero:

$$\frac{\partial E}{\partial \alpha} = -2 \sum_i \big(y_i - \beta x_i - \alpha\big) = 0, \qquad \frac{\partial E}{\partial \beta} = -2 \sum_i \big(y_i - \beta x_i - \alpha\big) x_i = 0,$$

whence, after rearranging, $\alpha = \bar{y} - \beta \bar{x}$ and

$$\beta = \frac{\sum_{i=1}^{p} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{p} (x_i - \bar{x})^2}$$ (i.e. the sample covariance divided by the sample variance in x).

Prediction: $\hat{y} = \beta x + \alpha$.

Multi-dimensional linear regression. We assume a linear dependence of the explained quantity Y on several explaining quantities $X_1, \ldots, X_m$: $\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m$, in matrix notation $\hat{Y} = X\beta$, where X is the extended matrix of input patterns (the first column being ones) and $\beta = (\beta_0, \ldots, \beta_m)$. The quadratic deviation is $E = (Y - X\beta)^\top (Y - X\beta)$; from $\partial E / \partial \beta = 0$ we obtain the normal equations $X^\top X \beta = X^\top Y$, hence

$$\beta = \big(X^\top X\big)^{-1} X^\top Y.$$

For complex practical tasks this computation is expensive ⇒ approximative (iterative) solutions are used.

Non-linear regression assumes a more complex functional dependence (quadratic, exponential, ...). Its most important case is *logistic regression*, which models a categorical (two-valued) output — since it is at the same time a classifier, it is treated in Chapter 3, Section 3.2.5.

2.10 Summary for the examination

- **The goal of data preparation**: select the relevant data, represent it for the chosen algorithm, the result being *one* table of objects × attributes.
- **Missing values**: ignore the object / a new value "I don't know" / an existing value (mode, proportionally, arbitrarily) / impute with a model. Be able to say when each choice is appropriate (absence as information → "I don't know").
- **Outliers**: IQR $= Q_3 - Q_1$, an outlier lies outside $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$. Be able to compute the quartiles from ascending data (the $n/4$ and $3n/4$ rule, rounding up) and to draw a box plot.

- **Standardisation**: decimal scaling, min-max to $[0, 1]$ and $[-1, 1]$, z-score by the mean absolute difference (robust to outliers). The motivation: without it the distance is dominated by the attribute with the largest range.
- **Discretisation**: equi-width (same length), equi-depth (same number of objects), using the class (entropy top-down by splitting, χ^2 bottom-up by merging); **fuzzy discretisation** — the procedure Interval, blurred boundaries, two categories per value with characteristic functions summing to 1, the weight of an object is $\prod_j \mu_j$. **KEX** groups the values of a categorical attribute (Assign: a class code by the χ^2 test against the a-priori class distribution, otherwise "?"; Group: values with the same code).
- **Data reduction**: too many objects → sampling (representativeness!), efficient storage, several models over subsets; unbalanced classes → error weights or different selection probabilities. Too many attributes → *transformation* (PCA: fewer new attributes as linear combinations, but all the original ones must be known and the interpretation is lost) vs. *selection* (a subset of the original ones, the interpretation is preserved).
- **Filters vs. wrappers**: a filter computes a characteristic for each attribute separately and is independent of the model (fast, but blind to interactions); a wrapper builds a model over a subset and evaluates it (expensive, takes interactions into account). Search *bottom-up* (adding) or *top-down* (removing).
- **Filter criteria**: χ^2 (bigger = better), entropy $H(A)$ (smaller = better), $\mathrm{MI}(A, C)$, $\mathrm{ID}(A, C) = \mathrm{MI}/H(C)$ (bigger = better). Be able to justify the normalisation in ID: without it, attributes with many values are favoured.
- **The χ^2 test**: $(R-1)(S-1)$ degrees of freedom, the condition $r_k s_l / n \geq 5$; for low frequencies **Fisher's exact test**. Be able to go through the computation on a four-field table (the credit example: $\chi^2 = 42.857 > 3.84 \Rightarrow$ dependence).
- **Regression**: the least squares method, β as covariance divided by variance, $\alpha = \bar{y} - \beta\bar{x}$; multidimensionally $\beta = (X^\top X)^{-1} X^\top Y$.

3 Methods for data mining

The most extensive sentence of the topic. It names three groups of methods that cover three different situations: *association rules* for transactional data without a target attribute, *supervised learning* for data with a target attribute, and *cluster analysis* for metric data without a target attribute. The chapter is divided into three sections accordingly.

3.1 Association rules and frequent patterns

3.1.1 Market basket analysis

Market basket analysis (MBA) is the archetypal task: "which items occur together in the basket?" The results are expressed in the form of rules and can be applied immediately — in planning and the layout of shops, in offering coupons and time-limited discounts, or in "packaging" products.

An **item** is a product or service on offer, a **transaction** contains one or more items. The **frequency table** indicates the number of co-occurrences of any two items within the transactions; the values on its diagonal correspond to the number of transactions that contain the respective item.

Example — frequency table

Transactions in a grocery shop:

Cust.	Items
1	bread, butter
2	milk, bread, butter
3	bread, coffee
4	bread, butter, coffee
5	coffee, butter

	bread	butter	milk	coffee
bread	4	3	1	2
butter	3	4	1	2
milk	1	1	1	0
coffee	2	2	0	3

The character of the sales is apparent from the frequency table at once: *bread and butter tend to be bought together*, *milk is never purchased together with coffee*.

The rules have the form `IF` *condition* `THEN` *conclusion*. Association rules should be:

- **easy to understand** — once a relationship is found, it can be easily verified;
- **applicable** — they comprise useful information that can lead to further interventions;

and conversely they should *not* be **trivial** (everyone already knows the result) nor **inexplicable** (there is no explanation for them and they do not lead to an action).

3.1.2 Support, confidence and lift

Support, confidence, lift

Support — how often can we use the rule:

$$\sup(R) = \frac{\text{number of transactions containing } i \text{ and } j}{\text{number of all transactions}} \cdot 100\,\%.$$

Confidence — how reliable are the results of the rule:

$$\text{conf}(R) = \frac{\text{number of transactions containing } i \text{ and } j}{\text{number of transactions containing } i} \cdot 100\,\%.$$

Lift — how many times is it better to use the rule for prediction instead of just assuming its result:

$$\text{lift}(R) = \frac{p(i \wedge j)}{p(i)\,p(j)}.$$

If lift < 1, the rule is *worse than a random choice* for prediction and the **negation of the conclusion** may yield a better rule (`IF` *condition* `THEN NOT` *conclusion*).

Example — support, confidence, lift

Over the data of the previous example (5 transactions):
Rule 1: "if the customer buys bread then he buys also butter." sup $= 3/5 = 60\,\%$, conf $= 3/4 = 75\,\%$.
Rule 2: "if the customer buys coffee then he buys also butter." sup $= 2/5 = 40\,\%$, conf $= 2/3 = 66\,\%$.
Rule 3: "if the customer buys milk then he buys also butter." sup $= 1/5 = 20\,\%$, conf $= 1/1 = 100\,\%$, lift $= \frac{1/5}{(1/5)\cdot(4/5)} = \frac{5}{4} = 1.25$.
Rule 3 has *one hundred percent* confidence but a support of only 20 % — it rests on a single transaction. The lift of 1.25 says that the gain over chance is only small (80 % of the customers buy butter anyway). This is the basic trap: *high confidence alone is not enough.*

3.1.3 Choice of items, taxonomies and virtual items

Transaction data is often of worse quality and requires a more extensive preprocessing; moreover the relevance of the items changes with time. The adequate level of processing is a compromise between a growing number of combinations and the immediate applicability of the results (specific items), while keeping a sufficiently high support.

A **taxonomy** is a hierarchy of categories. In the beginning more general items are used; later, rules are generated for specific items, and only from the transactions that contain them. For the results to be applicable, the items considered should occur in roughly the same number of transactions: *rare items are moved to higher levels* of the taxonomy (where they appear relatively more frequently) and *the more common items are left at lower levels* (in order not to allow the most frequent items to dominate the rules).

Virtual items exceed the framework of a taxonomy: brands across product types (Calvin Klein) or information about the transaction itself (day, time, customer). The risk is *redundancy* in the rules (a virtual item corresponds to exactly one taxonomy item, or meets the general item in one rule). A typical use is the *comparison of shops*: a virtual item marks the type of the shop, MBA is applied to comparably sized data from the new and the existing shops, and one watches the rules in which the virtual items appear.

Pruning according to minimum support eliminates the less frequent items — the actions undertaken should concern enough transactions. Two options: eliminate the rare items (and subsequently the corresponding rules), or use a taxonomy to create general items (which must meet the set threshold). A variable minimum support leads to a *cascade effect*.

3.1.4 The Apriori algorithm

Frequent itemset and the Apriori principle

A *frequent itemset* is a combination (conjunction) of categories that reaches a preset frequency on the data — the support *minsup*.
The **downward closure property** — the key idea for Apriori: *any subset of a frequent itemset is also a frequent itemset*. Equivalently: if an itemset is infrequent, so are all its supersets.
When looking for combinations of length k we therefore use the known combinations of length $k-1$ — a *breadth-first* generation. To generate a combination of length k we demand that *all* its subcombinations of length $k-1$ fulfil the frequency requirement.

Algorithm APRIORI (R. Agrawal)

1: $L_1 \leftarrow$ all categories that reach at least the required frequency
2: $k \leftarrow 2$
3: **while** $L_{k-1} \neq \emptyset$ **do**
4: $\quad C_k \leftarrow$ APRIORI-GEN(L_{k-1}) ▷ candidate generation
5: $\quad L_k \leftarrow$ those combinations from C_k that reached at least the required frequency
6: $\quad k \leftarrow k+1$
7: **end while**

The function APRIORI-GEN(L_{k-1}):

1: **for** all pairs of combinations $Comb_p, Comb_q \in L_{k-1}$ **do**
2: $\quad$ **if** $Comb_p$ and $Comb_q$ match in $k-2$ categories **then**
3: $\quad\quad$ add $Comb_p \cup Comb_q$ to C_k ▷ the join step
4: $\quad$ **end if**
5: **end for**
6: **for** each combination $Comb \in C_k$ **do** ▷ the pruning step
7: $\quad$ **if** any of its subcombinations of length $k-1$ is not contained in L_{k-1} **then**
8: $\quad\quad$ remove $Comb$ from C_k
9: $\quad$ **end if**
10: **end for**

Rule generation. After finding the combinations with a satisfactory frequency, the association rules are created (note that *supercombination frequency* $\leq$ *combination frequency*). Each combination *Comb* is divided into all possible pairs of subcombinations *Ant* and *Suc* such that

$$Suc = Comb \setminus Ant, \qquad Ant \cap Suc = \emptyset, \quad Ant \cup Suc = Comb.$$

Then, for the rule $Ant \Rightarrow Suc$, the *support* corresponds to the frequency of the combination *Comb* and the *confidence* to the ratio of the frequencies of the combinations *Comb* and *Ant*.

Advantages of MBA	**Drawbacks of MBA**
clear and understandable results — IF-THEN rules that are immediately applicable	computational costs: the space of association rules is exponential, $O(2^m)$ for m items
mining without target output values (without a-priori knowledge)	taxonomies and virtual items are necessary
can process data of a variable length	difficulty in determining an adequate number of items; the items should have about the same frequency
simple and easy-to-understand computation	*it tends to produce excessive numbers of rules* — thousands, tens of thousands, millions

3.1.5 Multiple minimum supports: MS-Apriori

The rare item problem: a single *minsup* does not fit frequent and rare items at the same time (bread vs. a food processor) — a high one misses rules with rare items, a low one explodes combi-

natorially on the frequent ones.

The main difference from Apriori

Each item has its *own minimum support* $\mathrm{MIS}(i)$ (*minimum item support*); **the minsup of a rule is the minimum of the MIS values of its items**. To prevent very frequent and very rare items from mixing in one itemset, the support difference is also bounded: $\max_{i \in s} \sup(i) - \min_{i \in s} \sup(i) \leq \varphi$.

Consequence: downward closure no longer holds. Counterexample: $\mathrm{MIS}(1) = 10\,\%$, $\mathrm{MIS}(2) = 20\,\%$, $\mathrm{MIS}(3) = 5\,\%$; the itemset $\{1, 2\}$ with support 9 % is infrequent, but $\{1, 2, 3\}$ could be frequent — $\{1, 2\}$ cannot be discarded. MS-Apriori copes with this by three modifications: *(a)* it sorts the items in *ascending order of MIS* and tests the frequency of an itemset against the MIS of its *first* item ($c.\mathrm{count}/n \geq \mathrm{MIS}(c[1])$); *(b)* it generates the level-2 candidates from the set of seeds L (items satisfying the MIS of some earlier item), not from F_1; *(c)* for rule generation it additionally counts `tailCount` — the support of the itemset without its first item — because the supports of the subsets are no longer guaranteed to be available. The single-minsup model is a special case (MIS = 100 % prohibits rules with the item).

3.1.6 Class association rules (CAR)

Usual mining finds all rules; sometimes only rules with a *target class* in the consequent are of interest (which words are associated with which topic of a document).

Class association rule

The transactions are additionally labelled with classes; a *class association rule* (CAR) is an implication $X \to y$, where $X \subseteq I$ (items) and $y \in Y$ (classes), $I \cap Y = \emptyset$. Support and confidence are defined the same way as for usual association rules. E.g. on a collection of documents: Student, School → Education [sup = 2/7, conf = 2/2].

CARs are mined **directly in one step** (by a modified Apriori) via *rule-items* $(condset, y)$ with a support above *minsup*. *Different minsups for different classes* can be specified (the multiple-supports idea); a class minsup of 101 % prohibits generating its rules.

3.1.7 Sequential pattern mining

Association rules ignore the *order* of the transactions; the order matters e.g. in MBA (a bed first, bed sheets later) or when looking for navigational patterns in web logs (*web usage mining*).

Sequence, size, length, subsequence

A *sequence* $s = \langle a_1 a_2 \ldots a_r \rangle$ is an ordered list of non-empty itemsets (*elements*). The *size* = the number of elements, the *length* = the number of items (k-sequence). $s_1 = \langle a_1 \ldots a_r \rangle$ is a *subsequence* of $s_2 = \langle b_1 \ldots b_v \rangle$ if there exist indices $1 \leq j_1 < \cdots < j_r \leq v$ such that $a_1 \subseteq b_{j_1}, \ldots, a_r \subseteq b_{j_r}$. Beware: $\langle \{3\}\{8\} \rangle$ and $\langle \{3, 8\} \rangle$ do *not* contain each other.

The task: find all sequences with a support (the fraction of the data sequences containing them) of at least *minsup* — the *sequential patterns*. The **GSP** algorithm (*Generalized Sequential Pattern*) works similarly to Apriori: breadth-first, candidates of length k by joining frequent sequences of length $k - 1$. Add-ons: a cause–effect analysis by introducing items for the transactions *before* and *after* an event; *dissociation rules* (IF A AND NOT B) via complementary items — at the cost of twice as many items.

3.1.8 FP-Growth and the FP-tree

Apriori uses a *generate-and-test* approach: candidate generation is expensive (both in space and time) and support counting is expensive too — subset checking is computationally expensive and the database is scanned *repeatedly* (I/O). **FP-Growth** allows frequent itemset discovery *without candidate generation* in two steps: *(1)* build a compact data structure, the *FP-tree* (using two passes over the data); *(2)* extract the frequent itemsets directly from the FP-tree by traversing it.

The FP-tree

The nodes of the FP-tree correspond to items and have a counter. FP-Growth reads one transaction at a time and maps it to a *path* in the tree. A *fixed order* of items is used, so paths can overlap when the transactions share a *prefix*; in that case the counters are incremented. Pointers are maintained between the nodes containing the same item, creating singly linked lists. The more paths overlap, the higher the compression, and the more likely the FP-tree fits in memory.

Construction (2 passes). *Pass 1:* scan the data and find the support of each item, discard the infrequent items, sort the frequent ones in *decreasing* order of support (e.g. a, b, c, d, e) — this order is used when building the tree so that common prefixes can be shared. *Pass 2:* construct the tree. E.g. transaction $\{a, b\}$ creates the path `null`$\rightarrow a \rightarrow b$ with counters 1; transaction $\{b, c, d\}$ creates a *disjoint* path `null`$\rightarrow b \rightarrow c \rightarrow d$ (it shares no common prefix with the first one), and a link is added between the two b nodes; transaction $\{a, c, d, e\}$ shares the common prefix a with the first one, so the paths overlap and the counter of node a is incremented.

The size of the FP-tree. Usually smaller than the uncompressed data — transactions typically share items, and hence prefixes. *Best case:* all transactions contain the same set of items $\Rightarrow$ one path. *Worst case:* every transaction has a unique set of items $\Rightarrow$ the tree is at least as large as the original data and the storage requirements are *higher* (the pointers and counters have to be stored). The size depends on how the items are ordered — decreasing support is only a *heuristic* and does not always lead to the smallest tree.

Extracting frequent itemsets from the FP-tree

The algorithm proceeds *bottom-up*, from the leaves towards the root, by *divide and conquer*: it first looks for frequent itemsets ending in e, then in de, ..., then d, cd, and so on.

1. The *prefix path sub-tree* ending in a given item is obtained using the linked lists.
2. Check whether the item is frequent — add the counts along the linked list. (In the example with $minSup = 2$ we get $\text{count}(e) = 3$, so $\{e\}$ is extracted as a frequent itemset.)
3. If the item is frequent, find recursively the frequent itemsets ending in de, ce, be, ae. For that, the *conditional FP-tree* must be obtained.

The **conditional FP-tree** for the item e is the tree that would be built if we only considered the *transactions containing* e and then removed e from all transactions. It is obtained from the prefix path sub-tree in three steps:

1. update the support counts along the prefix paths to reflect the number of transactions containing e;
2. remove the nodes containing e (the information about them is no longer needed);
3. remove the infrequent items from the prefix paths — e.g. if b has a support of 1 (which really means that be has a support of 1), then be is infrequent and b can be removed.

The conditional tree is then used to find the itemsets ending in de, ce, ae (be is not considered, because b is not in the conditional tree); for each of them the prefix paths are obtained again, frequent itemsets are extracted and a conditional tree is generated — recursively (e.g. $e \rightarrow de \rightarrow ade$).

Advantages of FP-Growth	Disadvantages of FP-Growth
only 2 passes over the data set	the FP-tree may not fit in memory
it "compresses" the data set	the FP-tree is expensive to build (but afterwards the frequent itemsets are read off easily)
no candidate generation, much faster than Apriori	pruning can only be done on single items; the support can be computed only after *all* the data has been added to the tree

[♦ beyond the slides] As a supplement: a *closed* frequent itemset is one no proper superset of which has the same support; a *maximal* one is one no proper superset of which is frequent. Maximal itemsets are the most compact representation (but they lose the information about the supports of the subsets), whereas closed itemsets are a compromise — a lossless representation of all supports.

3.2 Approaches based on the principle of supervised learning

3.2.1 Formulation of the task

Classification and prediction

A training set $\{(x_1, y_1), \ldots, (x_p, y_p)\}$ is given, where x_i is a vector of input attribute values and y_i the value of the *target* attribute. The task is to find a model $\hat{y} = f(x)$ that predicts the target attribute well also for previously unseen inputs.
If the target attribute is *discrete*, this is **classification**; if it is *continuous*, **regression** (prediction). The solution always has two phases: *(1)* **induction** (learning, training) of the model from the training data; *(2)* **application** of the model to new data.

It is essential that the model be evaluated on *new* data — the ability to memorise the training data is worthless (*overfitting*). The methods for error estimation and the quality measures are in Chapter 5, Section 5.2.

3.2.2 Decision trees

Decision tree

Let D be a database $\{d_1, \ldots, d_p\}$, $\{A_1, \ldots, A_n\}$ a set of attributes and $C = \{C_1, \ldots, C_m\}$ a set of class labels. A *decision tree* for D is a tree where:

- each inner node is labelled by an attribute A_a;
- each edge is labelled by a predicate (applicable to the attribute of the parent node);
- each leaf has a class label C_j.

The tree divides the feature space into *rectangular regions*; patterns are classified according to the region they belong to.

Advantages	Disadvantages
easy to implement and interpret	difficult to process continuous data (attribute categorisation needed → rectangular regions)
efficient	cannot always be used — the class boundary may be diagonal and an axis-parallel tree can only approximate it in steps (in the bank-loan example the boundary runs across the *account balance* × *income* plane)
extraction of simple rules	difficult to process missing data
applicable also to big data sets	over-fitting ⇒ *pruning* required
	mutual correlations among the attributes are not considered

Terminology: the *decision attributes* for the division of the data set label the decision nodes of the tree; the *decision predicates* label the edges; the *stopping criterion* is e.g. "all patterns from the reduced set belong to the same class".

The TDIDT algorithm (Top Down Induction of Decision Trees)

Induction of the tree top-down by *divide and conquer*:

1. Choose one attribute as the *root* of the partial (sub)tree.
2. Divide the data arriving at this node into subsets according to the values of the chosen attribute and add a node for each subset.
3. If there is a node such that the data patterns arriving at it do not all belong to the same class, repeat the process for it starting from step 1; else stop.

Entropy and the choice of the splitting attribute

The objective is to choose the attribute that would best divide the patterns from different classes. *Entropy* as a measure of disorder ("surprise") in the system:

$$H = -\sum_{j=1}^{m} p_j \log_2 p_j,$$

where p_j is the probability of occurrence of class j (its relative frequency on the set of patterns considered) and m the number of classes. Convention: $0 \log_2 0 = 0$; conversion $\log_2 x = \log_{10} x / 0.301$.
For an attribute A: for each of its possible values v compute the entropy $H(A(v))$ on the set of examples covered by the category $A(v)$,

$$H\big(A(v)\big) = -\sum_{j=1}^{m} p_j^{A(v)} \log_2 p_j^{A(v)},$$

and the *weighted average entropy* of the attribute as the weighted sum

$$H(A) = \sum_{v} \frac{|A(v)|}{|D|} H\big(A(v)\big).$$

The attribute with the *minimum* $H(A)$ is chosen.

Example — an application for a loan

Client	Income	Account	Gender	Unemployed	Loan approved
C1	high	high	female	no	yes
C2	high	high	male	no	yes
C3	low	low	male	no	no
C4	low	high	female	yes	yes
C5	low	high	male	yes	yes
C6	low	low	female	yes	no
C7	high	low	male	no	yes
C8	high	low	female	yes	yes
C9	low	middle	male	yes	no
C10	high	middle	female	no	yes
C11	low	middle	female	yes	no
C12	low	middle	male	no	yes

Step 1 — choosing the attribute by minimum entropy. The four-field table for *income* and *loan approved*: high income 5 yes / 0 no, low income 3 yes / 4 no.

$$\begin{aligned} H(\text{income(high)}) &= -\tfrac{5}{5}\log_2\tfrac{5}{5} - \tfrac{0}{5}\log_2\tfrac{0}{5} = 0, \\ H(\text{income(low)}) &= -\tfrac{3}{7}\log_2\tfrac{3}{7} - \tfrac{4}{7}\log_2\tfrac{4}{7} = 0.9852, \\ H(\text{income}) &= \tfrac{5}{12}\cdot 0 + \tfrac{7}{12}\cdot 0.9852 = 0.5747. \end{aligned}$$

Analogously $H(\text{account}) = \frac{1}{3}\cdot 0 + \frac{1}{3}\cdot 1 + \frac{1}{3}\cdot 1 = 0.6667$, $H(\text{gender}) = \frac{1}{2}\cdot 0.9183 + \frac{1}{2}\cdot 0.9183 = 0.9183$ and $H(\text{unemployed}) = \frac{1}{2}\cdot 1 + \frac{1}{2}\cdot 0.65 = 0.825$. The minimum belongs to **income** $\Rightarrow$ the first branching.
Step 2. Income(high): loan approved(yes) for all patterns $\rightarrow$ a leaf. Income(low): 7 clients, further branching; $H(\text{account}) = \frac{2}{7}H(\dots) + \frac{3}{7}H(\dots) + \frac{2}{7}H(\dots) = 0.3935$, $H(\text{gender}) = 0.965$, $H(\text{unemployed}) = 0.9792$ $\Rightarrow$ branch on **account**.
Step 3. Account(high): yes for all; account(low): no for all; account(middle): 3 clients, $H(\text{gender}) = \frac{2}{3}\cdot 1 + \frac{1}{3}\cdot 0 = 0.6667$, $H(\text{unemployed}) = 0$ $\Rightarrow$ the third branching on **unemployed**.
The resulting tree, transformed into rules:

`IF` income(high) `THEN` loan approved(yes)

`IF` income(low) $\wedge$ account(high) `THEN` loan approved(yes)

`IF` income(low) $\wedge$ account(low) `THEN` loan approved(no)

`IF` income(low) $\wedge$ account(middle) $\wedge$ unempl.(yes) `THEN` loan approved(no)

`IF` income(low) $\wedge$ account(middle) $\wedge$ unempl.(no) `THEN` loan approved(yes)

Transformation of a tree into rules in general: each path between the root and a leaf corresponds to one rule; the non-leaf nodes (together with their value for the respective edge) appear on the left-hand side (in the body) of the rule, and the leaf node (the target class) represents the right-hand side (the head).

Factors important for the induction of a decision tree:

- *the choice of the decision attributes* — it has an impact on the efficiency of the built tree; an "informed" expert entry for the area considered is useful;
- *the order of the decision attributes* — it eliminates redundant comparisons;
- *branching* — the number of necessary divisions; more demanding for continuous values or many values;
- *the form of the built tree* — balanced trees with as few levels as possible are more suitable (at the cost of more complex comparisons); some algorithms create binary trees only (often quite deep);
- *stopping criteria* — correct classification of the training data; *early stopping* prevents building large trees and overtraining (accuracy vs. efficiency); *Occam's razor* (William of Occam, 1320) — preference for the simplest hypothesis for D;
- *the training data and its quantity* — too little $\Rightarrow$ the tree might not be reliable enough for more general data; too much $\Rightarrow$ the danger of overtraining;
- *pruning* — a higher accuracy and efficiency for the tested data; removal of redundant comparisons, of entire subtrees and of irrelevant attributes; it can be done already during tree induction (which avoids forming excessively large trees), or on the already formed tree;
- *time and space complexity* — depends on the number of training data p, the number of attributes n and the form of the tree. Tree induction: $O(n\,p\log p)$; classification of q patterns with a tree of depth $O(\log p)$: $O(q\log p)$.

The ID3 algorithm (Iterative Dichotomiser, Ross Quinlan, 1986): training of functions with Boolean values, a *greedy* method, builds the tree top-down; in each node it chooses the attribute that best classifies the local training data — the best attribute has the *biggest information gain*. The process continues until all the training data is classified correctly, or until all the attributes have been used.

The ID3 algorithm and information gain

ID3(*EXAMPLES*, *TARGET_ATTRIBUTE*, *ATTRIBUTES*):

1: build a root *ROOT* of the tree

2: **if** all *EXAMPLES* are positive **then return** *ROOT* with a single node labelled "+"
3: **end if**
4: **if** all *EXAMPLES* are negative **then return** *ROOT* with a single node labelled "−"
5: **end if**
6: **if** $ATTRIBUTES = \emptyset$ **then return** *ROOT* with a single node labelled by the most frequent value of *TARGET_ATTRIBUTE* in *EXAMPLES*
7: **end if**
8: $A \leftarrow$ the attribute from *ATTRIBUTES* that best classifies *EXAMPLES*
9: decision attribute for $ROOT \leftarrow A$
10: **for** all possible values v_i of the attribute A **do**
11: attach a new edge to *ROOT* corresponding to the predicate $A = v_i$
12: $EXAMPLES_{v_i} \leftarrow$ the subset of *EXAMPLES* with the value v_i for A
13: **if** $EXAMPLES_{v_i} = \emptyset$ **then**
14: attach a leaf to the edge, labelled by the most frequent value of *TARGET_ATTRIBUTE*
15: **else**
16: attach the subtree ID3($EXAMPLES_{v_i}, TARGET\_ATTRIBUTE, ATTRIBUTES \setminus \{A\}$)
17: **end if**
18: **end for**
19: **return** *ROOT*

Information gain — the expected entropy reduction after dividing the data according to the attribute considered:

$$\mathrm{GAIN}(D, A) = H(D) - \sum_{v \in \mathrm{VALUES}(A)} \frac{|D_v|}{|D|} H(D_v).$$

Since $H(D)$ is the same for all attributes, maximising the information gain is equivalent to minimising the weighted entropy $H(A)$.

The C4.5 algorithm (Quinlan, 1993) — a modification of ID3:

- **Missing data** — ignored during tree construction (when evaluating the criterion for data division, only the known values of the given attribute are considered); during classification the missing values are estimated based on the values known for other patterns.
- **Continuous data** — categorised according to the values of the respective attribute on the patterns from the training set.
- **Pruning by validation** — the training data is divided into a training ($\frac{2}{3}$) and a validation ($\frac{1}{3}$) set. *Reduced-error pruning*: subtrees are replaced by leaves labelled by the most frequent class of the respective training patterns; the new tree *must not* perform worse on the validation set than the original one, otherwise the subtree is not replaced. *Subtree raising*: a subtree is replaced by its most frequently used subtree, which is then raised to a higher level; the classification accuracy is tested.
- **Rule post-pruning** — a decision tree is inferred from the training set (over-training is allowed), converted into an equivalent set of rules (one rule per path from the root to a leaf), the rules are pruned (generalised) by eliminating all possible assumptions if this does not worsen the estimated classification accuracy, and are ordered by their expected accuracy; they are then used in this order to classify the patterns. Pruning is easier and more radical than for complete decision trees, and the resulting rules are transparent and easy to understand.
- **Estimating the accuracy of rules** — a standard approach uses a validation set; C4.5 evaluates the accuracy *on the training data* as the ratio of patterns correctly classified by the rule to all patterns covered by the rule, $acc = k/n$. The standard deviation of the accuracy is assessed assuming a binomial distribution; the *lower accuracy bound* for the chosen confidence interval is

taken as the rule characteristic, at the 95 % confidence level

$$acc - 1.96\sqrt{\frac{acc(1-acc)}{n}}.$$

- **Information gain ratio** as the criterion for data division:

$$\text{GAIN\_RATIO}(D, A) = \frac{\text{GAIN}(D, A)}{-\sum_v \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}}.$$

  The attribute with the *biggest* information gain ratio is used for the division; the denominator penalises *excessive branching* (attributes with many values).

The C5.0 algorithm — a commercial successor of C4.5 for large databases: a more efficient generation of rules and a higher accuracy thanks to *boosting* (see the ensemble methods below).

CART — Classification And Regression Trees

Generates *binary* trees — during each data division just two edges are formed. The choice of the best decision attribute uses entropy or the **Gini index**: for an attribute with r values $v_1, \ldots, v_r$, n data patterns and m classes

$$G = \sum_{i=1}^{r} \frac{n_i}{n} G(v_i), \qquad G(v_i) = 1 - \sum_{j=1}^{m} p_j^2, \qquad \sum_{i=1}^{r} n_i = n,$$

where n_i is the number of patterns taking on the value v_i and p_j the probability of class j in these n_i patterns. *Lower* values of G imply better discrimination.
The division is performed according to the "best" data point found for the respective node t; all possible values v of the decision attribute A are checked (in the case of equality the right-hand-side subtree is used). Let P_L, P_R be the probabilities of processing by the left-hand-side, resp. right-hand-side subtree, and $P(C_j \mid t_L), P(C_j \mid t_R)$ the probabilities that a pattern belongs to class C_j and is in the left-hand-side, resp. right-hand-side subtree of node t. The criterion for data division:

$$\Phi = 2P_L P_R \sum_{j=1}^{m} \left| P(C_j \mid t_L) - P(C_j \mid t_R) \right|,$$

which attains its maximum when all patterns from the class considered are grouped in the same subtree.
Characteristic properties of CART: the order of the attributes corresponds to their impact during classification; missing data is ignored and is not counted towards the evaluation of the decision criteria; the stopping criterion — no further division would improve the classification accuracy; a high accuracy achieved on the training set does not have to reflect the accuracy on the test data.

3.2.3 Bayesian classification

Bayes' formula and the MAP hypothesis

The probability that the hypothesis H is true if we observe E:

$$P(H \mid E) = \frac{P(E \mid H)\, P(H)}{P(E)},$$

where $P(H)$ is the *a-priori* probability of the hypothesis, $P(H \mid E)$ the *a-posteriori* (conditional) probability and $P(E)$ the probability of the event E.
The *most probable hypothesis* H_{MAP} (maximum a-posteriori) has the biggest a-posteriori prob-

ability. Since the denominator $P(E)$ is the same for all hypotheses, it can be neglected:

$$H_{MAP} = \arg\max_{H \in \mathcal{H}} P(H \mid E) = \arg\max_{H \in \mathcal{H}} P(E \mid H)\, P(H).$$

Naive Bayesian classifier

Assumption: the events $E_1, \ldots, E_k$ are *conditionally independent* subject to the validity of the hypothesis H. In real-world tasks this assumption is rarely fulfilled — hence the name "naive". Then

$$P(H \mid E_1, \ldots, E_k) = \frac{P(E_1, \ldots, E_k \mid H)P(H)}{P(E_1, \ldots, E_k)} = \frac{P(H)}{P(E_1, \ldots, E_k)} \prod_{i=1}^{k} P(E_i \mid H),$$

and we classify by the hypothesis with the biggest a-posteriori probability

$$H_{MAP} = \arg\max_{H_t} P(H_t) \prod_{i=1}^{k} P(E_i \mid H_t).$$

The estimates from the data: $P(H_t) = P(\text{class } t) = \frac{\text{number of patterns from } t}{\text{number of all patterns}}$ and $P(E_k \mid H_t) = \frac{\text{number of patterns from } t \text{ matching } E_k}{\text{number of patterns from } t}$.
Advantage: the possibility to classify even *incompletely described* patterns. **Disadvantage:** a zero a-posteriori probability $P(E_k \mid H_t)$ if there is no corresponding pattern in the training set, resp. underestimated terms in the case of a low mutual incidence of E_k and H_t.

Example — Bayesian classification of a loan

The simple case: $P(\text{borrow}) = 0.667$, $P(\text{reject}) = 0.333$, $P(\text{high income} \mid \text{borrow}) = 0.91$, $P(\text{high income} \mid \text{reject}) = 0.12$. Then $0.91 \cdot 0.667 = 0.607$ vs. $0.12 \cdot 0.333 = 0.040$, hence $H_{MAP} = \text{borrow}$.
The naive classifier over the loan data (12 clients): $P(\text{credit(yes)}) = 8/12 = 0.667$, $P(\text{credit(no)}) = 4/12 = 0.333$; $P(\text{account(middle)} \mid \text{yes}) = 2/8 = 0.25$, $P(\text{account(middle)} \mid \text{no}) = 2/4 = 0.5$; $P(\text{unempl.(no)} \mid \text{yes}) = 5/8 = 0.625$, $P(\text{unempl.(no)} \mid \text{no}) = 1/4 = 0.25$.
For a client with a middle balance on their account who is not unemployed:

$$0.667 \cdot 0.25 \cdot 0.625 = 0.1042 \quad \text{vs.} \quad 0.333 \cdot 0.5 \cdot 0.25 = 0.0416,$$

so the client is assigned to the class **credit(yes)**.

3.2.4 Support Vector Machines

SVMs were invented by V. Vapnik and his co-workers in the 1970s in Russia and became known to the West in 1992. They are *linear* classifiers that find a hyperplane separating two classes (positive and negative); for non-linear separation, *kernel* functions are used. SVM not only has a rigorous theoretical foundation but also performs classification more accurately than most other methods in applications, especially for high-dimensional data — it belongs to the best classifiers for text classification.

Linear SVM, the separable case

The training set $D = \{(x_1, y_1), \ldots, (x_r, y_r)\}$, $x_i \in X \subseteq \mathbb{R}^n$, $y_i \in \{1, -1\}$. SVM finds a linear function $f(x) = \langle w \cdot x \rangle + b$ such that

$$y_i = \begin{cases} 1 & \text{if } \langle w \cdot x_i \rangle + b \geq 0, \\ -1 & \text{if } \langle w \cdot x_i \rangle + b < 0. \end{cases}$$

The hyperplane $\langle w \cdot x\rangle + b = 0$ is called the *decision boundary* (surface). There are many such hyperplanes — SVM looks for the one with the *largest margin*; machine learning theory says that this hyperplane minimises the error bound.

The (perpendicular) distance from a point x_i to the hyperplane is $\dfrac{|\langle w \cdot x_i\rangle + b|}{\|w\|}$, where $\|w\| = \sqrt{\langle w \cdot w\rangle}$. For the margin hyperplanes H_+ and H_-,

$$d_+ = d_- = \frac{1}{\|w\|}, \qquad \text{margin} = d_+ + d_- = \frac{2}{\|w\|}.$$

The optimisation problem:

$$\text{minimise} \quad \frac{\langle w \cdot w\rangle}{2} \quad \text{subject to} \quad y_i\big(\langle w \cdot x_i\rangle + b\big) \geq 1,\ i = 1, \ldots, r,$$

which summarises the constraints $\langle w \cdot x_i\rangle + b \geq 1$ for $y_i = 1$ and $\langle w \cdot x_i\rangle + b \leq -1$ for $y_i = -1$.

The dual formulation and support vectors

The problem is solved by the standard *Lagrangian* method — the constraints are pulled into the objective function using *Lagrange multipliers* α_i, one per training constraint:

$$L_P = \frac{\langle w \cdot w\rangle}{2} - \sum_{i=1}^{r} \alpha_i \big[y_i(\langle w \cdot x_i\rangle + b) - 1\big], \qquad \alpha_i \geq 0.$$

Kuhn–Tucker conditions. An optimal solution must satisfy them. In general they are only *necessary*, but for a *convex* objective function with *linear* constraints — which is exactly our case — they are also *sufficient*:

$$\frac{\partial L_P}{\partial w_j} = w_j - \sum_{i=1}^{r} y_i \alpha_i x_{ij} = 0, \quad j = 1, \ldots, n \qquad \text{stationarity in } w$$

$$\frac{\partial L_P}{\partial b} = -\sum_{i=1}^{r} y_i \alpha_i = 0 \qquad \text{stationarity in } b$$

$$y_i\big(\langle w \cdot x_i\rangle + b\big) - 1 \geq 0, \quad i = 1, \ldots, r \qquad \text{the original constraints}$$

$$\alpha_i \geq 0, \quad i = 1, \ldots, r \qquad \text{non-negativity of multipliers}$$

$$\alpha_i\big(y_i(\langle w \cdot x_i\rangle + b) - 1\big) = 0, \quad i = 1, \ldots, r \qquad \textbf{complementarity}$$

where x_{ij} is the j-th component of the vector x_i (in vector form $w = \sum_i y_i \alpha_i x_i$).

Why complementarity yields the support vectors: the product is zero, so for every i at least one of the two factors must vanish. If $\alpha_i > 0$, the second bracket must be zero, i.e. $y_i(\langle w \cdot x_i\rangle + b) = 1$ — the point lies *exactly on* the margin hyperplane H_+ or H_-. These are the **support vectors**. For all the other points $\alpha_i = 0$, so they do not enter the solution at all: an SVM is determined by a small subset of the training data only.

From the primal to the dual program — two steps:

1. *Set to zero the partial derivatives of L_P with respect to the primal variables w* and b. This gives the first two KKT conditions, rewritten as

$$w = \sum_{i=1}^{r} y_i \alpha_i x_i, \qquad \sum_{i=1}^{r} y_i \alpha_i = 0.$$

2. *Substitute these relations back into L_P.* The primal variables w and b are thereby **eliminated** and only the multipliers α_i remain.

The result is the **Wolfe dual** problem. *Note:* it is a **constrained** problem — the relations obtained in step 1 are part of it:

$$\text{maximize} \quad L_D = \sum_{i=1}^{r} \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle x_i \cdot x_j \rangle \quad \text{subject to} \quad \sum_{i=1}^{r} y_i \alpha_i = 0, \quad \alpha_i \geq 0.$$

The maximum of L_D occurs at the same values of w, b, α_i as the minimum of L_P. The dual is easier to solve than the primal (numerical techniques, in practice e.g. SMO) and, crucially, it involves the input vectors *only* through the dot products $\langle x_i \cdot x_j \rangle$ — precisely what makes the kernel trick possible (see below).

Recovering w and b. Solving the dual yields the α_i; from them

$$w = \sum_{i \in sv} y_i \alpha_i x_i, \qquad b = y_k - \langle w \cdot x_k \rangle \quad \text{for any support vector } x_k,$$

where the expression for b follows from complementarity: for a support vector $y_k(\langle w \cdot x_k \rangle + b) = 1$, and since $y_k \in \{1, -1\}$, hence $y_k^2 = 1$, it suffices to multiply by y_k. Numerically, b is usually *averaged* over all support vectors.

The decision boundary and **testing** an instance z:

$$\langle w \cdot x \rangle + b = \sum_{i \in sv} y_i \alpha_i \langle x_i \cdot x \rangle + b = 0, \qquad \operatorname{sign}\Big(\sum_{i \in sv} \alpha_i y_i \langle x_i \cdot z \rangle + b \Big).$$

If the expression returns 1, z is classified as positive; otherwise as negative.

The kernel trick

Real-life data may require non-linear separation. The data is therefore *transformed* into another (usually much higher-dimensional) space in which a *linear* boundary separates the classes: $\phi : X \to F$, $x \mapsto \phi(x)$; the transformed space F is called the *feature space*, the original one the *input space*.

The problem with explicit transformation: the number of dimensions of the feature space can be huge for useful transformations even with a reasonable number of input attributes — the *curse of dimensionality* makes explicit computation infeasible.

The crucial observation: in the dual formulation, both the construction of the optimal hyperplane and the evaluation of the decision function require *only the dot products* $\langle \phi(x) \cdot \phi(z) \rangle$, never the mapped vector $\phi(x)$ itself. So if we have a way to compute the dot product directly from the input vectors, we do not need to know ϕ at all — a *kernel function*

$$K(x, z) = \langle \phi(x) \cdot \phi(z) \rangle.$$

In all expressions the dot product is replaced by the kernel — the **kernel trick**.

Example (polynomial kernel with $d = 2$, 2-D input):

$$\begin{aligned} \langle x \cdot z \rangle^2 &= (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2 x_1 z_1 x_2 z_2 + x_2^2 z_2^2 \\ &= \big\langle (x_1^2, x_2^2, \sqrt{2} x_1 x_2) \cdot (z_1^2, z_2^2, \sqrt{2} z_1 z_2) \big\rangle = \langle \phi(x) \cdot \phi(z) \rangle. \end{aligned}$$

The kernel $\langle x \cdot z \rangle^d$ is therefore a dot product in a transformed feature space without ever evaluating ϕ. Whether a given function is a kernel (i.e. a dot product in some feature space) is answered by *Mercer's theorem*. Commonly used kernels: linear (the identity), polynomial, Gaussian (RBF), sigmoid.

Further properties of SVM. It works only in a real-valued space — categorical attributes have to be converted to numeric values. It does only *two-class* classification; for multi-class problems, strategies such as *one-against-rest* or *error-correcting output coding* are applied. The hyperplane

produced by SVM is hard to understand by human users and the kernels make the matter worse — SVM is therefore commonly used in applications that do not require human understanding. [♦ beyond the slides] For non-linearly separable data, *slack variables* $\xi_i \geq 0$ are introduced and $\frac{\langle w \cdot w \rangle}{2} + C \sum_i \xi_i$ is minimised; the parameter C controls the trade-off between the width of the margin and the number of errors.

3.2.5 Logistic regression

Logistic regression is the most important case of non-linear regression (Chapter 2) and at the same time a classifier: the dependent variable y is categorical, typically two-valued, and what is modelled is the *probability* that y takes a certain value, depending on the combination of the values of the independent quantities.

Logistic regression

For binary $y \in \{0, 1\}$ one models the logarithm of the "conditional odds":

$$\ln \frac{P(y = 1 \mid x_1, \ldots, x_m)}{1 - P(y = 1 \mid x_1, \ldots, x_m)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_m x_m = z,$$

whence

$$P(y = 1 \mid x) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z) \qquad \text{(the sigmoid function)}.$$

Learning by maximum likelihood:

$$L = \prod_{i=1}^{p} P\big(y_i \mid x_{i,1}, \ldots, x_{i,m}\big) = \prod_{i=1}^{p} \Big(\frac{1}{1 + e^{-z_i}}\Big)^{y_i} \Big(\frac{1}{1 + e^{z_i}}\Big)^{1 - y_i}.$$

Instead of maximising L, one minimises the *cross-entropy* loss $E = -\log L$:

$$E = -\sum_i \Big[y_i \log \sigma(z_i) + (1 - y_i) \log \big(1 - \sigma(z_i)\big) \Big].$$

Since the derivative of the sigmoid is $\sigma' = \sigma(1 - \sigma)$, after simplification we obtain very simple adjustment rules

$$\Delta\beta_j = -\eta \frac{\partial E}{\partial \beta_j} = \eta \sum_i \big(y_i - \sigma(z_i)\big) x_{i,j}, \qquad \Delta\beta_0 = \eta \sum_i \big(y_i - \sigma(z_i)\big),$$

where η denotes the learning rate. The error thus enters the weight update directly as the difference between the *observed* and the *predicted* value.

Logistic regression is a common choice for the base classifier wherever a probabilistic output and interpretable coefficients are required — e.g. in link prediction in social networks (Chapter 6).

3.2.6 Discriminant analysis

Discriminant analysis

Classification of patterns into known classes: we search for the dependence of the nominal quantity (that impacts the class membership) on the other numerical quantities. We assume that to every class C_j, $j = 1, \ldots, m$ there exists a *discriminative function* g_j; a pattern $x = (x_1, \ldots, x_m)$ belongs to class C_j if and only if

$$g_j(x) = \max_k g_k(x).$$

Linear discriminant analysis: $g_j(x) = \beta_{j0} + \beta_{j1} x_1 + \cdots + \beta_{jm} x_m$. For two classes, instead

of g_1, g_2 we can look for the single function $g = g_1 - g_2$ and classify according to its *sign*: "+" indicates class 1, "−" class 2.
An optimum classification in the sense of a minimum error occurs when the discriminant functions are $\propto$ the a-posteriori (conditional) probabilities of classifying the observation x into class C_j: $g_j(x) = P(C_j \mid x) = \dfrac{p(x \mid C_j)P(C_j)}{p(x)}$; for two classes $g(x) = p(x \mid C_1)P(C_1) - p(x \mid C_2)P(C_2)$.

For a *normal distribution* with different means μ_1, μ_2 and general covariance matrices S_1, S_2 we get a *quadratic* discriminant function. If the covariance matrices are *identical* ($S_1 = S_2 = S$), it reduces to a *linear* one,

$$g(x) = (\mu_1 - \mu_2)^\top S^{-1} x - \tfrac{1}{2}(\mu_1 - \mu_2)^\top S^{-1}(\mu_1 + \mu_2) - \ln \frac{P(C_2)}{P(C_1)},$$

and if moreover both classes are equally probable and $S = I$, it is a simple comparison of the distances from the means. For normal distributions the search for discriminant functions therefore reduces to **estimating the means** (from the sample means) and the **covariance matrices** (composed of corrected sample variances).

3.2.7 Neural networks and extreme learning machines (ELM)

The ELM network (Extreme Learning Machine, G.-B. Huang)

A feedforward network with *one* hidden layer (n inputs, L hidden neurons, 1 output): $f_L(x) = \sum_{i=1}^{L} \beta_i G(a_i, b_i, x)$, where G is e.g. a sigmoid $g(\langle a_i \cdot x \rangle + b_i)$ or an RBF $g(b_i \|x - a_i\|)$.
The key idea: the parameters of the hidden neurons (a_i, b_i) are *not adapted* — they are generated *randomly*, even without the knowledge of the training data (with probability one, $\lim_{L\to\infty} \|f - f_L\| = 0$). Only the output weights β are learned, and *once* — by solving the system $H\beta = T$, where H is the *output matrix of the hidden layer* (its j-th row = the outputs of the hidden neurons for the pattern x_j):

1: choose *randomly* the parameters of the hidden neurons (a_i, b_i), $i = 1, \ldots, L$
2: compute the output matrix of the hidden layer H
3: $\beta \leftarrow H^+ T$ ▷ the Moore–Penrose pseudoinverse

Regularisation (adding I/C to the diagonal of $H^\top H$) gives a more stable solution with better generalisation.

Advantages of ELM: three non-iterative steps — very fast, no local minima or initialisation issues; the hidden-neuron parameters do not depend on the data; any bounded piecewise continuous transfer function suffices (gradient algorithms require a smooth one).

3.2.8 Ensemble methods

Bagging (bootstrapped aggregating)

Attempts to reduce the *variance* of the classifier: if the variance of a classifier is σ^2, then the variance of the average of k independent and identically distributed (i.i.d.) classifiers reduces to σ^2/k.
Bootstrapping approximates i.i.d. classifiers: the data points are sampled uniformly from the original data *with replacement*; the sample is drawn of approximately the same size as the original data, may comprise duplicates, and contains a fraction of roughly

$$1 - \left(1 - \frac{1}{n}\right)^n \approx 1 - \frac{1}{e} = 63.2\,\%$$

distinct data points from the original data (the probability of a data point not being included is $(1 - 1/n)^n$). A total of k different bootstrapped samples of size n are drawn independently, a

classifier is trained on each of them, and for a test instance the predicted class label is reported as the *dominant vote* of the different classifiers.
Suitability: decision trees are an ideal choice for bagging — they have a *low bias* and *high variance* (if grown sufficiently deep). **Limitation:** bagging does *not reduce the model bias*; for cases in which the bias is the primary source of error it may even slightly worsen accuracy due to correlations between the ensemble components (and hence the failed i.i.d. assumption).

Random forests

If k classifiers, each with variance σ^2, have a positive pairwise *correlation* ρ between them, then the variance of the averaged prediction is

$$\rho\sigma^2 + \frac{(1-\rho)\sigma^2}{k}.$$

The term $\rho\sigma^2$ is *invariant* to the number of components k — it is precisely this term that limits the performance gains from bagging. Bootstrapped decision trees, moreover, tend to be positively correlated.
A *random forest* is an ensemble of decision trees into whose model-building process randomness has explicitly been inserted, so that the components are less correlated. Two options: *(1)* **bagging** — make the trees different by training them on slightly different data (bootstrap samples); *(2)* a **limited number of features** — at each node, a *random subset* of m features is given to the tree and the information gain is computed only on that subset. The second option also speeds up training (fewer features to search over) and the trees need not be pruned; the reduced variance does not negatively affect the bias. The output of the forest is the majority vote of the trees.
The algorithm: for each of the N trees: create a new bootstrap sample of the training set; use it to train a decision tree; at each node randomly select m features, compute the information gain only on that set and select the optimal one; repeat until the tree is complete.
Parameters: the number of features m — a subset size that is the square root of the number of features seems to be common (random forests are not too sensitive to this parameter); the number of trees — building trees can be continued until the error stops decreasing.

Boosting and AdaBoost (Freund & Schapire, 1996)

The principle: a *weight* is associated with each training pattern and different classifiers are trained using these weights; the weights are iteratively modified based on the performance of the classifier. The models to be constructed therefore *depend on the previous ones* — each classifier is constructed using the same algorithm on a different weighted training set. **The idea:** in future iterations, focus on the *misclassified* patterns by increasing their relative weight; by iteratively using this approach and creating a weighted combination of the various classifiers, it is possible to create a classifier with a lower overall *bias* (unlike bagging, which reduces the variance).
AdaBoost(data set D, base classifier A, maximum rounds T):

1: $t \leftarrow 0$; for each instance i initialise $W_1(i) = 1/n$
2: **repeat**
3: $\quad t \leftarrow t + 1$
4: $\quad$ determine the weighted error rate ϵ_t on D when the base algorithm A is applied to the data weighted with $W_t(\cdot)$
5: $\quad \alpha_t \leftarrow \frac{1}{2}\ln\left((1-\epsilon_t)/\epsilon_t\right)$
6: $\quad$ **for** each *misclassified* $X_i \in D$ **do**
7: $\quad\quad W_{t+1}(i) \leftarrow W_t(i)e^{\alpha_t}$
8: $\quad$ **end for**
9: $\quad$ **for** each *correctly classified* $X_i \in D$ **do**
10: $\quad\quad W_{t+1}(i) \leftarrow W_t(i)e^{-\alpha_t}$

11: **end for**
12: normalise $W_{t+1}(i) \leftarrow W_{t+1}(i) / \sum_j W_{t+1}(j)$
13: **until** $t \geq T$ **or** $\epsilon_t = 0$ **or** $\epsilon_t \geq 0.5$
14: classify test instances with the ensemble using the component weights α_t

ϵ_t is the fraction of incorrectly predicted training instances (computed after weighting with W_t) by the model in the t-th iteration.
Properties: it produces a *sequence* of classifiers with the same base learner, each depending on the previous one and focusing on its errors. At testing, the results of the sequence are combined.
Sensitivity to noise: when the number of outliers is very large, the emphasis placed on the hard examples can hurt performance.

Random forests	**Boosting**
fast — they can run *in parallel* and search over a small set of features at each stage	exhaustive — searches over the *entire* feature set at each stage, and each stage depends on the previous one
as the trees are cheap to train, more of them can be grown in the same computational time, even on large and complicated data sets	must run *sequentially*, which can be expensive
can produce surprisingly good results in comparison to the small parts of the problem space seen by each tree	for the *same number* of trees, boosting outperforms random forests (as the forests search only a small subset at each stage)

3.2.9 Comparison of the classification methods

Method	**Comprehensibility**	**Accuracy**	**Speed**	**Note**
Decision tree	high	medium	high	rectangular regions, pruning needed, ignores attribute correlations
Rules from C4.5	highest	medium	high	rule pruning is more radical than tree pruning
Naive Bayes	medium	surprisingly good	very high	handles incomplete patterns; problem with zero probabilities
SVM	very low	high	low (training)	best for high-dimensional data (text); only 2 classes, only a real-valued space
Logistic regression	high	medium	high	probabilistic output, interpretable coefficients
Discriminant analysis	medium	good under normality	high	only μ_j and S_j need to be estimated
Neural network / ELM	very low	high	ELM very high	ELM has no local minima and does not train the hidden layer
Random forest	low	very high	high (parallel)	reduces variance, does not worsen bias
Boosting / AdaBoost	low	very high	low (sequential)	reduces bias; sensitive to noise and outliers

3.3 Cluster analysis

3.3.1 Formulation of the task and distance measures

Cluster analysis divides the observed patterns into groups (clusters) of mutually similar patterns. **Assumption:** we can measure the *distance* between patterns. Each pattern is characterised by m numerical quantities.

Distance measures

For patterns $x_1 = (x_{11}, \ldots, x_{1m})$ and $x_2 = (x_{21}, \ldots, x_{2m})$:

1. **Hamming** (Manhattan, L_1): $d_H(x_1, x_2) = \sum_{i=1}^{m} |x_{1i} - x_{2i}|$
2. **Euclidean** (L_2): $d_E(x_1, x_2) = \sqrt{\sum_{i=1}^{m} (x_{1i} - x_{2i})^2}$
3. **Chebyshev** (L_∞): $d_C(x_1, x_2) = \max_i |x_{1i} - x_{2i}|$
4. **Minkowski metric** (L_q) — the previous three are its special cases: $d_q(x_1, x_2) = \left(\sum_{i=1}^{m} |x_{1i} - x_{2i}|^q \right)^{1/q}$

We have $d_H = d_1$, $d_E = d_2$, $d_C = \lim_{q \to \infty} d_q$; for the pattern $(0, \ldots, 0)$ and a pattern at unit distance all three metrics give the value 1.

The choice of the metric depends on the quantities involved and their range ⇒ the quantities have to be **normalised** (divided by their mean, standard deviation, range max − min, ...; see Chapter 2). The metrics above assume the *same variance* for all the quantities. For quantities with *different* variance one uses

Mahalanobis distance

$$d_M^2(x_1, x_2) = (x_1 - x_2)^\top S^{-1} (x_1 - x_2),$$

where S is the covariance matrix. It takes both the different variances and the correlations between the quantities into account (for $S = I$ it reduces to the squared Euclidean distance).

The distance between two clusters C_k and C_l

The method of the nearest neighbour (single linkage) — the minimum of the distances between their elements: $D(C_k, C_l) = \min_{x_i, x_j} d(x_i, x_j)$, $x_i \in C_k$, $x_j \in C_l$.
The method of the farthest neighbour (complete linkage) — the maximum: $D(C_k, C_l) = \max_{x_i, x_j} d(x_i, x_j)$.
The method of average distance (average linkage) — the average of the distances between the patterns (n_k, n_l being the numbers of patterns in the clusters):

$$D(C_k, C_l) = \frac{1}{n_k n_l} \sum_{i=1}^{n_k} \sum_{j=1}^{n_l} d(x_i, x_j).$$

The centroid method — the distance between the centres of the clusters: $D(C_k, C_l) = d(c_k, c_l)$, where c_k, c_l are the centres (centroids) of the clusters.

A *centroid* is the centre of a cluster — a "prototype" representing the given cluster. The same cluster can be represented by one or even more centroids at the same time, depending on the shape of the cluster and the metric chosen.

3.3.2 The k-means method

The k-means algorithm (Lloyd, 1982)

1: divide the patterns *randomly* into k clusters $C_1, \ldots, C_k$ (disjoint, their union being all patterns)
2: determine the centroids of all the clusters in the recent division, e.g. $c_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$
3: **for** each pattern x **do**
4: determine the distances $d(x, c_k)$ for $1 \le k \le K$
5: let $d(x, c_l) = \min_k d(x, c_k)$
6: **if** x does not belong to cluster l **then**
7: move x to cluster l
8: **end if**

9: **end for**
10: **if** there was a transfer **then**
11: goto step 2
12: **else**
13: **finish**
14: **end if**

Afterwards, the clusters are represented by their centroids.
Variants: at the initial division, pronounce the *first k patterns* to be centroids (which eliminates step 2 in the first iteration); adjust the centroids *after every* transfer (inside the cycle rather than after it).

Extensions to the Lloyd algorithm. Alternating minimisation steps do not provide an approximation guarantee, yet Lloyd's algorithm *never increases* the cost of the initial clustering ⇒ it pays to provide the algorithm with a *provably good initialisation*:

- **k-means++** (Arthur, Vassilvitskii, 2007) — sample k of the P patterns with probability proportional to their squared distance to the closest existing centroid: $\frac{d^2(x_i, c)}{\sum_{j=1}^{P} d^2(x_j, c)}$.
- **LocalSearch++** (Lattanzi, Sohler, 2019) — first use k-means++ to pick k initial centroids; afterwards, new patterns are sampled with an analogous probability and replace existing centroids if they decrease the overall squared distance between the patterns and their closest centroids.

The k-medians method uses the Manhattan distance as the objective function, $d(x_i, c) = \|x_i - c\|_1$. It can be shown that the optimal representative c is then the *median* of the data points along each dimension in the cluster. As the median is chosen independently along each dimension, the resulting m-dimensional representative will typically *not* belong to the original data set. k-medians selects the cluster representatives in a *more robust* way than k-means, because the median is not as sensitive to outliers as the mean.

3.3.3 The k-medoids method, CLARA and CLARANS

The k-medoids algorithm

For a data set D of n d-dimensional points $x_1, \ldots, x_n$, the goal is to determine k representatives $y_1, \ldots, y_k$ *out of the original database* D (k is specified by the user) that minimise the objective function given by the sum of the distances between the data points and their closest representatives:

$$O = \sum_{i=1}^{n} \min_j d(x_i, y_j).$$

The time complexity of each iteration is $O(k\,n\,d)$; usually a small constant number of iterations is necessary.
Why select the representatives from D? *(1)* k-means clusters may be distorted by outliers — the cluster representatives can be located in empty regions, unrepresentative of most of the data points in that cluster, which may lead to a partial merging of different clusters (this can be resolved at least partially with careful outlier handling). *(2)* For complex data (e.g. a set of time series of varying length) it might be difficult to compute the optimal central representative at all. The k-medoids algorithm can therefore be defined virtually for *any data type*, as long as an appropriate similarity or distance function can be provided.
The procedure: *hill-climbing* — the representative set S is initialised to a set of points from D and is iteratively improved by exchanging a point from S with a data point from D. Possible strategies for the exchange:
1. try all $|S| \cdot |D|$ possibilities and select the best one — *extremely expensive*;
2. use a randomly selected set of r pairs (x, y) (x from D, y from S) and use the best of the

r pairs for the exchange. This requires time proportional to $r \cdot |D|$ and is said to have converged when the objective function does not improve, or if the average improvement is below a user-specified threshold.

k-medoids is *slower* than k-means but applicable to various data types.

Scalable variants: CLARA and CLARANS

Scalable implementations of k-medoids; the base variant *PAM* (Partitioning Around Medoids) tests all $k(n-k)$ medoid–non-medoid exchanges, $O(kn^2d)$ per iteration — too expensive.

- **CLARA** (Clustering LARge Applications): apply PAM to a *sample* only and assign the remaining points to the medoids found; repeat over independent samples, select the best. Risk: a sample may not include good medoids.
- **CLARANS** (... based on RANdomized Search): works with the *full* data and tries *random* exchanges; a local optimum after *MaxAttempt* failures, the whole process *MaxLocal* times. Explores a greater diversity of solutions than CLARA.

3.3.4 Hierarchical clustering

Agglomerative hierarchical clustering

A "bottom-up" approach.

Initialisation: *(1)* determine the mutual distances between all patterns; *(2)* assign each pattern to an individual cluster.

Main cycle: while there are more clusters than one: *(1)* find the two mutually closest clusters and merge them together; *(2)* for the new cluster, compute its distance from the other clusters.

The result is displayed as a **dendrogram** — it shows (from the left to the right) the gradual merging of clusters; the height of a join corresponds to the distance at which it occurred. *The optimum number of clusters is not known in advance* — it is chosen by cutting the dendrogram at a suitable height.

Divisive hierarchical clustering ("top-down") proceeds the other way round: it starts with one cluster containing all patterns and divides it gradually. The best-known example in the context of networks is the Girvan–Newman algorithm (Chapter 6).

There is also a scalable agglomerative variant, **CURE** (*Clustering Using REpresentatives*): sampling and hierarchical merging per partition; it is very fast and discovers clusters of an *arbitrary shape*.

3.3.5 Vector quantisation: the LVQ algorithm

LVQ — Learning Vector Quantisation

Used for **supervised clustering** (the classes are denoted by C):

1: initialise all weight vectors $w_j(0)$, the learning rates $\mu(0)$, $k \leftarrow 0$
2: **while** the stop condition is not satisfied **do**
3: **for** each training pattern x **do**
4: determine the index $j = q$ of the weight vector with the *minimum* Euclidean distance, i.e. $\min_j \|x - w_j\|_2^2$
5: **if** the class of x agrees with the class of w_q **then**
6: $w_q(k+1) \leftarrow w_q(k) + \mu(k)\big(x - w_q(k)\big)$ ▷ pull towards
7: **else**
8: $w_q(k+1) \leftarrow w_q(k) - \mu(k)\big(x - w_q(k)\big)$ ▷ push away
9: **end if**
10: **end for**
11: $k \leftarrow k+1$; decrease the learning rate, e.g. $\mu(k) = \mu(k-1)/(k+1)$ for $k > 0$
12: **end while**

The weight vectors are usually initialised by the *first m vectors* of the training set (which must include patterns from all the classes) — this automatically "calibrates" the neurons with the classes.

3.3.6 Grid-based and density-based methods

Grid-based and density-based methods find clusters of an *arbitrary shape* as **connected dense regions** (the connected components of a connectivity graph). *Grid-based* methods (e.g. MAFIA) discretise the space into p^d hypercubes and connect *dense* cells (at least τ points) that share a side. The density-based **DBSCAN** builds on *core points* instead of cells (at least τ points within the radius ε): the clusters are the connected components of the graph of core points at most ε apart, *border* points are assigned to the closest cluster, and *noise* points are reported as outliers.

Method	**Cluster shape**	**Number of clusters**	**Outliers**	**Note**
k-means	spherical	given	sensitive	fast; depends on initialisation → k-means++
k-medians	spherical	given	more robust	median per dimension; the representative is not from the data
k-medoids	spherical	given	robust	representatives *from the data*; any data type; slower
CLARA / CLARANS	spherical	given	robust	scalable k-medoids (sampling / randomised search)
Hierarchical	depends on the linkage	read off the dendrogram	sensitive	no need to give k in advance; $O(n^2)$–$O(n^3)$
CURE	arbitrary	given	robust	sampling + hierarchical merging per partition
LVQ	spherical	given by the number of neurons	—	*supervised* clustering
Grid-based (MAFIA)	arbitrary	follows from the data	noise drops out	sensitive to the grid granularity
DBSCAN	arbitrary	follows from the data	*detects them*	parameters ε, τ; struggles with clusters of differing density

3.4 Summary for the examination

Association rules.

- **Support** = the fraction of transactions with both i and j; **confidence** = the fraction of transactions with both among the transactions with i; **lift** $= p(i \wedge j)/\big(p(i)p(j)\big)$. Lift $< 1 \Rightarrow$ the rule is worse than chance and negating the conclusion may be better. Be able to compute all three from a small transaction table.
- **Apriori** uses the *downward closure property* (any subset of a frequent itemset is frequent) and searches *breadth-first*; Apriori-Gen = the join step (combinations matching in $k-2$ categories) + the pruning step (a subcombination of length $k-1$ is missing). The rule space is $O(2^m)$ and MBA produces an *excessive number of rules*.
- **Taxonomies and virtual items** address the choice of the item level; rare items go higher in the taxonomy, frequent ones stay lower; virtual items carry information about the transaction (day, time, customer) — beware of redundant rules.
- **MS-Apriori**: the rare item problem → each item has its own MIS, the minsup of a rule is the *minimum* MIS of its items, plus the support difference constraint φ. *Downward closure ceases to hold* $\Rightarrow$ the items are sorted in ascending order of MIS, the set of seeds L is used (not F_1) and `tailCount` is kept for rule generation. Be able to give the counterexample to downward closure.
- **CAR**: transactions labelled with a class, rule $X \to y$; mined *directly in one step* via rule-items $(\mathit{condset}, y)$; different minsups can be given to different classes (101 % = prohibit a class).

- **Sequential patterns**: a sequence = an ordered list of itemsets; *size* = the number of elements, *length* = the number of items; subsequence via nested inclusions. The **GSP** algorithm works analogously to Apriori. Note: $\langle\{3\}\{8\}\rangle$ and $\langle\{3,8\}\rangle$ do *not* contain each other.
- **FP-Growth**: 2 passes over the data, no candidate generation; the FP-tree shares *prefixes* (item order by decreasing support — a heuristic) and keeps linked lists per item; extraction bottom-up by *divide and conquer* via *prefix path sub-trees* and *conditional FP-trees* (update counts → remove the item's nodes → remove infrequent items). Worst case: unique transactions ⇒ the tree is larger than the data.

Supervised learning.

- **Decision trees**: TDIDT (divide and conquer), attribute choice by *minimum weighted entropy* $H(A) = \sum_v \frac{|A(v)|}{|D|} H(A(v))$, equivalently maximum *information gain*. Be able to go through the loan example (income 0.5747; account 0.6667; gender 0.9183; unemployed 0.825) and to convert the tree into rules. Induction complexity $O(np \log p)$, classification $O(q \log p)$.
- **ID3** (Quinlan 1986) — greedy, information gain. **C4.5** (1993) — missing and continuous data, pruning by validation (*reduced-error pruning*, *subtree raising*), *rule post-pruning*, accuracy estimated by the lower bound $acc - 1.96\sqrt{acc(1-acc)/n}$, the *information gain ratio* (which penalises excessive branching). **C5.0** — commercial, boosting. **CART** — *binary* trees, the Gini index $G(v_i) = 1 - \sum_j p_j^2$ (lower = better), the criterion $\Phi = 2P_L P_R \sum_j |P(C_j|t_L) - P(C_j|t_R)|$.
- **Bayes**: $H_{MAP} = \arg\max_H P(E \mid H)P(H)$ (the denominator can be neglected); the *naive* assumption of conditional independence ⇒ $\arg\max_{H_t} P(H_t) \prod_i P(E_i \mid H_t)$. Handles incomplete patterns; the problem of zero probabilities. Be able to compute the example (0.1042 vs. 0.0416).
- **SVM**: the maximal margin $2/\|w\|$, the primal $\min \frac{\langle w \cdot w \rangle}{2}$ subject to $y_i(\langle w \cdot x_i \rangle + b) \geq 1$; the Kuhn–Tucker conditions are, for a convex problem with linear constraints, necessary *and* sufficient; $\alpha_i > 0$ only for the *support vectors* (this follows from *complementarity*). **The Wolfe dual** arises in two steps: set $\partial L_P/\partial w$ and $\partial L_P/\partial b$ to zero (giving $w = \sum_i y_i \alpha_i x_i$ and $\sum_i y_i \alpha_i = 0$) and substitute back into L_P, which eliminates w, b; it is a *constrained* problem — $\max L_D$ subject to $\sum_i y_i \alpha_i = 0$, $\alpha_i \geq 0$. Back again: w from stationarity, $b = y_k - \langle w \cdot x_k \rangle$ from complementarity. **The kernel trick**: the dual needs only dot products ⇒ $K(x,z) = \langle \phi(x) \cdot \phi(z) \rangle$ and ϕ is never evaluated (Mercer's theorem). Only 2 classes, only a real-valued space, not interpretable.
- **Logistic regression**: $\ln \frac{p}{1-p} = \beta^\top x$, hence $p = \sigma(z) = 1/(1+e^{-z})$; learning by maximum likelihood / minimising cross-entropy; $\sigma' = \sigma(1-\sigma)$ ⇒ the update rule $\Delta\beta_j = \eta \sum_i (y_i - \sigma(z_i)) x_{ij}$.
- **Discriminant analysis**: a $g_j(x)$ for each class, classify by the maximum; for two classes the sign of $g = g_1 - g_2$ suffices; the optimum is $g_j \propto P(C_j \mid x)$; a normal distribution → a quadratic function, with identical covariance matrices *linear* ⇒ only μ_j and S need to be estimated.
- **ELM**: random parameters of the hidden neurons (even without knowing the data), output weights by *one* pseudoinverse $\beta = H^+ T$; regularisation I/C on the diagonal. Fast, no local minima, works with any bounded piecewise continuous transfer function.
- **Ensembles**: *bagging* reduces the *variance* ($\sigma^2 \to \sigma^2/k$ for i.i.d.), the bootstrap contains $\approx 63.2\,\%$ distinct points $(1 - 1/e)$, trees are the ideal components (low bias, high variance); it does not reduce the bias. *Random forests* address the correlation of the components — the variance is $\rho\sigma^2 + (1-\rho)\sigma^2/k$ and the first term does not depend on k; randomness comes from the bootstrap *and* from a random subset of $m \approx \sqrt{\#\text{features}}$ features at each node. *Boosting*/**AdaBoost** reduces the *bias*, the weights of misclassified patterns grow, $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$, it runs *sequentially* and is sensitive to noise. Be able to compare forests vs. boosting (parallel and cheap vs. exhaustive and more accurate for the same number of trees).

Cluster analysis.

- **Metrics**: Hamming/Manhattan (L_1), Euclidean (L_2), Chebyshev (L_∞), Minkowski (L_q, of which the others are special cases); **Mahalanobis** $d_M^2 = (x_1 - x_2)^\top S^{-1} (x_1 - x_2)$ for quantities with different variance. Normalise before clustering.
- **Cluster distance**: nearest neighbour (min), farthest neighbour (max), average distance, centroid

method.

- k-**means** (Lloyd 1982): random division → centroids → transfers → repeat while transfers occur. It never increases the cost but gives no approximation guarantee ⇒ k-**means**++ (probability $\propto$ the squared distance to the closest centroid), **LocalSearch**++.
- k-**medians** (Manhattan, median per dimension, more robust) vs. k-**medoids** (representatives *from the data*, hill-climbing with exchanges, $O(knd)$ per iteration, any data type). **PAM** tests all $k(n-k)$ exchanges ($O(kn^2d)$); **CLARA** applies PAM to samples (risk of a bad sample), **CLARANS** tries random exchanges over the full data (*MaxAttempt*, *MaxLocal*).
- **Hierarchical clustering** bottom-up: every pattern its own cluster → merge the two closest; the output is a **dendrogram** and the number of clusters is chosen only by cutting it. **CURE**: a scalable variant (sampling, hierarchical merging per partition); fast, arbitrary cluster shapes.
- **LVQ**: *supervised* clustering — the winning weight vector is pulled towards the pattern if the classes agree and pushed away if they do not; μ decreases as $\mu(k-1)/(k+1)$.
- **Grid-based and density-based methods** (MAFIA, **DBSCAN**): clusters of arbitrary shape as connected dense regions; DBSCAN — core points ($\geq \tau$ points within radius ε), clusters = the connected components of the core-point graph, noise = outliers.

4 Characteristic and discriminant rules and their interestingness

The topic sentence asks for *methods for the extraction of characteristic discriminant rules and measuring their interestingness*. The chapter is built primarily on the slides of the course *Internet and classification methods* (Holeňa), above all its 5th lecture *When is a classification comprehensible to the user?*: why rules (Section 4.1), extraction of rules from classification trees (Section 4.4) and by evolution (Section 4.5), observational rules (Section 4.6), the statistical view (Section 4.7), measuring interestingness (Section 4.8) and applications (Section 4.9). The notions of a *characteristic* and a *discriminant* rule with the t-weight and d-weight measures (Section 4.3), together with *attribute-oriented induction* (Section 4.2) which produces those rules, are added following Han–Kamber, because the topic names them explicitly.

Sources of the chapter

Of the six lectures of the course *Internet and classification methods* (`ikm1`–`ikm6`), the key one for this chapter is `ikm5` (comprehensibility of classification: rules, their extraction and the observational calculus); the chapter further uses `ikm1` (applications: spam, recommending, network threats), `ikm2` (quality measures of binary classification) and `ikm3` (discriminant analysis). The material of `ikm4` (SVM: margin maximisation, support vectors, the kernel trick) and `ikm6` (teams of classifiers: bagging, boosting, random forests) coincides with Chapter 3, and the confusion matrix and ROC/AUC of `ikm2` with Chapter 5 — they are treated there. The slides `nmr1`–`nmr5` from the same folder belong to a different (optional) course by Holeňa and do not cover the subject matter of this chapter. Sections 4.2 and 4.3 follow Han–Kamber; regression and discriminant analysis are also backed by the NDBI023 slides (*Data Analysis*).

4.1 Rules as a comprehensible form of classification

Mathematically, a classifier is a mapping $f\colon X \to \{C_1, \dots, C_m\}$ from the feature space into a finite set of classes; more precisely $f\colon X \times \Theta \to \{C_1, \dots, C_m\}$, where learning means choosing the parameters $\theta \in \Theta$ according to the experience with the quality of classification under previous choices (Section 3.2). The concrete learned prescription — say, the sign of $\langle w, x\rangle + b$ for an SVM or the composed nonlinear function of a neural network — is, however, hardly comprehensible to an ordinary user. If a binary classifier $f\colon X \to \{1, 0\}$ describes the truth of some implication $\varphi \Rightarrow \psi$ or equivalence $\varphi \Leftrightarrow \psi$ between properties of the object, it suffices to know this *rule* instead of the whole f: a rule of logic is more comprehensible than a mathematical formula.

Boolean and fuzzy rules

Boolean rules work with truth values $t(\varphi) \in \{0, 1\}$: the *equivalence* $\varphi \Leftrightarrow \psi$ is true iff $t(\varphi) = t(\psi)$; the *implication* $\varphi \Rightarrow \psi$ is true iff $t(\varphi) = 0$ or $t(\varphi) = t(\psi) = 1$. Rules are typically applied to objects on which the antecedent holds ($t(\varphi) = 1$) — there the implication coincides with the equivalence.

Fuzzy rules admit degrees of truth $t(\varphi) \in [0, 1]$. The fuzzy implication is the *residuum* of a t-norm T:

$$t(\varphi \Rightarrow \psi) = \sup\{\, c \in [0,1] : T(t(\varphi), c) \le t(\psi) \,\},$$

and the fuzzy equivalence is $t(\varphi \Leftrightarrow \psi) = T\big(t(\varphi \Rightarrow \psi),\, t(\psi \Rightarrow \varphi)\big)$. Important special cases: if $t(\varphi) \le t(\psi)$, then $t(\varphi \Rightarrow \psi) = 1$; if $t(\varphi) = t(\psi)$, then $t(\varphi \Leftrightarrow \psi) = 1$.

Intuition. A fuzzy degree of truth is *not* the fraction of objects on which the rule holds (that is the confidence of an association rule, a different layer) but the degree to which *one particular object* satisfies a vague property. The rule "whoever buys *many* rolls buys *a lot of* butter" is evaluated for each customer separately. For Petr, 8 rolls are "many" to degree 0.7 and 250 g of butter is "a lot" to degree 0.9; the antecedent $0.7 \le 0.9$, so the rule holds for him *fully*, $t = 1$. His butter could drop to 0.7 and the rule would still hold fully — that slack is not added anywhere, truth is capped at one. The residuum handles the opposite case: Jana has rolls "many" to 0.9 but butter only to 0.4, so she violates the rule and the question is *by how much*. In Boolean logic the rule would simply

be false; fuzzy logic says "partly", and the different t-norms are differently strict ways of penalising the gap between antecedent and consequent (for $t(\varphi) > t(\psi)$): minimum (Gödel) gives $t(\psi) = 0.4$, product (Goguen) gives $t(\psi)/t(\varphi) \approx 0.44$, Łukasiewicz gives $1 - t(\varphi) + t(\psi) = 0.5$. Only averaging these values over all customers yields the fuzzy counterpart of rule confidence.

Comprehensibility is the reason why rules are used wherever the result of the classification is judged by a human: a malware analyst wants to know *why* a program is suspicious, and recommender systems increase the user's trust in a recommendation by explaining it (Section 4.9).

4.2 Attribute-oriented induction (AOI)

[♦ beyond the slides] This section follows Han–Kamber; the topic speaks of characteristic and discriminant rules, and AOI is the method that produces them.

Motivation. Data in a database sit at *too low a level of abstraction.* A single tuple "John Newman, 23, Brno, physics" says nothing about a class — describing a class requires *concepts*, not individual values. **Attribute-oriented induction** (AOI) therefore *lifts* the data to a higher conceptual level and only then looks for regularities.

Concept hierarchy

Background domain knowledge supplied by the user (or derived from the data) telling us how the values of an attribute are to be generalised. For example

$$\text{Brno} \to \text{South Moravia} \to \text{Czechia} \to \text{Europe} \to \text{ANY}$$

The top **ANY** means "the value does not matter" — generalising this far effectively removes the attribute. For numeric attributes the hierarchy consists of intervals (age 21 → 20–25 → young) obtained by discretisation (Chapter 2).

The two basic AOI operations

1. Attribute removal — applied when an attribute has *many distinct values* and *no concept hierarchy is available* (name, birth number, telephone). Such an attribute cannot be generalised, and keeping it would mean that no two tuples ever merge.
2. Attribute generalization — if a hierarchy exists, the values are replaced by the concepts one level up. This is repeated until the number of distinct values drops below a chosen threshold. After each step, **identical generalised tuples are merged** and their number is accumulated into a `count` attribute (possibly along with `sum`, `avg`).

When to stop generalising — two thresholds

Attribute generalization threshold: each attribute may have at most k distinct values, typically 2–8. Generalisation continues until it falls below the threshold.
Generalized relation threshold: the resulting relation may have at most k tuples, typically 10–30. If it has more, generalisation continues (the attribute with the most values is lifted one level).
The choice of thresholds is essential and it is a *trade-off*: thresholds that are too low lead to **over-generalisation** — the rule "all clients are people" is true and worthless; thresholds that are too high leave the result unreadable.

The AOI algorithm:

Input: relation W (the result of a query on the target class), concept hierarchies, thresholds
Output: the generalised (*prime*) relation P

1: **for all** attributes A of the relation W **do**
2: determine the number of distinct values of A
3: **if** A has no hierarchy and many values **then**

```
        remove A                                              ▷ attribute removal
    else
        while number of distinct values of A > attribute threshold do
            replace the values of A by the concepts one level up      ▷ attribute generalization
        end while
    end if
end for
merge identical tuples, accumulate count
while |P| > relation threshold do
    generalise further the attribute with the most distinct values and merge again
end while
return P
```

Example. The raw student table has the attributes *name, birth number, gender, field, country, age, grade*. *Name* and *birth number* are removed (many values, no hierarchy); *field* is generalised (physics, chemistry, biology → science), *country* (Czechia, Slovakia → domestic; otherwise foreign), *age* (21, 22, 23 → 20–25) and *grade* (1, 2 → excellent). Hundreds of rows collapse to a few:

gender	field	country	age	grade	count
M	science	domestic	20–25	excellent	84
F	science	foreign	25–30	excellent	36
M	humanities	domestic	20–25	good	30

Connections that get asked about

AOI vs. OLAP. Generalisation corresponds to the *roll-up* aggregation operation over a data-warehouse cube (the opposite direction being *drill-down*). The difference: OLAP works over a *precomputed* cube and is driven manually by the user, whereas AOI runs *automatically* over a relation and decides itself which attributes to remove and which to generalise.
Characterisation vs. discrimination. Run on the *target class alone*, AOI yields a *characterisation*; run on the *target and the contrasting classes at once* (at the same level of generalisation, so that they are comparable), it yields a *discrimination*. The t-weight and d-weight are then computed over the resulting relation (Sect. 4.3).
Complexity. It is essentially a single pass over the data with aggregation, hence linear in the number of tuples — which is why AOI is usable on large databases.
Forms of presentation: the generalised relation (the table above), a cross-tab, quantitative rules with weights, or charts (pie, bar).

4.3 Characteristic and discriminant rules: t-weight and d-weight

[♦ beyond the slides] This section follows Han–Kamber — the topic names both kinds of rules explicitly.

Both kinds of rules answer two different questions about the *target class* (e.g. clients with a mortgage). **Characterisation:** "what does a typical member of the class look like?" **Discrimination:** "how does it differ from the *contrasting class*?" (clients without a mortgage). Both are read off the same generalised relation produced by AOI (Sect. 4.2): rows $q_1, \ldots, q_N$ with `counts`, for discrimination separately for the target and the contrasting class. Each row gets one number for each of the two questions.

t-weight — typicality (characteristic rule)

Purpose: measures how large a part of the target class the row q_a describes. **Formula:**

$$t\text{-weight}(q_a) = \frac{\text{count}(q_a)}{\sum_{i=1}^{N} \text{count}(q_i)},$$

i.e. the number of target-class objects in the row divided by the size of the whole target class; the sum over the rows is 100 %. **Meaning:** "45 % of clients with a mortgage are 30–40 years old, high-income, from Prague." It appears in the *characteristic rule*, which goes *from* the class to the properties (a necessary condition):

$$\text{target}(X) \Rightarrow \text{cond}_1(X)\,[t\colon w_1] \;\vee\; \cdots \;\vee\; \text{cond}_N(X)\,[t\colon w_N].$$

It says nothing about whether the property *distinguishes* the class from the others.

d-weight — discriminating power (discriminant rule)

Purpose: measures how reliably the row q_a points to the target class rather than to a contrasting one. **Formula:**

$$d\text{-weight}(q_a) = \frac{\text{count}(q_a \in C_t)}{\text{count}(q_a \in C_t) + \sum_{j=1}^{m} \text{count}(q_a \in C_j)},$$

i.e. of all objects (target and contrasting) that satisfy the row, the fraction from the target class — the precision of the row as a classifier. **Meaning:** "90 % of people aged 30–40 with a high income from Prague have a mortgage." It appears in the *discriminant rule*, which goes from the properties *into* the class (a sufficient condition):

$$\text{target}(X) \Leftarrow \text{cond}(X)\,[d\colon w].$$

Always compare it with the *a-priori proportion* of the target class: a d-weight equal to the a-priori proportion means zero discriminating power, a d-weight below it rather excludes the class. The *quantitative description rule* states both weights at once ($\Leftrightarrow$, $[t\colon w,\ d\colon w']$).

Example — clients with a mortgage (200) vs. without one (800)

The generalised relation:

Age	Income	Region	count		t-weight	d-weight
			target	contrast	(target)	
30–40	high	Prague	90	10	**45 %**	**90 %**
30–40	middle	Prague	50	150	25 %	25 %
40–50	high	outside Prague	40	160	20 %	20 %
20–30	low	outside Prague	20	480	10 %	4 %

First row: $t = 90/200 = 45\,\%$ (almost half of all mortgages are here), $d = 90/(90+10) = 90\,\%$ (whoever falls here has a mortgage with 90 %). **The trap:** the 3rd row has a decent $t = 20\,\%$, but $d = 40/200 = 20\,\%$ is exactly the a-priori proportion of mortgages (200/1000) — typical, yet it distinguishes nothing; the 4th row with $d = 4\,\%$ rather excludes a mortgage.

4.4 Extraction of rules from classification trees

The most direct method of rule extraction starts from a classification tree: the nodes of a binary tree test Boolean conditions (typically $x_i \le c$, i.e. cuts perpendicular to the coordinate axes), the input falls left or right according to the outcome of the test, and every leaf corresponds to a single class.

Rules from a tree

- To every root → leaf path corresponds a *conjunction* of the conditions tested along the path; the class of the leaf provides the consequent: $\big(\text{cond}_1 \wedge \cdots \wedge \text{cond}_k\big) \Rightarrow C_j$.
- Paths to leaves of the *same* class are joined by a *disjunction* — the class is the union of the rectangular regions delimited by the individual paths.

Learning the tree is an optimisation of the split in a node with respect to the choice of the variable

and the splitting point (subspaces of continuous inputs, subsets of discrete ones); the two most frequently optimised criteria are the *information gain* $IG(\text{parent}; L, R) = H_{\text{parent}} - \left(\frac{n_L}{n} H_L + \frac{n_R}{n} H_R\right)$ and the *Gini impurity index* $1 - \sum_j p_j^2$; in detail (TDIDT, ID3, C4.5, CART) see Section 3.2. New inputs are classified by passing the node tests; in the leaves, the distribution of the new inputs is assumed to match the distribution of the training data.

Pruning reduces overfitting and at the same time *shortens* the extracted rules (cf. rule post-pruning in C4.5, Section 3.2). **Generalised splits** — a general hyperplane or a quadratic surface instead of an axis-perpendicular cut — improve the approximation of the class boundary but reduce the comprehensibility of the rules obtained as conjunctions of conditions.

4.5 Evolutionary construction of rules

Rules can also be constructed by a genetic algorithm — the computer-science application of the biological evolution of species: crossover and mutation of a string (genome) encoding the rule reinforce the desired properties, here the classification quality measures (accuracy AC, precision PR, TPr, TNr or AUC — Section 4.8). The approach is suitable when both the antecedent and the consequent are conjunctions of a small number of atoms. All evolutionary algorithms are *population-based*; according to what constitutes an individual of the population, two approaches are distinguished:

Approach	Individual	Properties
Michigan	a single rule; the whole population is sought	simple, but the population needlessly converges to similar rules
Pittsburgh	the entire sought set of rules	the result is directly a consistent set; the evolution of a population of sets is more complex and computationally demanding

4.6 Observational rules: confidence, support and hypothesis tests

Classical logic can say about data only "$\varphi \Rightarrow \psi$ holds for *all* objects" (almost never) or "it holds for at least *one*" (almost nothing). The *observational calculus* (Hájek–Havránek 1978, the basis of the Czech GUHA method) therefore replaces the quantifiers $\forall$ and $\exists$ by a *generalised quantifier* $\sim$ meaning "holds for *enough* objects", and defines "enough" statistically. An observational rule $\varphi \sim \psi$ is thus the statement "in the data, φ and ψ occur together sufficiently often", and the individual quantifiers differ only in how they measure "sufficiently".

Four-fold table — the only input of all quantifiers

For a pair φ, ψ four numbers are counted from the data:

	ψ	$\neg\psi$
φ	a	b
$\neg\varphi$	c	d

a = objects satisfying both, b = only φ, c = only ψ, d = neither, $n = a + b + c + d$. Every generalised quantifier is just a condition on a, b, c, d.

Type 1 — confidence and support (association rules)

Purpose: "if φ, then often enough also ψ, and this concerns a large enough part of the data."
Formula:

$$\text{conf} = \frac{a}{a+b} \quad (\text{estimate of } P(\psi \mid \varphi)), \qquad \text{sup} = \frac{a}{n},$$

$$(\varphi \sim \psi) = 1 \iff \text{conf} \geq c_0 \ \wedge \ \text{sup} \geq s_0.$$

Meaning: confidence is the *reliability* of the rule (of the baskets with rolls, how many also contain butter), support is its *significance* (how many baskets it concerns at all). The thresholds

c_0, s_0 are chosen by the user. Weakness: a confidence of 60 % means nothing if 60 % of *all* customers buy butter — the confidence must be compared with $\frac{a+c}{n}$, which leads to type 2.

Type 2 — hypothesis test

Purpose: instead of a fixed threshold we ask whether the joint occurrence of φ and ψ could be *chance*. **Formula:** H_0: φ and ψ are independent, $P(\psi \mid \varphi) = P(\psi)$; H_1: positive dependence, $P(\psi \mid \varphi) > P(\psi)$. The one-sided Fisher test (Section 2.8.1) computes the p-value = the probability that, under independence and with the given marginal sums of the table, a would come out at least as large as observed;

$$(\varphi \sim \psi) = 1 \iff p \leq \alpha \quad \text{(e.g. } \alpha = 0.05\text{)}.$$

Meaning: the rule holds when φ and ψ occur together *demonstrably* more often than chance would explain. The test automatically accounts for both the strength of the dependence and the number of objects: with a small n even a high confidence is not enough.

Example. $n = 1000$ baskets, rolls in 100 of them, butter in 400; together $a = 60$, hence $b = 40$, $c = 340$, $d = 560$. Confidence $60/100 = 60\,\%$, support $60/1000 = 6\,\%$; with thresholds $c_0 = 0.5$, $s_0 = 0.05$ the rule holds. Overall 40 % of customers buy butter, among roll buyers 60 % — under independence we would expect $a \approx 40$, we observed 60, the Fisher test gives $p \ll 0.05$ and the rule holds by type 2 as well.

4.7 The statistical view: regression and discriminant analysis

A characteristic and discriminant description of classes need not take the form of logical rules — classical statistics solves the same two tasks by *regression* and *discriminant analysis*. The context: classification methods are divided according to whether they primarily *seek* the boundary between the classes (neural networks, decision trees, SVM — modern, computationally demanding), or do *not* seek it and start from the experience of similar cases or from the class probabilities (k-NN, Bayesian classification, discriminant analysis — traditional, computationally cheap).

Regression analysis determines the relationship of a variable Y to one or more variables $X_1, \ldots, X_n$: linear regression of one variable by the least squares method, multi-dimensional linear regression, and logistic regression for a categorical output — in detail in Sections 2.9 and 3.2.5. A regression model is the quantitative counterpart of a *characteristic* description: it says which output values are typical for the given inputs.

Discriminant analysis (DA) classifies patterns into *known* classes but does not seek the boundary: to every class C_j it *fits* a probability distribution from a pre-chosen family of densities $F = \{f(\cdot \mid \theta) : \theta \in \Theta\}$ — in Fisher's original DA the multivariate normal distribution. Learning is the maximum likelihood estimation of the parameters θ_j from the training data of the class C_j; a new pattern x is classified by $\arg\max_j p_j\, f(x \mid \theta_j)$, where p_j is the prior probability of the class.

Linear vs. quadratic discriminant analysis

LDA: the covariance matrix Σ is given in advance and shared by all classes; the discriminant function is

$$g_j(x) = \ln p_j - \tfrac{1}{2}\,(x - \mu_j)^\top \Sigma^{-1} (x - \mu_j),$$

only μ_j is estimated (q parameters per class for $x \in \mathbb{R}^q$); the set $\{x : g_j(x) = g_k(x)\}$ between two classes is a *hyperplane*.
QDA: in addition, each class has its own covariance matrix Σ_j (another $\frac{q(q+1)}{2}$ parameters per class); the boundary between the classes is a *quadratic surface*.

For two classes, instead of g_1, g_2 one can seek a single function $g = g_1 - g_2$ and classify by its *sign* — the condition $g(x) > 0$ is the quantitative counterpart of a *discriminant rule* (a sufficient condition

for membership of the first class). For the derivation of the linear and quadratic discriminant functions from normal distributions see Section 3.2.6.

Task	Rule form	Statistical form
characterisation	characteristic rule ($\Rightarrow$, t-weight)	regression analysis
discrimination	discriminant rule ($\Leftarrow$, d-weight)	discriminant analysis

4.8 Measuring the interestingness of rules

Extraction typically yields more rules than the user can bear — the rules have to be measured, sorted and filtered. A rule $\varphi \Rightarrow \psi$ is itself a binary classifier, so the quality measures of binary classification apply to it (Section 5.2): accuracy AC, precision (predictive value) PR, sensitivity (recall) TPr, specificity TNr $= 1 -$ FPr, the F-measure $FM = \frac{2\,PR \cdot TPr}{PR+TPr}$ and AUC. For observational rules they correspond to the *confidence* $\frac{a}{a+b}$ (the precision of the rule) and the *support* $\frac{a}{n}$; for characteristic and discriminant rules, the d-weight is the precision of the rule and the t-weight its recall with respect to the target class.

The roles of the measures in working with rules:

- the *fitness* in the evolutionary construction of rules (Section 4.5);
- *sorting*: the rules for malware families are ordered by precision (Section 4.9);
- *filtering*: removal of redundant and imprecise rules;
- *thresholding*: an observational rule is "interesting" if its confidence and support exceed the chosen thresholds, or if the test rejects independence (Section 4.6) — every choice of a measure and a threshold is a different generalised quantifier, and hence a different set of rules found in the data.

Different costs of different errors. Traditional accuracy and error rate assume that every error matters equally — unrealistically: an undelivered ham matters more than an unrecognised spam, an unrecognised network intrusion more than a false alarm. The generalisation of the error rate is the *mean cost* $\frac{1}{n}\sum_{i,j} c(i,j)\, n_{ij}$, where $c(i,j)$ is the cost of classifying class i as j (the traditional error rate: $c(i,i) = 0$, otherwise 1); the interestingness of a rule is then measured by the cost of the errors it commits.

[♦ beyond the slides: subjective measures following Han–Kamber] Besides the objective measures, *subjective* ones decide: comprehensibility (the whole point of rule extraction), simplicity (the length of the conjunction, the number of rules), unexpectedness (a contradiction with the user's expectation) and actionability (one can act on the rule). In practice: a rule is interesting if it is at the same time comprehensible, valid on new data, potentially useful and novel.

4.9 Applications: spam, malware and network threats, recommending

The three internet applications of classification methods from the 1st lecture of the course illustrate the role of rules:

Spam filtering is a classification {combinations of feature values} $\to$ {spam, ham}: features from the content of the message (words, combinations of words, images) and from the meta-information (domain, encoding, a forged sender address; anomalies of the SMTP session; the same message sent to many recipients). If low FP and FN are not attainable simultaneously, a third class between spam and ham helps — quarantine/uncertainty (e.g. SpamBayes). Rule-based filters work mainly with the meta-information; a plain Boolean disjunction of the rules leads to high false positivity ("graphical body of the message $\Rightarrow$ automatically spam"), the Łukasiewicz fuzzy disjunction of rules with the same consequent works better. And FP (an undelivered ham) matters more than FN.

Malware and network threat detection. Suspicious programs are in the end evaluated by an analyst — who wants to understand *why* they are suspicious, hence rules. Many of them arise, so redundant and imprecise ones are removed; typical is a hierarchical classification with two layers of

rules (the first precise for normal software and sensitive for malware, the second sorting malware families, ordered by precision). For network threats (R2L, U2R, DoS and probe attacks), the counterpart of rules are the *signatures* of known attacks — patterns with a certain (one hundred per cent) conclusion.

Recommender systems. In collaborative filtering by the user's behaviour, rules of the form "the user visited the pages $S_1, \ldots, S_n \Rightarrow$ recommend the object O (more generally: offer the next page S_d)" are used. They are obtained by a statistical analysis of clickstream data — probability estimates and tests of the validity of models (hypotheses), from whose results the rules are constructed heuristically; they are thus observational rules of both types from Section 4.6. Recommender systems often explain *why* they recommend an object — the explanation increases the comprehensibility of the recommendation for the user.

4.10 Summary for the examination

- **Why rules**: a classifier $f\colon X \to \{C_1, \ldots, C_m\}$ is a mathematical prescription, hardly comprehensible; a rule of logic ($\varphi \Rightarrow \psi$, $\varphi \Leftrightarrow \psi$) is more comprehensible. The Boolean implication is true at $t(\varphi) = 0$ or $t(\varphi) = t(\psi) = 1$; the fuzzy implication = the residuum of a t-norm, $t(\varphi) \leq t(\psi) \Rightarrow$ the implication is true to degree 1.
- **AOI — attribute-oriented induction**: lifts the data to a higher conceptual level following *concept hierarchies* and only then looks for regularities. Two operations: attribute *removal* (many values and no hierarchy — name, birth number) and attribute *generalization* (values → concepts one level up), with identical tuples merged after each step and a `count` accumulated. When to stop: a threshold on the number of distinct values of an *attribute* (2–8) and a threshold on the number of tuples of the *relation* (10–30); thresholds that are too low lead to **over-generalisation** ("all clients are people"). It corresponds to the *roll-up* operation in OLAP but runs automatically over a relation. Run on the target class it yields a *characterisation*, run on the target and the contrasting classes a *discrimination*.
- **Characteristic vs. discriminant rule** (Han–Kamber): characterisation describes the target class ($\Rightarrow$, a necessary condition, **t-weight** = $\frac{\mathrm{count}(q_a)}{\sum_i \mathrm{count}(q_i)}$); discrimination distinguishes it from the contrasting ones ($\Leftarrow$, a sufficient condition, **d-weight** = the fraction of the covered objects belonging to the target class). Be able to compute both from a table; always compare the d-weight with the *a-priori* class proportion.
- **Extraction from a tree**: a root → leaf path = a conjunction of conditions, paths to leaves of the same class = a disjunction; learning by splitting according to the information gain or the Gini index; pruning shortens the rules; generalised (oblique) splits reduce comprehensibility.
- **Evolution of rules**: a genetic algorithm, fitness = AC/PR/TPr/TNr/AUC; the Michigan approach (individual = a rule) vs. the Pittsburgh approach (individual = a set of rules).
- **Observational rules** (Hájek–Havránek 1978, GUHA): $\varphi \sim \psi$ with a generalised quantifier over a four-fold table (a, b, c, d); (1) probability estimates — association rules, $\frac{a}{a+b} \geq c_0$ (confidence) $\wedge\ \frac{a}{n} \geq s_0$ (support); (2) hypothesis tests — the one-sided Fisher test rejects independence in favour of positive dependence.
- **Statistically**: regression analysis = a characteristic description of the relationship of Y to $X_1, \ldots, X_n$ (least squares, multi-dimensional, logistic); discriminant analysis = classification into known classes by fitting distributions: ML estimation of θ_j, prediction $\arg\max_j p_j f(x \mid \theta_j)$; **LDA** (shared Σ, hyperplane boundary, q parameters) vs. **QDA** (per-class Σ_j, quadratic boundary, $+\frac{q(q+1)}{2}$ parameters); two classes → the sign of $g = g_1 - g_2$.
- **Interestingness**: a rule = a binary classifier → AC, PR (= confidence), TPr (recall), TNr, F-measure, AUC; roles: fitness, sorting (malware by precision), filtering out redundant rules, thresholding (confidence and support = a generalised quantifier); different costs of errors (the mean cost). Subjectively: comprehensibility, simplicity, unexpectedness, actionability.
- **Applications**: spam (meta-information, the 3rd class quarantine, the Łukasiewicz disjunction against FP), malware (a two-layer hierarchy of rules), network threats (signatures), recommend-

ing (rules from clickstream data).

5 Representation, evaluation and visualisation of knowledge

The chapter has three parts, matching the three words in the sentence of the topic: *in what form* we express the knowledge, *how we verify* that it is good, and *how we display* it. The middle part — the evaluation of classifiers — is the only one the slides cover in full, and it is at the same time the one most frequently asked about in the examination.

5.1 Forms of knowledge representation

[♦ beyond the slides: the table as an overview; the individual methods in it are backed by the slides]

Form	Typical source	Properties
IF-THEN rules	association rules, C4.5	the most comprehensible; local (each rule is read on its own); a risk of conflicts and overproduction
Decision tree	ID3/C4.5/CART	a global model, easily readable up to depth 4–5; unreadable beyond
Decision table	discretised data	a complete enumeration of the combinations; readable only for few attributes
Mathematical function	regression, SVM, NN	compact and accurate; hard to interpret (except a linear model with normalised weights)
Prototypes / centroids	k-means, k-medoids, LVQ	the "typical representative" of a group; very intuitive for business
Probabilistic model	naive Bayes, Bayesian network	explicit uncertainty; the network graph is readable
Graphs and networks	social network analysis	relations between entities; visually strong
Instances (case memory)	k-NN, CBR	the "model" is the data itself; explanation by reference to similar cases

Interpretability and explainability

[♦ beyond the slides: explainability and the post-hoc methods; interpretability as a criterion for evaluating a model is in the slides — see Section 5.2] *Interpretability* — a human's ability to understand how the model arrived at its output. *Explainability* — the ability to give a comprehensible justification of a concrete decision.

We distinguish *inherently interpretable models* (linear models, decision trees, rule sets, prototypes) and *post-hoc explanations of black boxes* — attribute importance (permutation, SHAP), local surrogate models (LIME), partial dependence plots (PDP), rule extraction from a neural network, sensitivity analysis, counterfactual explanations ("had your income been 5000 higher, the loan would have been approved").

Interpretability is not merely a convenience: in regulated domains (loans, insurance, medicine) it is a legal requirement (GDPR Art. 22 — the right to an explanation of an automated decision; the AI Act for high-risk systems).

5.2 Evaluation of models

5.2.1 Criteria for evaluating classification methods

- **Predictive accuracy** — the fraction of correctly classified patterns:

$$Accuracy = \frac{\text{number of correct classifications}}{\text{number of all test cases}}.$$

- **Efficiency** — the time to construct the model and the time to use it.
- **Robustness** — handling noise and missing values.
- **Scalability** — efficiency for disk-resident databases.
- **Interpretability** — the clarity and insight provided by the model.
- **Compactness of the model** — the size of the tree, the number of rules, ...

5.2.2 Methods of error estimation

Holdout, cross-validation, leave-one-out, validation set

Holdout set: the available set D is divided into two disjoint subsets — the *training* set D_{train} (for learning the model) and the *test* set D_{test} (for testing it). **The essential rule:** the training set must not be used in testing and the test set must not be used in learning — only an unseen test set provides an *unbiased* estimate of accuracy. Used mainly for a *large* D.
k-fold cross-validation: the data is partitioned uniformly into k equal-size disjoint subsets. Each subset is used as a test set and the remaining $k-1$ are combined as the training set. The procedure is run k times, yielding k accuracies $Accuracy_i$; the final estimate is their average

$$Accuracy_{CV} = \frac{1}{k}\sum_{i=1}^{k} Accuracy_i.$$

10-fold and 5-fold cross-validation are common; the method is used rather for *smaller* data.
Leave-one-out: a special case of cross-validation where each fold has only a single test example and all the rest is used in training — for m examples this is m-fold cross-validation. Used for *very small* data.
Validation set: the data is divided into *three* subsets — a training, a validation and a test set. The validation set is used for estimating the *parameters* of learning algorithms: the values that give the best accuracy on it are used as the final parameter values. Cross-validation can be used for parameter estimation as well.

Confidence interval for cross-validation

The approximate $N\%$ confidence interval:

$$Accuracy_{CV} \pm t_{N,k-1} \cdot s_{Accuracy}, \qquad s_{Accuracy} = \sqrt{\frac{1}{k(k-1)}\sum_{i=1}^{k}\left(Accuracy_i - Accuracy_{CV}\right)^2}.$$

Values of $t_{N,k-1}$ for two-sided confidence intervals:

	confidence level N		
k	90 %	95 %	99 %
2	2.92	4.30	9.92
10	1.81	2.23	3.17
30	1.70	2.04	2.75
∞	1.64	1.96	2.58

Often $t = 1.96$ is used (for $k \to \infty$ and $N = 95\,\%$).

5.2.3 The confusion matrix and the derived measures

Accuracy is only *one* measure ($error = 1 - accuracy$) and in some applications it is not suitable. In text mining we may be interested only in the documents of a particular topic, which are a small portion of a big collection. In classification involving strongly *imbalanced* data (network intrusion, financial fraud detection) we are interested only in the minority class: with 1 % intrusions we achieve 99 % accuracy *by doing nothing* — and detect no intrusion at all. The class of interest is called the *positive* class, the rest *negative*.

Confusion matrix, precision, recall, F_1

		classified as positive	classified as negative
actual	positive	TP	FN
	negative	FP	TN

TP — the number of correct classifications of the positive examples (true positive); FN — incorrect classifications of positive examples (false negative); FP — incorrect classifications of negative examples (false positive); TN — correct classifications of negative examples.

$$p = precision = \frac{TP}{TP + FP}, \qquad r = recall = \frac{TP}{TP + FN}.$$

Precision p is the number of correctly classified positive examples divided by the total number of examples classified as positive. *Recall* r is the number of correctly classified positive examples divided by the total number of actual positive examples in the test set. **Both measures evaluate the classification only on the positive class.**
The F_1 **measure** combines both into one — it is their *harmonic mean*:

$$F_1 = \frac{2pr}{p + r} = \frac{2}{\frac{1}{p} + \frac{1}{r}}.$$

The harmonic mean of two numbers tends to be closer to the *smaller* of the two, so F_1 is large only if *both* measures are large.

An illustration of the trap: a classifier that correctly labels a single positive example and labels no negative one wrongly has $p = 100\,\%$ and $r = 1\,\%$. High precision alone therefore says nothing about usefulness.

5.2.4 Scoring, ranking and lift analysis

Scoring is related to classification: we are interested in a single (positive) class — e.g. the buyers class in a marketing database. Instead of assigning each test instance a definite class, scoring assigns a *probability estimate* (PE) that it belongs to the positive class. Classification systems can be used for scoring, but they must produce a probability estimate — e.g. in decision trees the confidence value at each leaf node can be used as the score.

After scoring, the examples are *ranked* by their PE and divided into n (say 10) bins; a **lift curve** is drawn according to how many positive examples are in each bin — this is *lift analysis*.

Example — lift analysis for a targeted campaign

We want to send promotion materials to potential customers (a watch). Each package costs \$0.50 to send; if a watch is sold, we make \$5 profit. How many packages should we send, and to whom?
The test set has 10 000 instances, out of which 500 are positive. After scoring and ranking into 10 bins (1000 instances each):

Bin	1	2	3	4	5	6	7	8	9	10
Positives	210	120	60	40	22	18	12	7	6	5
% of total	42	24	12	8	4.4	3.6	2.4	1.4	1.2	1.0
Cumulative	42	66	78	86	90.4	94	96.4	97.8	99	100

The first bin contains 42 % of all positive cases, the first three 78 %. Random mailing would give 10 % in each bin — the gap between the lift curve and this diagonal is the model's contribution. The decision "how many packages" then follows from the economics: mailing to the first bin earns $210 \cdot \$5 - 1000 \cdot \$0.50 = \$550$, to the tenth $5 \cdot \$5 - 1000 \cdot \$0.50 = -\$475$.

5.2.5 The ROC curve and AUC

The ROC curve (Receiver Operating Characteristic)

A plot of the *true positive rate* (TPR) against the *false positive rate* (FPR):

$$TPR = \frac{TP}{TP + FN}, \qquad FPR = \frac{FP}{FP + TN}, \qquad TNR = \frac{TN}{FP + TN}.$$

TPR is the fraction of actual positive cases that are correctly classified — it is essentially the *recall* of the positive class, also called *sensitivity*. FPR is the fraction of actual negative cases classified as positive. TNR (the recall of the negative class) is called *specificity*, and $FPR = 1 -$ specificity. Each ROC curve starts at $(0, 0)$ (every case classified as negative) and ends at $(1, 1)$ (every case classified as positive). **The main diagonal** represents random guessing — predicting each case to be positive with a fixed probability, when $TPR = FPR$.

Drawing a ROC curve given the classification as a ranking of test cases:

1. Obtain the ranking by sorting the test cases in *decreasing* order of the classifier's output values (e.g. posterior probabilities).
2. Partition the rank list into two subsets *at every* test case, and regard every test case in the upper part as a positive case and every one in the lower part as negative.
3. For each such partition, compute a pair (FPR, TPR). When the upper part is empty we obtain the point $(0, 0)$; when the lower part is empty, the point $(1, 1)$.
4. Connect the adjacent points.

AUC — the area under the ROC curve

Two classifiers may *cross* in the ROC space: in the example from the slides, C_1 is better for $FPR < 0.43$ and C_2 is better for $FPR > 0.43$ — so there is no unambiguous answer to "which one is better". Overall, the *area under the ROC curve* (AUC) can be used: if $\text{AUC}(C_i) > \text{AUC}(C_j)$, we say that C_i is better than C_j. A perfect classifier has AUC $= 1$, random guessing AUC $= 0.5$.

5.3 Visualisation techniques

[♦ beyond the slides: the table as an overview; the box plot, dendrogram and ROC/lift curve themselves are in the slides]

Technique	Used for	Limitation / note
Histogram	the distribution of one numeric quantity	sensitive to the choice of bin width
Box plot	median, quartiles, outliers; comparison of groups	does not show bimodality (the violin plot does)
Scatter plot	the relation of two numeric quantities	with many points, overplotting → transparency, hexbin
Scatter plot matrix (SPLOM)	all pairs of attributes at once	usable up to roughly 10 attributes
Parallel coordinates	high-dimensional data — an object is a polyline across vertical attribute axes	strongly dependent on the axis order; scaling required; crossing lines = negative correlation
Heatmap	correlation matrix, confusion matrix	the choice of the colour scale (diverging for correlations)
Dendrogram	the result of hierarchical clustering	the leaf order is not unique
Projections (PCA, MDS, t-SNE, UMAP)	a 2-D view of high-dimensional data	t-SNE/UMAP distort global distances — the *sizes* of clusters and the gaps between them must not be interpreted
ROC / lift curve	the quality of a classifier	see Section 5.2
Mosaic plot	a contingency table of categorical attributes	a visual alternative to χ^2
Network visualisation	social networks, relation graphs	a "hairball" for large graphs → aggregate the communities

Three roles of visualisation in KDD: *(1) exploratory analysis* (the data understanding phase) — it reveals distributions, outliers, missing values, unexpected relations; Anscombe's quartet shows four data sets with identical means, variances and correlations but completely different structure, which only a plot reveals; *(2) model visualisation* — a tree, a dendrogram, a network, decision boundaries; *(3) presentation of the results* to a manager — aggregation, simplicity, one chart = one message.

Principles of good visualisation: a high data-to-ink ratio (Tufte); do not distort (a zero axis for bar charts, linear axes or clearly labelled logarithmic ones); choose the colour according to the type of data (categories → a qualitative palette, order → sequential, deviation from a centre → diverging) and with regard to colour blindness; always label and give units; avoid 3-D effects and pie charts with many slices.

5.4 Evaluation of knowledge from the user's point of view

The notion of "evaluating the acquired knowledge" has three different levels that must be kept apart:

- *statistical evaluation of the model* — accuracy, confusion matrix, ROC/AUC, error estimation by cross-validation (Section 5.2);
- *evaluation of the interestingness of individual patterns* — support, confidence, lift, χ^2, t-weight and d-weight (Section 3.1 and Chapter 4);
- *evaluation from the user's point of view* — whether the knowledge found is comprehensible, valid, useful and novel.

The first two levels are objective and measurable, the third is necessarily subjective — the same pattern may be a discovery for one user and common knowledge for another. Part of the results may be uninteresting or obvious from the user's point of view, part can be used directly and part has to be presented more comprehensibly. It is useful to summarise the results in an *analytical report*; the output may also be the execution of a reasonable action.

5.5 Summary for the examination

- **Forms of knowledge representation**: IF-THEN rules (the most comprehensible, local), decision tree (global, readable up to depth 4–5), decision table, mathematical function (compact, not interpretable), prototypes/centroids, probabilistic model, graphs and networks, instances (k-NN, CBR). Be able to say, for each form, which method produces it and what its comprehensibility costs.
- **Criteria for evaluating methods**: predictive accuracy, efficiency (time to construct and to use), robustness, scalability, interpretability, compactness of the model.
- **Error estimation**: *holdout* (large data; the training and test sets must not be mixed), *k-fold cross-validation* (smaller data; 5 and 10 are common; accuracy as the average of k accuracies + a confidence interval with $t_{N,k-1}$), *leave-one-out* (very small data, m-fold CV), *validation set* (a third part of the data, on which the *parameters* of the algorithm are tuned).
- **Confusion matrix**: TP, FN, FP, TN; $p = \frac{TP}{TP+FP}$, $r = \frac{TP}{TP+FN}$, $F_1 = \frac{2pr}{p+r}$ (the harmonic mean — closer to the smaller of the two). Be able to explain *why* accuracy is not enough for imbalanced data (1 % intrusions ⇒ 99 % accuracy by "doing nothing").
- **ROC**: $TPR = \frac{TP}{TP+FN}$ (sensitivity = the recall of the positive class) against $FPR = \frac{FP}{FP+TN}$; TNR = specificity = $1 - FPR$. The curve runs from $(0, 0)$ to $(1, 1)$, the diagonal = random guessing. Be able to describe the construction from a ranking (partition at every case). **AUC**: 1 for a perfect and 0.5 for a random classifier; needed because the ROC curves of two classifiers may *cross*.
- **Scoring and lift**: instead of a class, a probability estimate is assigned (for a tree, the confidence at the leaf), the examples are ranked and divided into bins; the lift curve is compared against the diagonal of random selection. Be able to carry out the economic reasoning (how many packages it pays to send).
- **Three levels of "evaluation"**: the statistical quality of the model, the interestingness of the patterns, and the user's evaluation. The first two are objective, the third subjective. **Four criteria of interesting knowledge**: comprehensibility, validity, usefulness, novelty.
- **Visualisation**: the technique according to the task (histogram, box plot, scatter, SPLOM, parallel coordinates, heatmap, dendrogram, projections, ROC/lift); three roles in KDD (exploration — Anscombe's quartet, model visualisation, presentation); the principles (Tufte, do not distort axes, colour by data type). Know the limitation: t-SNE and UMAP distort global distances.

6 Models for social network analysis, centrality and community detection

The sentence of the topic names three things: *models* for social network analysis, *centrality measures*, and *community detection*. The chapter is structured accordingly: first the representation of networks and the measures at the actor level (centrality) and the network level (cohesion), then the models — of the properties of real networks, of influence spreading, of homophily, segregation and structural balance, plus link analysis (HITS, PageRank) — and finally community detection and link prediction.

6.1 Delimitation of the field and motivation

Social network analysis

Social network analysis (SNA) is an interdisciplinary field with its origin in three areas:

- *social science* — questions and problems that explain social behaviour through an analysis of societies;
- *network analysis* — the formulation and solution of problems that have a network structure;
- *graph theory* — the mathematical apparatus for representing and analysing graphs.

A fundamental shift of perspective: SNA focuses on the **relationships** between social entities (individuals, groups, institutions) *rather than on the entities themselves* and their attributes. Studying society from a network perspective considers individuals as *embedded* in a network of relations and seeks explanations for social behaviour in the structure of these networks rather than in the individuals alone.

Link analysis as motivation: the objectives are to find relationships among the data and to visualise the links. Application areas: telecommunications; criminalistics and law (groups of criminals are mutually interlinked, and an analysis of these relationships can help uncover them); marketing (relationships among customers).

Six degrees of separation (Milgram, 1967): an experiment on the small-world phenomenon. Random people from Nebraska were asked to send a letter (via intermediaries) to a stockbroker in Boston; they could only send it to someone they knew on a first-name basis. Among the letters that found the target, the average number of links was *six*. However, not many of the letters arrived. The experiment presaged some later discoveries — hubs, the tendency to go by geography or profession. The *Erdős numbers* confirm this: Paul Erdős wrote more than 1500 papers with 507 co-authors, and the average Erdős number among mathematicians is 4.65, the maximum 13.

Scale-free (SF) networks as a second motivation: some nodes have extremely many connections (*hubs*), most nodes have just a few. The consequence is *resilience to a random disturbance* together with *vulnerability to coordinated attacks*: 80 % of randomly chosen nodes can fail without leading to the fragmentation of the cluster, but the elimination of 5–15 % of all the *hubs* can cause the entire system to collapse. Applications: protection against computer viruses spread through the internet (the eradication of internet viruses is essentially impossible in an SF architecture), medicine (vaccination campaigns focused on hubs; new drugs targeting the key molecules), business (cascaded financial crashes; marketing — the spread of "contagion", influencer marketing).

Examples of SF networks: *social* (scientific collaboration — researchers co-authoring papers; Hollywood — actors playing roles in the same movie), *biological* (cell metabolism, protein regulatory networks), *socio-technical* (the internet — routers and connections; the World Wide Web — web pages and URLs).

6.2 Representing networks

Basic representations

A network is a graph $G = (V, E)$: the *vertices* (nodes) are actors, the *edges* (links) are the relations between them. Three equivalent notations:

- **a drawing of the graph**;
- **an edge list** — pairs of vertices, plus a column of weights for weighted graphs;
- **an adjacency matrix** $A = (a_{ij})$, where $a_{ij} = 1$ if vertex i is linked to vertex j and 0 otherwise; for weighted graphs it contains the weights.

For **directed** graphs (who contacts whom) the adjacency matrix *does not have to* be symmetric; for **undirected** ones (who knows whom) it is symmetric. The same edge list has a different interpretation in the two cases.

Edge weights can express: the frequency of interaction within the period of observation; the number of items exchanged; individual perceptions of the strength of a relationship; costs in communication or exchange (e.g. distance); combinations of the above. Edges can represent interactions, flows of information or goods, similarities and affiliations, or social relations.

Ego networks and "whole" networks. An *ego network* is the personal network of one actor (the *ego*) and their neighbours (the *alters*); the ego is *not* needed for further analysis, as all alters are by definition connected to the ego. A "whole" network also contains *isolates*. **No studied network is "whole" in practice** — it is usually a partial picture of the real-life network (the *boundary specification problem*).

6.3 Tie strength

Social media connect users to each other extremely easily — a user might have thousands of "friends" online — but the connections are not of the same strength and play different roles in community formation and information diffusion. *Strong ties* are close friends, *weak ties* acquaintances. **The strength of weak ties** (Granovetter, 1973): occasional encounters with distant acquaintances can provide important information about new opportunities (e.g. in a job search).

Homophily, transitivity, bridging

Homophily — the tendency to relate to people with similar characteristics (status, beliefs, ...). It leads to homogeneous groups (clusters) where forming relations is easier; *extreme* homogenisation, however, can act counter to innovation and idea generation, so *heterophily* is desirable in some contexts. Homophilous ties can be strong or weak.

Transitivity — "if there is a tie between A and B and a tie between B and C, then A and C will also be connected in a transitive network." Strong ties are more often transitive than weak ties, so transitivity *indicates* the existence of strong ties (though it is neither a necessary nor a sufficient condition). Transitivity and homophily together lead to the formation of *cliques* (fully connected clusters).

Bridging — a *bridge* is a node or an edge that connects different groups; usually these are weak ties, but not every weak tie is a bridge. Bridges facilitate inter-group communication, increase social cohesion and help spur innovation.

Learning tie strength from the network topology. Bridges connecting two different communities are weak ties; an edge is a *bridge* if its removal results in the disconnection of its terminal nodes. Bridges are relatively rare in real-life networks $\Rightarrow$ the definition is *relaxed* to a **shortcut bridge**: one checks whether the distance between the two terminal nodes *increases* (to more than 2) if the edge is removed. The larger the distance, the *weaker* the tie. E.g. $d(2,5) = 4$ if $(2,5)$ is removed, but $d(5,6) = 2$ if $(5,6)$ is removed $\Rightarrow$ $(5,6)$ is a stronger tie than $(2,5)$.

Neighbourhood overlap

For two nodes i, j connected by a tie — the larger the overlap, the *stronger* the tie:

$$O(i,j) = \frac{\text{number of shared friends of both } i \text{ and } j}{\text{number of friends adjacent to at least } i \text{ or } j} = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2},$$

where the -2 in the denominator excludes i and j themselves.
Examples: $O(2,5) = 0$; $O(5,6) = \frac{|\{4\}|}{|\{2,4,5,6,7,10\}|-2} = \frac{1}{4}$; for the graph $G = (\{1,2\}, \{(1,2)\})$ we get $O(1,2) = 0$.

Learning tie strength from profiles and interactions. On Twitter (one can follow others without their confirmation), the real friendship network is determined by the *frequency* with which two users talk to each other rather than by the follower–followee network — and it is this real friendship network that is more influential in driving Twitter usage. On Facebook, tie strength can be predicted from various information (friend-initiated posts, messages exchanged in wall posts, the number of mutual friends). Numeric link strength can be learned e.g. by maximum likelihood estimation: user profile similarity determines the strength and the link strength in turn determines user interaction ⇒ maximise the likelihood based on the observed profiles and interactions.

6.4 Key players: centrality measures

Degree centrality

The node's in- and out-degree = the number of links that lead into / out of the node. In undirected graphs the in- and out-degrees are identical.
It measures the node's *connectedness*, *influence* and/or *popularity*; it helps assess which nodes are central with respect to spreading information and influencing others in their "neighbourhood".

Paths and shortest paths

A *path* between two nodes is any sequence of non-repeating nodes that connects them. The *shortest path* connects the two nodes with the shortest number of edges (the *distance* between the nodes). Shorter paths are preferred for quick communication or exchange — but not always, for instance when spreading disease.
Shortest path algorithms. *Single source:* general graphs (directed, negative edge weights allowed) — the Bellman–Ford algorithm, $O(|V||E|)$; graphs without negative edge weights — Dijkstra's algorithm, $O(|V|^2)$; to find the shortest path between only two vertices the algorithm can stop once it is found. *All pairs:* negative edge weights allowed — the Floyd–Warshall algorithm, $O(|V|^3)$; sparser graphs with negative weights — Johnson's algorithm, $O(|E||V| + |V|^2 \log_2 |V|)$; without negative weights — Dijkstra, $O(|V|^3)$.

Betweenness centrality

The number of shortest paths passing through a node, divided by all shortest paths in the network: for a given node v, calculate for each pair s, t the number of shortest paths between s and t that pass through v, divide by all shortest paths between s and t, and sum the computed values over all node pairs; it is sometimes normalised so that the highest value is 1.
It indicates the nodes that are on *communication paths* between other nodes, and helps determine the points where the network might *break apart*.

Closeness centrality

A measure of *reach* — the speed with which information can reach other nodes from a given starting node: calculate the *mean* length of all shortest paths from the node to all other nodes in the network (how many hops it takes on average to reach every other node) and take the

reciprocal. Higher values indicate higher closeness (they are "better"). Useful when the speed of dissemination is of main concern.

Eigenvector centrality

It is proportional to the sum of the eigenvector centralities of all nodes directly connected to the node considered: a node with a high eigenvector centrality is connected to other nodes with a high eigenvector centrality. The metric is based on the idea that the power and status of an actor are *recursively* defined by the power and status of their neighbours; it measures how well an actor is connected to other *well-connected* actors.
Formally: for $G = (V, E)$ with adjacency matrix $A = (a_{i,j})$,

$$x_i = \frac{1}{\lambda} \sum_{j \in M(i)} x_j = \frac{1}{\lambda} \sum_{j \in V} a_{i,j} x_j \quad \iff \quad Ax = \lambda x,$$

where $M(i)$ is the set of neighbours of i. There are many eigenvalues λ for which a non-zero eigenvector solution exists; the additional *requirement that all the entries be non-negative* implies, however, that only the *greatest* eigenvalue results in the desired centrality measure. The eigenvector is defined only up to a common factor, so normalisation is needed (e.g. the sum over all vertices being 1 or $|V|$).
It is closely related to Google's PageRank (links from highly linked-to pages count more, Section 6.14).

Measure	Interpretation in social networks
Degree	How many people can this person reach *directly*? — *Network of music collaborations:* how many people has he or she collaborated with?
Betweenness	How likely is this person to be on the most direct route in the network? — *Network of spies:* through whom is most of the information likely to flow?
Closeness	How fast can this person reach everyone in the network? — *Network of sexual relations:* how fast will an STD spread to the rest of the network?
Eigenvector	How well is this person connected to other well-connected people? — *Network of citations:* who is the author cited most by other well-cited authors?

Sets of key players. Individual measures are not enough. In the example from the slides, node 10 is the most central according to degree centrality, but nodes 3 and 5 *together* reach more nodes; moreover, the tie between 3 and 5 is *critical* — if severed, the network breaks into two isolated sub-networks. Other things being equal, nodes 3 and 5 *together* are therefore more important to this network than node 10 alone. *Thinking about sets of key players can help.*

Structural equivalence

It expresses the *similarity* between actors in a social network; it is based on the neighbours they share (equivalently, on the number of identical ties they have). Two actors are *structurally equivalent* if their positions can be swapped *without modifying the overall structure* of the network. Approximate structural equivalence can be used for hierarchical clustering of social networks.

6.5 Cohesion: measures at the network level

Reciprocity, density, clustering coefficient

Reciprocity — the ratio of *reciprocated* relations (an edge in both directions) over all relations in the network; **it makes sense only in directed graphs**.
Density — the ratio of the number of edges over the number of *possible* edges:

$$\text{density} = \frac{|E|}{|V|(|V|-1)/2} \quad \text{(undirected graph)}, \qquad \frac{|E|}{|V|(|V|-1)} \quad \text{(directed)}.$$

A *clique* (a perfectly connected network) has density 1; directed graphs have half the density, as there are twice as many possible edges.
Clustering coefficient of node i — the density of its *neighbourhood* $N_i = \{v_j : e_{ij} \in E \vee e_{ji} \in E\}$, $k_i = |N_i|$:

$$C_i = \frac{2\,|\{e_{jk} : v_j, v_k \in N_i,\ e_{jk} \in E\}|}{k_i(k_i-1)} \text{ (undirected)}, \qquad C_i = \frac{|\{e_{jk}\}|}{k_i(k_i-1)} \text{ (directed)}.$$

The network coefficient $C = \frac{1}{|V|}\sum_i C_i$ indicates the presence of (sub)communities.

Diameter, average distance, eccentricity, small worlds

The network's **diameter** = the longest shortest path between two nodes; the **average distance** = the average of all shortest paths; the **eccentricity** of a vertex = its greatest geodesic distance from another vertex.
A **small world** — a network with a *significantly high clustering coefficient* and a *short average path length*: clusters arise from the transitivity of strong ties, short paths are provided by weak ties (bridges) between the clusters. Very common in social networks.

6.6 A model of real-world complex networks and their properties

Six characteristic properties of real-world complex networks

Real-world complex networks are *non-random and non-regular* graphs with unique features. Examples: social networks, information or knowledge networks, technological networks, biological networks. The properties:

1. **The small-world effect** — Stanley Milgram was the first to point out the existence of small-world effects in real social networks (the experiment of the 1960s, about 300 participants; the median path length of the successful paths was 6). It rests on the concept of "six degrees of separation": everyone is, on average, six steps away from any other person. There are important consequences for the speed at which information and diseases spread.
2. **Transitivity or clustering** — in real-world, especially social networks there is a high probability of finding *complete sub-networks* within larger networks (everyone knows everyone).
3. **Power-law degree distributions** — the *degree distribution* is the probability distribution of the degrees of nodes over the whole network (a histogram of the fraction of nodes with degree k). Real networks have degree distributions quite different from those of random graphs.
4. **Network resilience** — the impact on the connectivity of the network when one or more nodes are removed. Most networks are *robust* against random vertex removal, but considerably less robust to the *targeted* removal of the highest-degree vertices. Also, when *gatekeepers* are deleted, there are significant changes in the network's communication ability, since some nodes become disconnected. The removal of a single node is usually not cause for alarm — it is more appropriate to test the resilience by removing a certain *percentage* of nodes.
5. **Mixing patterns** — in networks where different types of nodes coexist, it is common to observe a certain *selectivity* in the establishment of connections. Selective linking is called *assortative mixing*, or homophily; a classic example is mixing by race. Real networks show higher tendencies for assortative mixing.
6. **Community structure** — most real social networks show community structure, which arises as a consequence of both *global and local heterogeneity* of the edges' distribution in the graph: we often find high concentrations of edges within some areas of the graph (*communities*) and low concentrations between those regions.

Scale-free networks and preferential attachment

Scale-free networks (Barabási, Albert, 1999) have a *power-law* degree distribution (the WWW, citation and biological networks): a few highly connected *hubs* and a large number of low-degree nodes. They arise from two mechanisms: **growth + preferential attachment** — new nodes tend to connect to already well-connected ones (*rich-get-richer*, the *Matthew effect*, Price's *cumulative advantage*). The reason may be *popularity* (the rich get richer), *quality* (the good get better), or a mixed model — among similar nodes, those that reach critical mass first become stars (the *halo effect*). The resulting network has a long-tailed degree distribution and a small-world structure.

Core-periphery structures and centralisation

Centralisation — from the differences in node degrees (normalised to $[0, 1]$) it measures how much the network depends on a few hubs: centralised networks are better at coordination but *more prone to failure* if the key players disconnect. Many large communities have a densely interconnected *core* critical for connecting a much larger *periphery*; the cores are found visually, by the location of high-degree nodes, or by *bow-tie analysis* (the structure of the Web).

6.7 Influence modelling

Common properties of influence modelling methods

- the social network is represented as a *directed* graph, with each actor being one node;
- each node starts as *active* or *inactive*;
- a node, once activated, will activate its neighbouring nodes;
- **once a node is activated, it cannot be deactivated.**

The Linear Threshold Model (LTM)

An actor (node) would take an action if the number of their friends who have taken the action exceeds a certain *threshold*.

- Each node v chooses a threshold θ_v *randomly* from a uniform distribution on $[0, 1]$.
- In each discrete step, all nodes that were active in the previous step remain active.
- The nodes satisfying

$$\sum_{u \in N_v^{\text{active}}} w_{u,v} \geq \theta_v \sum_{u \in N_v} w_{u,v}$$

  will also be activated.

LTM focuses on the *receivers* of the influence.

The Independent Cascade Model (ICM)

It focuses on the *senders* rather than the receivers.

- A node v, once activated at step t, has *one* chance to activate each of its neighbours, randomly.
- For a neighbouring node u the activation succeeds with probability $p_{v,u}$ (e.g. $p = 0.5$).
- If the activation succeeds, u becomes active at step $t + 1$.
- In the subsequent rounds, v will *not* attempt to activate u any more.
- The diffusion process starts with an initial activated set of nodes and continues until no further activation is possible.

Influence maximisation

The task: given a network and a parameter k, which k nodes should be selected to be in the activation set B in order to maximise the influence in terms of active nodes at the end? Let

$\sigma(B)$ denote the expected number of nodes influenced by B:

$$\max_{|B|\leq k} \sigma(B).$$

The problem is **NP-hard**, but it can be proved that the *greedy* approach yields solutions that are **63 %** of the optimal.
The greedy approach: start with $B = \emptyset$; evaluate $\sigma(\{v\})$ for each node and pick the node with maximum σ as the first element of B; then repeatedly select the node that will increase $\sigma(B)$ the most, i.e. $\arg\max_{v \in V \setminus B} \sigma(B \cup \{v\})$.

6.8 Influence versus correlation

User attributes and behaviours quite often tend to correlate with their social networks. Suppose a binary attribute is associated with each node (e.g. whether or not the person is a smoker): if the attribute is correlated with the network, the actors are expected to share the same attribute values, positively correlated with the social connections (smokers are more likely to interact with other smokers).

Test for correlation

If the fraction of edges linking nodes with *different* attribute values occurs significantly *less* often than the expected probability, then there is evidence of correlation.
Example: if connections are independent of the smoking behaviour, the fraction of smokers is $p = 4/9$ and of non-smokers $1-p = 5/9$. One edge connects two smokers with probability $p \cdot p$, two non-smokers with $(1-p)(1-p)$, and a smoker and a non-smoker with $2p(1-p) = 2 \cdot \frac{4}{9} \cdot \frac{5}{9} = 49\,\%$. In the actual graph this fraction is $2/14 = 14\,\% < 49\,\%$, so the network **demonstrates some degree of correlation** with respect to the smoking behaviour.

Three social processes explaining correlation

- **Homophily** — the tendency to link to others who share a certain similarity with us ("birds of a feather flock together").
- **Confounding** — the correlation can also be forged by *external* influences from the environment ("individuals living in the same city are more likely to become friends than random individuals").
- **Influence** — a process causing behavioural correlations between adjacent actors ("if most of one's friends switch to another mobile company, one might be influenced and switch as well").

Why distinguish influence from homophily and confounding? *Analysis:* to predict the dynamics of the system — whether a new norm of behaviour, technology or idea can diffuse like an epidemic. *Design:* to design a system for inducing a particular behaviour, e.g. vaccination strategies (random, targeting a demographic group, random acquaintances) or viral marketing campaigns.

The logistic model of influence and the shuffle test

In studies of influence modelling, influence is determined by *timestamps*: for an edge (u, v), u can influence v at discrete time steps $t = 0, 1, 2, \ldots$; for each t, every inactive node becomes active with probability $p(a)$, where a is the number of active contacts. The probability of activation is modelled as a *logistic* function of the number of active friends:

$$p(a) = \frac{e^{\alpha \ln(a+1)+\beta}}{1 + e^{\alpha \ln(a+1)+\beta}}, \qquad \text{equivalently} \quad \ln \frac{p(a)}{1-p(a)} = \alpha \ln(a+1) + \beta,$$

where a is the number of active friends, α the *social correlation coefficient*, β a constant explaining the innate bias for activation, and $\ln(a+1)$ the explanatory variable.

Activation likelihood. Suppose at a time point t, $y_{a,t}$ users with a active friends become active and $n_{a,t}$ users who also have a active friends stay inactive. The likelihood of activation at time t is $\prod_a p(a)^{y_{a,t}}\big(1-p(a)\big)^{n_{a,t}}$ and for the whole sequence $\prod_t \prod_a p(a)^{y_{a,t}}\big(1-p(a)\big)^{n_{a,t}}$. Given the user activity log we can compute the α and β that maximise this likelihood; α **measures the correlation between nodes**.

The shuffle test (a test for influence). The key idea: *if influence does not play a role, the timing of activation should be independent of the timing of other actors* — even if we randomly shuffle the timestamps of the user activities, we should obtain a similar social correlation. *The test:* shuffle the timestamps of the user activities; if the new estimate of the social correlation is *significantly different* from the estimate based on the activity log, then there is evidence of **influence**.

The procedure: *(1)* for all a compute $y_{a,t}$ and $n_{a,t}$; *(2)* compute the α, β that maximise the likelihood; *(3)* pick a random permutation π of the activation times and set the activation time of node v_i to $t'_i = t_{\pi(i)}$; from the new times compute $y'_{a,t}$, $n'_{a,t}$; *(4)* compute the α', β' that maximise the new likelihood; *(5)* the model exhibits *no* social influence if α and α' are close to each other.

6.9 Homophily, selection and social influence

The homophily test

The motivation: "birds of a feather flock together" — your friends are more similar to you in age, race, interests and opinions than a random collection of individuals. *Homophily* is the principle that we tend to be similar to our friends.

The test: if each node were *randomly* assigned an attribute according to the balance in the real network, the number of "heterogeneous" edges should not change significantly relative to the real network. If a fraction p are males and q are females, then the probability of a male–male edge is p^2, of a female–female edge q^2, and of a *cross-gender* edge $2pq$.

If the fraction of heterogeneous (cross-gender) edges is significantly less than $2pq$, then there is evidence for homophily.

Example: there are 5 cross-gender edges out of 18; $p = 6/9 = 2/3$, $q = 3/9 = 1/3$; without homophily there should be $2pq = 4/9$, i.e. 8 out of 18 $\Rightarrow$ *evidence of homophily.*

Notes on the test. Significantly *more* than $2pq$ heterogeneous edges = *inverse homophily* (male–female dating relationships). The test works for any characteristic and also for more than two values — the number of heterogeneous edges is compared to a randomly generated graph using the real proportions as probabilities.

Selection vs. social influence

Triadic closure: when two individuals share a common friend, a friendship between them is more likely — the link may form even if neither is aware of the mutual friend. The formation of a link can rarely be attributed to a single factor.

Selection — people choose friends who are like themselves; it operates actively (choosing similar friends in a small group) as well as through the environment (a homogeneous school population).

Mutable vs. immutable characteristics: *immutable* ones do not change (gender, race) or change consistently with the population (age); *mutable* ones change over time (behaviours, interests, opinions).

Social influence is the *opposite* of selection: selection — characteristics drive the formation of links; influence — existing links shape mutable characteristics.

Longitudinal studies: from a single snapshot of a network, selection and influence cannot be distinguished — links and behaviour must be tracked over time (*does behaviour change after changes to the network, or the network after changes in behaviour?*). The answer matters for designing interventions: an anti-drug programme "friends persuade friends" only works if the homophily arose

by *influence*, not by selection. *Christakis and Fowler* (obesity, 12 000 people, 32 years) tested *selection*, *confounding* and *social influence* — and **found significant evidence for influence**: obesity is a "contagion" spreading through social influence.

6.10 Affiliation networks and closure processes

Affiliation and social-affiliation network

An **affiliation network** is a *bipartite graph* of persons and *affiliations* (*foci*: activities, organisations, ...). A **social-affiliation network** has *two types of edges*: *person–person* (a social relationship) and *person–focus* (participation).

Three closure processes

- **Triadic closure** — two people connect because they have a common friend.
- **Focal closure** — a shared focus connects them; closure due to *selection*.
- **Membership closure** — a person joins a focus where their friend already is; closure due to *social influence*.

Measurement: from two snapshots of the network (t_1, t_2), for each k define $T(k)$ = the fraction of pairs that had exactly k common friends (resp. foci) and no edge at t_1 and formed an edge by t_2. The baseline model of independent opportunities: $T_{\text{base}}(k) = 1 - (1-p)^k$.

Empirically (the emails of 22 000 students — Kossinets and Watts 2006; LiveJournal — Backstrom et al.; Wikipedia — Crandall et al.) all three closures show the same pattern: a *significant increase from 1 to 2* common friends/foci and *diminishing returns* afterwards; sharing a class has nearly the same effect as sharing a friend. On Wikipedia, moreover, the similarity of editors (the overlap of edited articles) was measured around the first interaction: the similarity grows *already before* the link is formed (*selection*) and continues to grow *after* it (*influence*).

6.11 The Schelling model of segregation

The Schelling model (T. Schelling, 1970s)

A simple model that shows how *global* patterns of self-chosen segregation might occur because of *local* homophily.

- The population is formed by two types of individuals — agents X and O; the two types represent an *immutable* characteristic (race, ethnicity, country of origin, native language).
- Agents are placed randomly in a grid.
- An agent is *satisfied* with their location if at least t of their neighbours are agents of the same type (t being a pre-set threshold).
- In each round, first identify all *unsatisfied* agents; then all unsatisfied agents move to a location where they are satisfied.

Sometimes an agent cannot find a new location that satisfies — then they are left alone or move to a random location. The moves may cause a *previously satisfied agent to no longer be satisfied* — which is why the process continues.

The result: the grid is more segregated than before.

Model variations: schedule agents to move in random order or in a sweeping motion; agents move to the nearest satisfying location or to a random one; the threshold could be a percentage or vary among or within population groups; the world could wrap (the left edge meets up with the right edge). There are many other variations, but *the results usually end up the same: self-imposed segregation.*

Conclusion: the Schelling model is an example of how *immutable* characteristics can become highly correlated with *mutable* ones: the choice of where to live (mutable) over time conforms to the agents' type (immutable), producing segregation (homophily).

6.12 Positive and negative relationships: structural balance

Until now a relation in a network has had a positive connotation; there exist, however, social networks where negative (antagonistic, enemy) relationships can also be expressed. *Positive* relations = friendly, *negative* = enemy. The question: **when is the network in a balanced state?** We shall see how local properties enforce global ones (cf. the Schelling model). A simplification: we consider networks where *all* pairs of nodes are connected by a positive or negative relation (i.e. a complete graph, a clique) — applicable to small groups of people or to countries; we relax it later.

The structural balance property

For three nodes there are (up to symmetry) four sign configurations:

Signs	Interpretation	State
$+,+,+$	A, B, C are mutual friends	*balanced*
$+,-,-$	A and B are friends with a mutual enemy C	*balanced*
$+,+,-$	B and C are friends of A, but they don't get along with each other	not balanced
$-,-,-$	A, B, C are mutual enemies	not balanced

The structural balance property: for every triangle, all three edges are positive, or *exactly one* is positive.

The Balance Theorem

Theorem. If a labelled *complete* graph is balanced, then either

- all pairs of nodes are friends, *or*
- the nodes can be divided into two groups X and Y such that every pair of nodes in X like each other, every pair in Y like each other, and everyone in X is the enemy of everyone in Y.

Proof. Let G be a balanced network. If all edges are positive, we are done. If there is at least one negative edge, pick a node A and put X = all friends of A (including A), Y = all enemies of A. Then: *(1)* every two nodes in X are friends — otherwise we would have a triangle consisting of A and two of its friends with a single negative edge, i.e. two positive and one negative, which is not balanced; *(2)* every two nodes in Y are friends — otherwise we would have a triangle of A and two of its enemies with three negative edges, which is not balanced; *(3)* every node in X is an enemy of every node in Y — otherwise the triangle of A, a friend from X and an enemy from Y would have two positive and one negative edge. □

Applications of structural balance. *The evolution of alliances in Europe, 1872–1907* (solid dark edges indicate friendship, dotted red edges enmity): the Three Emperors' League 1872–81, the Triple Alliance 1882, the lapse of the German–Russian treaty 1890, the French–Russian alliance 1891–94, the Entente Cordiale 1904, the British–Russian alliance 1907 — the network gradually *slides into a balanced labelling*, and into the First World War. *The conflict over Bangladesh's separation from Pakistan (1972):* the United States's somewhat surprising support of Pakistan becomes less surprising when one considers that the USSR was China's enemy, China was India's foe, and India had traditionally bad relations with Pakistan — since the U.S. was at that time improving its relations with China, it supported the enemies of China's enemies.

Weak structural balance and generalisations

The weak structural balance property: there is no set of three nodes whose edges consist of *exactly two positive edges and one negative edge.* (The "three negative" configuration is therefore allowed — it corresponds to a pair making an alliance against the third one; "two positive and one negative" is a stronger imbalance, because B and C try to resolve their animosity.)

Theorem. If a labelled complete graph is weakly balanced, then its nodes can be divided into *groups of mutual friends* such that every two nodes belonging to different groups are enemies.

Proof. *(1)* Let A be a node and X the set of friends of A, including A. *(2)* All A's friends are

friends with each other. *(3)* A and all its friends are enemies with everyone else. *(4)* Remove X from the network and continue with step 1. □

Generalisation to non-complete graphs. *(a)* Not all pairs in a network are connected — a (non-complete) graph is *balanced* if it can be "completed" by adding edges to form a signed complete graph that is balanced. *(b)* Not all triangles must be balanced — if *most* triangles are balanced, then the network can be *approximately* divided into two fractions.

6.13 Link analysis: hubs, authorities and the HITS algorithm

The year 1998 was an eventful one for web link analysis models — *both* the PageRank and the HITS algorithms were reported in it. The connections between them are quite striking; since then PageRank has emerged as the dominant model, due to its *query-independence*, its ability to combat spamming, and Google's business success.

Authorities and hubs

An **authority** — roughly, a page with many in-links. The idea: the page may have good or authoritative content on some topic and thus many people trust it and link to it.

A **hub** — a page with many *out-links*. It serves as an organiser of the information on a particular topic and points to many good authority pages.

The key idea of HITS: *a good hub points to many good authorities* and *a good authority is pointed to by many good hubs.* Authorities and hubs thus have a *mutual reinforcement* relationship (a densely linked bipartite sub-graph of hubs and authorities).

Why separate hubs from authorities? Competing firms will not link to each other and cannot be viewed as directly endorsing each other. In PageRank, by contrast, endorsement passes directly from one prominent page to another (a page is important if it is cited by other important pages) — this is the dominant mode of endorsement in academic and governmental pages, among bloggers, in personal pages, and in the scientific literature.

The HITS algorithm (Hypertext Induced Topic Search)

Unlike PageRank, which is a *static* ranking algorithm, HITS is search-query *dependent*. When the user issues a query, HITS first expands the list of relevant pages returned by a search engine and then produces *two* rankings of the expanded set: an authority ranking and a hub ranking.

Grabbing pages. Given a broad query q: send q to a search engine; collect the t highest-ranked pages ($t = 200$ in the HITS paper) — this set is called the *root set* S; grow S by including any page pointed to by a page in S and any page that points to a page in S — this gives the larger *base set* B.

Scores. Let $G = (V, E)$ be the hyperlink graph of B with n pages and adjacency matrix L ($L_{ij} = 1$ if there is an edge (i, j), 0 otherwise). Let $a(i)$ be the authority score and $h(i)$ the hub score of page i. The mutual reinforcement:

$$a(i) = \sum_{(j,i)\in E} h(j), \qquad h(i) = \sum_{(i,j)\in E} a(j),$$

in matrix form, with $a = (a(1), \ldots, a(n))^\top$ and $h = (h(1), \ldots, h(n))^\top$:

$$a = L^\top h, \qquad h = La.$$

Computation by power iteration:

1: $a_0 \leftarrow h_0 \leftarrow (1, 1, \ldots, 1)^\top$; $k \leftarrow 1$
2: **repeat**
3: $\quad a_k \leftarrow L^\top h_{k-1}; \quad h_k \leftarrow L a_k$
4: $\quad a_k \leftarrow a_k / \|a_k\|_1; \quad h_k \leftarrow h_k / \|h_k\|_1$ ▷ normalisation
5: $\quad k \leftarrow k + 1$

6: **until** $\|a_k - a_{k-1}\|_1 < \varepsilon$ **and** $\|h_k - h_{k-1}\|_1 < \varepsilon$
7: **return** a and h

Normalisation is indispensable, because the magnitude of the hub and authority values tends to grow with each update; convergence occurs only when normalisation is taken into account (the scores are divided by the sum of all scores).

Spectral analysis of HITS

After k steps (the *spectrum* of a matrix is the set of its eigenvalues),

$$a_k = \left(L^\top L\right)^{k-1} a_0, \qquad h_k = \left(LL^\top\right)^k h_0.$$

The vector a_k therefore has to converge to an *eigenvector* of the matrix $L^\top L$: there are normalising constants c_k such that a_k/c_k converges to a limit, and the direction does not change when multiplied by the matrix exactly when the vector is an eigenvector.
The matrix $L^\top L$ is *symmetric*, so it has n eigenvectors $z_1, \ldots, z_n$ with eigenvalues $c_1, \ldots, c_n$, $|c_1| \geq |c_2| \geq \cdots \geq |c_n|$. Expanding $a_0 = x_1 z_1 + \cdots + x_n z_n$, we get

$$a_k = c_1^k x_1 z_1 + \cdots + c_n^k x_n z_n, \qquad \frac{a_k}{c_1^k} = x_1 z_1 + x_2 \left(\frac{c_2}{c_1}\right)^k z_2 + \cdots$$

Assuming $c_1 > c_2 \geq \cdots$, every term except the first goes to 0 as $k \to \infty$, hence a_k/c_1^k converges to $x_1 z_1$ — to the *principal* eigenvector. (Convergence regardless of the initial vector and the relaxation of the assumption $c_1 > c_2$ are not shown here.)

Strengths and weaknesses of HITS. *Strength:* its ability to rank pages according to the *query topic*, which may provide more relevant authority and hub pages. *Weaknesses: (1)* it can be easily spammed — adding out-links in one's own page is very easy; *(2) topic drift* — many pages in the expanded set may not be on topic; *(3)* inefficiency at query time — collecting the root set, expanding it and performing the eigenvector computation are all expensive operations.

6.14 PageRank

Rank prestige

In prestige measures an important factor is considered: *the prominence of the individual actors who do the "voting".* In the real world, a person chosen by an important person is more prestigious than one chosen by a less important person (a CEO's vote counts for much more than a worker's). If one's circle of influence is full of prestigious actors, then one's own prestige is also high.
The rank prestige $P(i)$ is defined as a linear combination of the ranks of those who point to i:

$$P(i) = A_{1i}P(1) + A_{2i}P(2) + \cdots + A_{ni}P(n), \qquad \text{in matrix form} \quad P = A^\top P,$$

where $A_{ji} = 1$ if j points to i and 0 otherwise. This is the *characteristic equation* of the eigensystem of $A^\top$; P is an eigenvector of $A^\top$.

Basic PageRank

PageRank interprets a hyperlink from page u to page v as a *vote*, by page u, for page v. It looks at more than the sheer number of votes, though — it also analyses the page that casts the vote: *votes cast by important pages weigh more heavily and help make other pages more important.* Since a page may point to many others, its prestige score has to be *shared.*

The Web as a directed graph $G = (V, E)$, $n = |V|$. The PageRank of page i:

$$P(i) = \sum_{(j,i)\in E} \frac{P(j)}{O_j},$$

where O_j is the number of out-links of j. In matrix form: let $A_{ij} = 1/O_i$ for $(i, j) \in E$ and 0 otherwise; then $P = A^\top P$.
The intuition: PageRank is a kind of "fluid" that circulates through the network, passing from node to node across edges and pooling at the most important nodes. *The total PageRank in the network remains constant* — each page takes its PageRank, divides it up and passes it along its links; PageRank is never created nor destroyed, just moved around. Therefore *no normalisation is needed* — unlike HITS.

The problem with the basic definition. In many networks the "wrong" nodes end up with all the PageRank. If F and G point to each other rather than to A, the PageRank that flows to them from C can never circulate back — for large k we get $\frac{1}{2}$ for each of F and G and 0 for all the others. This is a "slow leak": small sets of nodes that can be reached from the rest of the graph but have no paths back. **The solution:** send a small fraction of PageRank to *all* nodes.

Scaled PageRank and the damping factor

Pick a *scaling factor* d between 0 and 1 and replace the basic update rule:
1. first apply the basic PageRank update rule;
2. scale down all PageRank values by a factor of d (the total PageRank in the network has shrunk from 1 to d);
3. divide the residual $1 - d$ units of PageRank *equally* over all nodes, giving $(1 - d)/n$ to each.

Why it works: it is a "water cycle" that evaporates $1 - d$ units of PageRank in each step and rains it down uniformly across all nodes.
Repeated application of the scaled rule *converges* to a set of limiting PageRank values and the equilibrium is *unique* — but the values depend on the choice of d. In practice d is between 0.8 and 0.9 ($d = 0.85$ in the PageRank paper); d is called the *damping factor*.

PageRank as a Markov chain

The Web is modelled as a *stochastic process*: each page is a *state*, a hyperlink is a *transition* with a certain probability. *Random surfing*: each transition probability is $1/O_i$ if we assume the surfer clicks the hyperlinks uniformly at random (the "back" button is not used and no URL is typed in).
Let A be the state transition probability matrix and p_0 the initial probability distribution. Then $p_1 = A^\top p_0$ and in general $p_k = A^\top p_{k-1}$. *A theorem of Markov chains:* a finite Markov chain defined by a stochastic matrix A has a **unique stationary probability distribution** if A is *irreducible* and *aperiodic*. A stationary distribution means that p_k converges to a steady-state vector π regardless of the choice of p_0. At the steady state $\pi = A^\top \pi$, so π is the principal eigenvector of $A^\top$ with eigenvalue 1 — and it is π that is used as the PageRank vector. This is reasonable and intuitive: it reflects the *long-run probabilities* that a random surfer will visit the pages, and a page has high prestige if the probability of visiting it is high.
Neither condition, however, holds for the web graph:

- *A is not stochastic* — many pages have no out-links, which is reflected in the matrix by rows of complete zeros (*dangling pages*). Solutions: *(1)* remove such pages during the computation (they do not directly affect the ranking of any other page); *(2)* add a complete set of outgoing links from each such page to all pages, i.e. $A_{ij} = 1/n$.
- *A is not irreducible* — irreducibility means that the web graph is *strongly connected* (for each pair i, j there is a path from i to j); a general web graph is not.
- *A is not aperiodic* — a state i is *periodic* with period $k > 1$ if k is the smallest number such

that all paths leading from i back to i have a length that is a multiple of k; if a state is not periodic ($k = 1$) it is aperiodic, and the chain is aperiodic if all states are. An example of a periodic chain with $k = 3$: starting from state 1, the only path back to 1 is $1 \to 2 \to 3 \to 1$, so any return takes 3 transitions.

Both of the latter problems are dealt with by a single strategy: add a link from each page to every page and give each such link a small transition probability controlled by a parameter d — the augmented transition matrix is obviously irreducible and aperiodic.

The final PageRank

After this augmentation the random surfer has two options at a page: with probability d they randomly choose an out-link to follow, and with probability $1 - d$ they jump to a random page:

$$P = \left(\frac{1-d}{n}\right)\mathbf{E} + d\,A^\top P,$$

where $\mathbf{E} = \mathbf{e}\mathbf{e}^\top$ ($\mathbf{e}$ being a column vector of all 1's), i.e. $\mathbf{E}$ is an $n \times n$ matrix of all 1's. The matrix $\left(\frac{1-d}{n}\right)\mathbf{E} + dA^\top$ is stochastic (transposed), irreducible and aperiodic. Scaling so that $\mathbf{e}^\top P = n$:

$$P = (1-d)\,\mathbf{e} + d\,A^\top P, \qquad \text{i.e. for each page } i: \quad P(i) = (1-d) + d\sum_{j=1}^{n} A_{ji}P(j),$$

which is equivalent to the formula given in the PageRank paper

$$P(i) = (1-d) + d\sum_{(j,i)\in E}\frac{P(j)}{O_j}.$$

Computation by power iteration:

1: $P_0 \leftarrow \mathbf{e}/n$; $k \leftarrow 1$
2: **repeat**
3: $\quad P_k \leftarrow (1-d)\mathbf{e} + dA^\top P_{k-1}$
4: $\quad k \leftarrow k + 1$
5: **until** $\|P_k - P_{k-1}\|_1 < \varepsilon$
6: **return** P_k

Advantages of PageRank. *Fighting spam:* a page is important if the pages pointing to it are important — and since it is not easy for a page owner to add *in-links* from other important pages, it is not easy to influence PageRank. *A global, query-independent measure:* the PageRank values of all pages are computed and saved *off-line* rather than at query time. **Criticism:** precisely the query-independence — PageRank could not distinguish between pages that are authoritative in general and pages that are authoritative on the query topic.

Summary. Within the framework of link analysis we have seen the principles of *centrality* and *prestige* and the algorithms *HITS* and *PageRank* (which powers Google). Yahoo! and MSN have their own unpublished link-based algorithms. **Important:** hyperlink-based ranking is not the only algorithm used in search engines — it is combined with many content-based factors to produce the final ranking presented to the user.

6.15 Themed communities and their discovery

Links can also be used to find *themed communities* — groups of content creators or people sharing some common interests (web communities, email communities, named entity communities). A related notion is *focused crawling*: combining contents and links to crawl web pages of a specific topic — follow links and use learning/classification to determine whether a page is on topic.

Themed community

Given a finite set of entities $E = \{e_1, \ldots, e_n\}$ *of the same type*, a *themed community* is a pair $K = (T, M)$, where T is the community theme and $M \subseteq E$ is the set of all entities in E that share the theme T. If $e \in M$, then e is said to be a *member* of the community K.

- *A theme defines a community:* given a theme T, the set of members is uniquely determined, so two communities are equal if they have the same theme.
- *A theme can be defined arbitrarily:* it can be an event (a sport event, a scandal) or a concept (web mining).
- *An entity can be in any number of communities:* communities can therefore *overlap* and multiple communities may share members.
- *The entities in E are of the same type:* people and organisations thus cannot be in the same community.

Communities can form a *hierarchical structure*: a themed community (T, M) may include a set of themed *sub-communities* $\{(T_1, M_1), \ldots, (T_k, M_k)\}$, where T_i is a sub-theme of T and $M_i \subseteq M$; (T, M) is then a *super-community*. Each sub-community can be further decomposed.

The objective of discovery: given a data set containing entities, discover the hidden communities — for each community find the *theme* (usually represented by a set of keywords) and its *members*.

Community manifestation in the data: community members are linked in some way and the associated texts contain words indicative of the theme. *Web pages:* the members are more likely to be mutually interconnected through *hyperlinks*; the pages contain *words* related to the theme. *Emails:* the members are more likely to communicate with one another; their emails contain words related to the theme. *Text documents:* the members are more likely to appear *together* in the same sentence and/or document; the words in the sentences indicate the theme.

Bipartite core communities

An (i, j)-*bipartite core* is a complete bipartite sub-graph with at least i *fan* nodes and at least j *center* nodes, i.e. it contains all possible edges between the i fan vertices and the j center vertices. The fans represent *specialised hubs*, the centers correspond to *authorities*.

The procedure for finding bipartite cores:

- *Step 1 — pruning.* Prune by in-degree: delete all pages that are highly referenced, i.e. whose number of in-links is greater than some k (e.g. $k = 50$). Then prune fans and centers iteratively (to find the (i, j)-cores): prune all fans with an out-degree smaller than j; prune all centers with an in-degree smaller than i; delete the edges associated with the pruned nodes.
- *Step 2 — generating all (i, j)-cores.* Find all $(1, j)$-cores, i.e. the set of all vertices with out-degree at least j; construct all $(2, j)$-cores by checking every fan that also points to any center in a $(1, j)$-core; continue in the same fashion up to i.

Limitation: the algorithm finds only the *core* pages of the communities, not all the member pages, and it finds neither the *themes* of the communities nor their hierarchical structure.

Overlapping communities of named entities

The objective: find entity communities from a text corpus. **The main idea:** two entities are linked if they *co-occur in the same sentence* (if two people are mentioned in the same sentence, there is usually a relationship between them); a single entity can appear in multiple communities; the top keywords are assumed to represent the theme of the community. There are promising results when applied to finding communities of political figures and celebrities from web documents.

The algorithm:

1. *Build a link graph* — parse each document and, for each sentence, identify the named entities contained in it; if a sentence has more than one named entity, these entities are *pair-wise* linked; the keywords in the sentence are attached to the linked pairs as textual content;

discard all the other sentences.
2. *Find all triangles* — a triangle consists of three entities bound together and represents the basic building block of communities; the observation is that a community tends to expand by triangles sharing a common edge.
3. *Find community cores* — a community core is a group of tightly bound triangles, i.e. a relaxed complete sub-graph (clique); intuitively, a core consists of a set of tightly connected members of a community.
4. *Cluster around community cores* — the triangles and entity pairs that are in no core are assigned to cores according to their *textual content similarities* with the discovered cores.

6.16 Community detection

Community and community detection

Communities are formed by individuals such that those *within* a group interact with each other *more frequently* than with those outside the group. In different contexts they are also called a *group*, *cluster*, *cohesive subgroup* or *module*.
Community detection is the discovery of groups in a network where the individuals' group memberships are *not explicitly given*.
The definition of a community can be subjective: a densely knit sub-graph may be a community, but so may every connected component.
Two types of groups in social media: *explicit* groups formed by user subscriptions, and *implicit* groups formed by social interactions. Why extract groups from the topology when some sites allow people to join groups? Not all sites provide a community platform; not all people want to make an effort to join groups; groups can change *dynamically*. Network interaction provides information about the relationships between users, complements other kinds of information, helps network visualisation and navigation, and provides basic information for other tasks.

Minimum cut methods

Minimum cut (Ahuja et al., 1993). A *cut* is a partition of the vertices of a graph into two disjoint sets; the *minimum cut problem* is to find the partition that minimises the number of edges running between the two sets (or the sum of the weights in weighted networks) — the so-called *cut size*. The motivation: most interactions occur *within* a group and interactions between groups are rare ⇒ community detection ≈ the minimum cut problem.
For $k > 2$ the strategy of *iterative bisecting* is implemented (in the second and further iterations, groups are sequentially divided into two sub-groups).
Drawback: it produces groups of unbalanced sizes (one set is often a singleton).
Complexity: a simple solution using the max-flow based *s-t* cut algorithm has complexity $O(n^5)$; an implementation using Karger's algorithm has $O(n^4)$ but is not guaranteed always to find the minimum cut. For unit weights $w_{ij} = 1$ and no balancing constraints, the 2-way cut problem is polynomially solvable in $O(n^2 \log n)$ (Karger's randomised minimum cut). Arbitrary edge weights *and* balancing constraints make the problem **NP-hard**.

Ratio cut and normalised cut

The minimum cut often returns an imbalanced partition (one set being a singleton) ⇒ we change the objective function to consider the *community size*. For a partition $\pi = \{C_1, \ldots, C_k\}$:

$$\text{RatioCut}(\pi) = \frac{1}{k}\sum_{i=1}^{k}\frac{\text{cut}(C_i, \bar{C}_i)}{|C_i|}, \qquad \text{NCut}(\pi) = \frac{1}{k}\sum_{i=1}^{k}\frac{\text{cut}(C_i, \bar{C}_i)}{\text{vol}(C_i)},$$

where $|C_i|$ is the number of nodes in C_i, $\text{vol}(C_i)$ the sum of degrees in C_i and $\text{cut}(A, B) = \sum_{i \in A,\, j \in B} a_{ij}$. **Both measures prefer a balanced partition.**

The Girvan–Newman algorithm (2002)

A *divisive hierarchical* algorithm (a top-down approach): it decomposes the initial full graph into progressively smaller connected pieces, until there are no edges to remove and each node represents a community itself.
The idea: identify the edges that connect vertices belonging to *different* communities (the so-called bridges) and iteratively remove them. **The criterion:** *edge betweenness* — the fraction of shortest paths that pass along the edge; edges with high betweenness lie on a large number of shortest paths between vertices and are therefore bridges.

1: compute the betweenness of all edges in the network
2: remove the edge with the *highest* betweenness
3: repeat the previous steps until there are no edges left to remove

Input: the entire graph. **Output:** a hierarchical structure, represented e.g. as a dendrogram.
Selecting the number of communities: the algorithm returns a *set* of possible solutions but does not indicate which one is the best. To select the best partition, compute the *modularity* of each network division and choose the partition with the highest modularity.
Drawback: suitable only for networks of *moderate size* due to the high computational costs — $O(m^2n)$ for a graph with m edges and n vertices, resp. $O(n^3)$ for sparse networks.

An example of computing edge betweenness: the edge betweenness of $e(1,2)$ is 4, as all the shortest paths from 2 to $\{4,5,6,7,8,9\}$ have to pass either $e(1,2)$ or $e(2,3)$, and $e(1,2)$ is the shortest path between 1 and 2. After removing $e(4,5)$, the betweenness of $e(4,6)$ becomes 20 (the highest); after removing $e(4,6)$, the edge $e(7,9)$ has the highest betweenness, 4.

Modularity Q

It quantifies the quality of a given graph division into communities. **The basic idea:** a network has a meaningful community structure if the number of edges *between* communities is smaller than would be expected based on a random choice. The expected number of edges between nodes i and j with degrees k_i, k_j is $k_ik_j/2m$.

$$Q = \frac{1}{2m}\sum_{i,j}\left(A_{ij} - \frac{k_ik_j}{2m}\right)\delta(c_i, c_j),$$

where A_{ij} is the entry of the adjacency matrix (the number of edges between i and j), m the number of edges, k_i the degree of vertex i, c_i the group to which i belongs, and δ the Kronecker delta.
Values: $Q \in [-1, 1]$ and can be either positive or negative. If positive, there is a possibility of finding a community structure in the network; if the values are large and positive, the corresponding partition may reflect the true community structure. For meaningful communities the modularity should be $Q \geq 0.3$ (Clauset et al., 2004).

The Louvain method (Blondel et al., 2008)

A *greedy* optimisation method for *agglomerative hierarchical* modularity maximisation. It comprises two phases:
Phase 1 — consider each node as a single community:
1. compute the modularity Q;
2. move the isolated node from its community to a neighbouring community;
3. compute the gain/loss in modularity yielded by the changed assignment;
4. if the modularity increases (there is a gain), keep the node in the "new" community;
5. repeat until no further improvements are possible.

Output: the k-th level partition (k being the number of the iteration).
Phase 2 — create a new network (a *supergraph*) derived from the original one, where each new

node (a *supervertex*) is the aggregation of the nodes assigned to a given community found in Phase 1; two supervertices are connected by an edge if there is at least one edge linking two vertices inside the corresponding communities.
Iteration: repeat Phases 1 and 2 until the maximum modularity is attained.
Advantages: good performance also on large network graphs at low computational costs — $O(n \log n)$, where n is the number of vertices. **Drawback:** the results are *order-sensitive*.
Further drawbacks: *disconnected communities* — when a node is moved to a different community, the original community may become *internally disconnected*, yet its nodes are still locally optimally assigned and will therefore remain in it; *getting stuck in local optima*; *failing to refine communities* after the initial merging.

The Leiden algorithm (Traag, Waltman, van Eck, 2019)

A refinement of the Louvain method for agglomerative hierarchical modularity maximisation. It has three phases: *(1)* local moving of nodes (similar to Louvain); *(2)* **refinement of the partition**; *(3)* network aggregation (similar to Louvain). Refinement means that every community from phase 1 is split from the inside into connected, well-linked subcommunities (nodes merge only within their own community) and only these are aggregated, so the disconnected communities known from Louvain cannot arise.
Advantages: more efficient than the Louvain method, but it may lead to extended processing times for massive graphs; parallel multicore implementations may boost the speed. **Drawback:** a *hard partition* — the nodes can belong to only one community.

Challenges of community detection. Despite the many algorithms, community detection is still considered a challenging problem, for two main reasons: *(1)* typically the number of communities is unknown and has to be determined; *(2)* communities are usually *heterogeneous* in terms of size and density.

6.16.1 A taxonomy of community detection methods

Four categories according to the level at which the criterion is placed: *node-centric* (each node of the group satisfies a property), *group-centric* (the group as a whole does), *network-centric* (partition the whole network into disjoint sets) and *hierarchy-centric* (a hierarchy of communities).

Node-centric community detection

- **Complete mutuality — cliques.** A *clique* = a maximal complete subgraph; finding the maximum clique is **NP-hard**. *Recursive pruning:* in a clique of size k each node has degree $\geq k-1$, so lower-degree nodes can be removed repeatedly (in scale-free networks most get pruned). Cliques serve rather as *cores* (seeds) of larger communities.
- **Clique percolation** (CPM) — finds *overlapping* communities: find all cliques of size k, build the *clique graph* (two cliques are adjacent iff they share $k-1$ nodes); the communities = the connected components of the clique graph.
- **Reachability.** A *k-clique* — a maximal subgraph with pairwise geodesic distance $\leq k$ (beware: the paths may run *outside* the subgraph, the inner diameter can exceed k); a *k-club* — diameter $\leq k$; a *k-clan* — a k-clique that is also a k-club.
- **Nodal degrees.** A *k-core* — each node connected to at least k members; a *k-plex* — in a group of n_s nodes each node connected to at least $n_s - k$ members. The definitions are complementary, but the sets of all k-cores and $(n_s - k)$-plexes differ (n_s is not fixed).
- **Within vs. outside ties.** An *LS set* — every proper subset has more ties inside the group than outside; too strict for practice. A relaxation is the *k-component* — connected even after removing fewer than k vertices or edges (computed via minimum cut / maximum flow).

Limitations: the definitions are strict, do not scale, and may not match the properties of scale-free networks — they are mainly useful as community cores.

Group-centric detection: quasi-cliques

The condition is placed on the group as a whole, typically its density: a subgraph $G_S = (V_S, E_S)$ is a *γ-dense quasi-clique* if $|E_S| \geq \gamma \cdot \frac{|V_S|(|V_S|-1)}{2}$. It is found by local search over a sample with heuristic pruning (remove nodes with degree $< \gamma(k-1)$, i.e. below the average degree).

Network-centric community detection

The connections are considered *globally*; the goal is to partition the whole network into *disjoint* sets:

- **vertex similarity** — k-means over the similarity of *neighbourhoods* (Jaccard $= \frac{|N_i \cap N_j|}{|N_i \cup N_j|}$, $\cos = \frac{|N_i \cap N_j|}{\sqrt{|N_i||N_j|}}$); structural equivalence is too restrictive;
- **latent space models** — MDS: map the nodes into a low-dimensional space ($S = V\Lambda^{1/2}$ from the double-centred proximity matrix), then k-means;
- **block models** — $A \approx S\,\Sigma\,S^\top$ (S the community indicator matrix, Σ the mixing matrix); after relaxing S, the optimum is the top eigenvectors of A;
- **spectral clustering** — the ratio and normalised cut = minimising $\mathrm{tr}(S^\top L S)$ with the *graph Laplacian* $L = D - A$, resp. $D^{-1/2}(D-A)D^{-1/2}$; after relaxation the optimum is the eigenvectors with the *smallest* eigenvalues (the first one is useless);
- **modularity maximisation** — $Q = \frac{1}{2m}\sum_C \sum_{i,j \in C} \left(A_{ij} - \frac{d_i d_j}{2m}\right)$; the optimum is the *top* eigenvectors of the *modularity matrix* $B = A - \frac{dd^\top}{2m}$, followed by k-means.

A unified view: all four matrix approaches are factorisations $X \approx S\,\Sigma\,S^\top$, with X being, in turn, the proximity matrix, the sociomatrix A, the graph Laplacian and the modularity matrix.
Limitation: the number of communities must be specified beforehand.

Hierarchy-centric community detection

Builds a hierarchy of communities (analysis at different *resolutions*). **Divisive** — e.g. Girvan–Newman ($O(m^2 n)$, for sparse networks $O(n^3)$): repeatedly remove the "weakest" edge (the highest *edge betweenness* = a bridge) until the network decomposes into the desired number of components. **Agglomerative** — e.g. Louvain ($O(n \log n)$): start with the nodes as communities and merge them by the *modularity increase*; *Walktrap* merges according to the results of short random walks.

6.17 Link prediction

The link prediction task

The objective: predict *future* links between pairs of nodes (friend recommendation on Facebook or LinkedIn). **Content-based measures** build on *homophily* (similar content → a link); they are, however, sensitive to the network at hand (noisy tweets), and content-based homophily does *not* guarantee structural connectivity. **Structural measures** build on *triadic closure* (a shared neighbourhood → a link); they work across domains and are *far more powerful* — structural connectivity usually implies content-based homophily. They include the *neighbourhood-based* measures, the *Katz* measure and *random walk*-based measures.

Neighbourhood-based link prediction measures

S_i = the neighbour set of node i; each measure fixes a weakness of the previous one:

- **common neighbours** $\mathrm{CN}(i,j) = |S_i \cap S_j|$ — ignores the degrees: two popular figures have many common neighbours *just by chance*;
- **Jaccard** $\frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ — normalises for the degrees of the end nodes, but not of the *intermediate* neighbours: a very popular common neighbour is common to many pairs and hence less important for the prediction;

- **Adamic–Adar** $\mathrm{AA}(i,j) = \sum_{k \in S_i \cap S_j} \frac{1}{\log |S_k|}$ — the weight of a common neighbour decreases with its degree.

The Katz measure

With few common neighbours the neighbourhood-based measures cannot distinguish; the Katz measure captures *indirect* connectivity through walks of all lengths ($W_{ij}^{(t)}$ = the number of walks of length t, $\beta < 1$ a *discount factor* de-emphasising long walks):

$$\mathrm{Katz}(i,j) = \sum_{t=1}^{\infty} \beta^t W_{ij}^{(t)}, \qquad K = \sum_{t=1}^{\infty} \beta^t A^t = (I - \beta A)^{-1} - I.$$

The sum of the Katz coefficients of a node with respect to the other nodes is its **Katz centrality**.

Link prediction as a classification problem

The presence of an edge = a binary class; the features of a pair = the similarity measures (neighbourhood-based, Katz, walk-based) + preferential-attachment features (node degrees). It is a *positive-unlabelled* problem: pairs with an edge are positive, and approximate "negative" examples are sampled from the unlabelled ones (a sparse network has far too many). The base classifier is typically *logistic regression*, in a cost-sensitive version due to the imbalance. **Advantages:** content features are easy to add; the classifier learns the relevance of the features *by itself*; for directed networks the features can be extracted *asymmetrically* (in- and out-degrees, personalised PageRank).

Summary: *neighbourhood-based* measures are cheap even for extensive data and perform almost as well as the other unsupervised ones; the *Katz* and *walk*-based measures help in *very sparse* networks with few common neighbours; *supervision* is more accurate and adaptable but expensive; content enhances the prediction, yet structural measures dominate.

6.18 Summary for the examination

- **SNA** = social science + network analysis + graph theory; it focuses on *relationships*, not on entities and their attributes. Representations: a drawing, an edge list, an adjacency matrix (asymmetric for directed graphs). *Ego network* vs. "whole" network — the *boundary specification problem*: no studied network is whole.
- **Tie strength**: strong vs. weak; *the strength of weak ties* (Granovetter 1973). Homophily, transitivity and bridging; homophily + transitivity → *cliques*. A bridge = an edge whose removal disconnects its endpoints; a *shortcut bridge* = the distance after removal grows beyond 2. **Neighbourhood overlap** $O(i,j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j| - 2}$ — the larger, the stronger the tie.
- **Four centrality measures**: *degree* (how many people it reaches directly), *betweenness* (on how many shortest paths it lies; where the network breaks apart), *closeness* (the reciprocal mean shortest path length; the speed of dissemination), *eigenvector* ($Ax = \lambda x$, only the *greatest* eigenvalue gives a non-negative vector; how well it is connected to well-connected nodes). Know the interpretation table and be able to explain why one measure is not enough (nodes 3 and 5 *together* > node 10). **Structural equivalence**: the positions can be swapped without changing the structure.
- **Cohesion**: reciprocity (directed graphs only), density ($|E| / \binom{|V|}{2}$; a clique = 1; directed graphs have half), the **clustering coefficient** $C_i = \frac{2|\{e_{jk}\}|}{k_i(k_i-1)}$ and the network average, the *diameter* = the longest shortest path, the average distance, eccentricity. A **small world** = high C + a short average path.
- **Six properties of real-world complex networks**: the small-world effect (Milgram, median 6), transitivity/clustering, power-law degree distributions, resilience (robust to random, vulnerable to targeted removal of hubs), mixing patterns (assortative mixing = homophily), community

structure. **Scale-free networks**: growth + *preferential attachment* (rich-get-richer, the Matthew effect, cumulative advantage); the reasons — popularity, quality, a mixed model (the halo effect). **Centralisation** and the *core-periphery* structure (bow-tie analysis).

- **Influence modelling**: the common features (a directed graph, active/inactive, *an activated node cannot be deactivated*). **LTM** — a threshold θ_v uniform on $[0, 1]$, activation when $\sum_{\text{active}} w_{u,v} \geq \theta_v \sum w_{u,v}$; focuses on the *receivers*. **ICM** — a *single* chance to activate each neighbour with probability $p_{v,u}$; focuses on the *senders*. **Influence maximisation** is NP-hard and the greedy approach gives **63 %** of the optimum.
- **Influence vs. correlation**: the *test for correlation* (the fraction of edges between different attribute values vs. $2p(1-p)$; the smoker example $14\,\% < 49\,\%$); the three processes — homophily, *confounding*, influence; the logistic model $\ln \frac{p(a)}{1-p(a)} = \alpha \ln(a+1) + \beta$ with $\alpha =$ the social correlation coefficient; the **shuffle test** — shuffle the timestamps, and if α' differs significantly from α this is evidence of *influence*.
- **The homophily test**: significantly fewer heterogeneous edges than $2pq \Rightarrow$ homophily; significantly *more* $\Rightarrow$ *inverse* homophily (dating relationships). Be able to give the example: 5 out of 18 vs. $2pq = 8$ out of 18.
- **Selection vs. social influence**: selection = characteristics drive link formation; influence = links shape *mutable* characteristics. *Immutable* (gender, race, age) vs. *mutable* (behaviour, opinions). They can be told apart only by *longitudinal* studies (obesity: Christakis & Fowler, 12 000 people over 32 years $\rightarrow$ evidence for *social influence*).
- **Affiliation networks**: bipartite (persons $\times$ foci); a social-affiliation network has two types of edges. **Three closures**: triadic (a common friend), *focal* (a common focus; *selection*), *membership* (a friend already in the focus; *social influence*). The methodology for measuring $T(k)$ from two snapshots; the baseline model $T_{\text{base}}(k) = 1 - (1-p)^k$; empirically a large increase from 1 to 2 and then *diminishing returns*.
- **The Schelling model**: agents of two types in a grid, satisfied with $\geq t$ neighbours of the same type, the unsatisfied move $\Rightarrow$ *self-imposed segregation*. It shows how an *immutable* characteristic becomes highly correlated with a *mutable* one (the choice of where to live) and how local homophily creates a global pattern.
- **Structural balance**: the balanced triangles are $+++$ and $+--$ (all three positive, or exactly one positive). **The Balance Theorem**: a complete balanced graph $\Rightarrow$ all friends, or a division into two groups of internal friends who are mutual enemies — be able to give the proof ($X =$ the friends of A, $Y =$ the enemies of A, three steps). **Weak balance**: only the *two positive and one negative* configuration is forbidden $\Rightarrow$ a division into *any number* of groups. Applications: the European alliances 1872–1907, Bangladesh 1972.
- **HITS**: query-*dependent*; the root set S ($t = 200$) $\rightarrow$ the base set B; $a = L^\top h$, $h = La$, power iteration with **indispensable normalisation**; $a_k = (L^\top L)^{k-1} a_0$ converges to the principal eigenvector of $L^\top L$. Weaknesses: easily spammed, topic drift, inefficiency at query time.
- **PageRank**: query-*independent*, $P(i) = \sum_{(j,i)\in E} P(j)/O_j$, *needs no normalisation* (the total PageRank is constant — the "fluid"). The "slow leak" problem $\Rightarrow$ the *scaled* rule with the damping factor d (0.8–0.9; 0.85 in the paper), the "water cycle". A Markov chain needs a *stochastic, irreducible and aperiodic* matrix — the web graph satisfies none of them (dangling pages, not strongly connected, periodicity); all of it is dealt with by a *single* strategy (a link from each page to every page with a small probability). The final form $P(i) = (1-d) + d \sum_{(j,i)\in E} P(j)/O_j$, computed by power iteration. Advantages: resistance to spam (in-links cannot easily be obtained), a global offline measure; the criticism: query-independence.
- **Themed communities**: $K = (T, M)$, the theme defines the community, communities can *overlap*, a hierarchy of sub-communities. *Bipartite cores* — an (i, j)-core, fans = hubs, centers = authorities; pruning + core generation; it finds only the cores, neither the themes nor the hierarchy. *Named entity communities*: a graph from co-occurrences in a sentence $\rightarrow$ triangles $\rightarrow$ cores $\rightarrow$ clustering by textual similarity.
- **Community detection — the definition is subjective**; explicit vs. implicit groups. *Mini-*

mum cut: unbalanced groups, $O(n^5)$ / Karger $O(n^4)$, **NP-hard** with general weights and balancing constraints ⇒ the *ratio* and *normalised cut* (dividing by $|C_i|$, resp. $\mathrm{vol}(C_i)$).

- **Girvan–Newman**: divisive, removes the edges with the highest *betweenness*, $O(m^2n)$ / $O(n^3)$; the number of communities is chosen by the highest **modularity** $Q = \frac{1}{2m}\sum_{i,j}(A_{ij} - \frac{k_i k_j}{2m})\delta(c_i, c_j)$, $Q \in [-1, 1]$, meaningful communities at $Q \geq 0.3$.
- **Louvain**: greedy agglomerative modularity maximisation in two phases (local moves → supergraph), $O(n \log n)$; drawbacks: order sensitivity, *disconnected communities*, local optima. **Leiden** adds a *partition refinement* phase; hard partition only. The challenges: the unknown number of communities, heterogeneous sizes and densities.
- **Four categories of methods**: *node-centric* (cliques — NP-hard, recursive pruning of nodes with degree $< k - 1$, **CPM** for *overlapping* communities via a clique graph of cliques sharing $k - 1$ nodes; k-clique / k-club / k-clan; k-core and k-plex are complementary; LS sets and k-components), *group-centric* (γ-dense quasi-cliques), *network-centric* (vertex similarity — Jaccard, cosine + k-means; MDS; block models; spectral clustering via the graph Laplacian with the *smallest* eigenvalues; modularity maximisation via the modularity matrix with the *largest* — all unified as the matrix factorisation $X \approx S\Sigma S^\top$, which requires the number of communities to be given), *hierarchy-centric* (divisive = Girvan–Newman, agglomerative = Louvain, Walktrap).
- **Link prediction**: content-based measures (homophily) vs. structural ones (triadic closure; stronger). Neighbourhood-based measures: *common neighbours* $|S_i \cap S_j|$ (does not account for degrees), *Jaccard* $\frac{|S_i \cap S_j|}{|S_i \cup S_j|}$ (normalises the degrees of the two nodes but not of the intermediate neighbours), *Adamic–Adar* $\sum_k \frac{1}{\log |S_k|}$ (the weight decreases with the degree of the common neighbour). **The Katz measure** $\sum_t \beta^t W_{ij}^{(t)} = (I - \beta A)^{-1} - I$ for *sparse* networks with few common neighbours; the sum over the other nodes = *Katz centrality*. **The classification view**: a positive-unlabelled problem, a sample of negatives, logistic regression as the base classifier, cost-sensitive variants; the advantage — the classifier learns the relevance of the features itself and *asymmetric* features can be used for directed networks.

7 Practical use of the techniques

The last sentence of the topic asks for both halves to be brought together: what the techniques covered are *actually* used for. In the examination it helps to be able to say, for each application, *which task* it solves (Chapter 1), *by which method* (Chapter 3) and *how it is evaluated* (Chapter 5).

7.1 Data mining

Application	Task type / method	Note
Market basket analysis	association rules, Apriori / FP-Growth	planning and layout of shops, coupons and time-limited discounts, "packaging" of products; comparison of new and existing shops using *virtual items*
Fraud detection	classification, outlier detection	20 million procedures billed to a health insurance company → 214 thousand odd ones → 14 thousand passed on for manual review → 8 thousand confirmed frauds; the machine *focuses* the expert's attention
Credit scoring	decision trees, naive Bayes, logistic regression	the running example throughout the text; *comprehensibility* is required (regulation), as is the handling of missing values
Customer segmentation	cluster analysis (k-means, hierarchical)	representing a segment by its *centroid* (the "typical customer") is very intuitive for business
Churn prediction	classification on imbalanced data	accuracy is not enough ⇒ precision/recall, F_1, ROC/AUC; *lift analysis* for deciding how many customers to contact
Targeted marketing campaigns	scoring and ranking, lift	the economic reasoning over the lift curve (the watch example: bin 1 +\$550, bin 10 −\$475)
Text classification	SVM with kernels	among the best classifiers for text (high-dimensional data); comprehensibility is not expected
Time series prediction	regression, neural networks	the series is transformed by a sliding window into a table of inputs and outputs
Classification of chemical compounds	classification of structural data	representation by SMILES strings or facts in Prolog
Web usage mining	sequential patterns (GSP)	navigational patterns of users based on the sequences of page visits
Recommender systems [♦ beyond the slides]	cluster analysis, classification, link prediction	collaborative filtering (similarity of users or items) vs. content-based filtering; the cold-start problem is addressed by content

A note on the motivation for decision trees. The lecture cites the case of Barclays bank, which introduced a telephone system routing callers by their credit rating: clients with considerable savings or a sufficient credit limit were connected to a British operator, the others to an Indian call centre. The aim was to "filter out" the customers who do not have the money to buy the bank's products. The criticism: the system is preferential towards wealthier customers and is not intended to give the best advice. It is a good illustration that a *technically correct* classification model can have problematic consequences — and that the *Evaluation* phase of CRISP-DM must include a view beyond the accuracy of the model.

7.2 Social network analysis

Area	Use
Businesses	analysis and improvement of the communication flow in an organisation, with partners or customers; marketing campaigns based on social networking (visualisation of the communication flows before and after the introduction of a content management system)
Law enforcement agencies and the army	identification of criminal and terrorist networks from traces of collected communication; identification of the *key players* in them (centrality measures, Chapter 6)
Social network sites (Facebook)	identification and recommendation of potential friends based on "friends-of-friends" — precisely the *link prediction* task
Civil society organisations	uncovering conflicts of interest in hidden connections between government bodies, lobbies and businesses
Network operators	optimisation of the structure and capacity of communication networks
Medicine	contagious diseases and their prevention; sexually transmitted diseases — the syphilis outbreak in Rockdale County (Atlanta, 1996): a group of young white girls about 16 years old, a group of white boys aged 17 to 21, and a group of African-American boys aged 17 to 21; syphilis was diagnosed in 6 white females, 2 white males and 2 African-American males
Scale-free networks	*computing:* networks with an SF architecture (the WWW) are extremely resistant to random failures but very vulnerable to coordinated attacks — the eradication of internet viruses is essentially impossible; *medicine:* vaccination campaigns focused on hubs (people with many contacts — though identifying them is difficult), new drugs targeting the key molecules (hubs) involved in a disease, minimising the side effects by using the link networks found inside the cells; *business:* understanding the mutual links between companies, industry and the economy in order to monitor and eliminate cascaded financial crashes; marketing — exploring the spread of contagion in SF networks and more efficient advertising of new products
Community detection	*A Belgian mobile phone operator* (the Louvain method): about 2 million customers; the size of a node is proportional to the number of individuals in the community and the colour represents the main language spoken in the community (red for French, green for Dutch); only communities of more than 100 customers are plotted. A zoom at higher resolution reveals an *intermediate* community made of sub-communities with less apparent language separation

Why and when to use SNA. To study a social network (offline or online) or improve its effectiveness; to visualise data so as to uncover patterns in relationships or interactions; to follow the paths along which information flows in social networks; in quantitative research (though for qualitative research a network perspective is also valuable) — the range of actions and opportunities afforded to individuals is often a function of their *positions* in social networks, and uncovering these positions (say as fathers, mothers, teachers, workers) can yield interesting and sometimes surprising results; a quantitative analysis of a network helps identify different types of actors (key players); to analyse social media in general, identify the causes of dysfunctional communities or networks, and promote social cohesion and growth in an online community.

Thoughts on design

SNA theory has inspired some of the first social network sites (e.g. SixDegrees), but it is still not used so often in conjunction with design decisions. Open questions:

- How can an online social media platform (or its administrators) leverage the methods and insights of SNA?
- How can a platform encourage its users to act (*locally*) in such a way that would expand the community?
- What measures can an online community take to optimise its structure? Example: *cliques*

can be undesirable because they shun newcomers.
- What are the desirable structures for different types of online platforms?
- How can online communities identify and utilise key players for their benefit?

7.3 Operational and ethical aspects of deployment

[♦ beyond the slides] Deploying a model (the *Deployment* phase of CRISP-DM, Section 1.5) is not the end of the project:

- **Monitoring and drift.** The distribution of the data changes over time (*data drift*), as does the relation between the inputs and the target (*concept drift*) — the model has to be retrained regularly and the drop in quality on production data monitored.
- **Feedback of the model into the data.** A deployed model influences the very reality it measures: a scoring model for approving loans only ever sees the applicants to whom it granted a loan, so the further training data is biased by the model itself.
- **Privacy.** Social network analysis works with data *about relationships* that the user often could not influence; even an anonymised graph can be de-anonymised from its structure. Link prediction and influence modelling are essentially tools for predicting and influencing the behaviour of concrete people.
- **Fairness and discrimination.** A model learned on historical data reproduces historical inequality (see the Barclays case above); a protected attribute can often be *reconstructed* from the other attributes, so simply omitting it is not enough.
- **The legal framework.** GDPR Art. 22 (the right to an explanation of an automated decision) and the AI Act for high-risk systems — in regulated domains interpretability is a legal requirement, not a convenience.

7.4 Summary for the examination

- For each application, be able to give the **triple**: the type of task (descriptive / predictive, supervised / unsupervised), the method used, and the way it is evaluated.
- **Flagship data mining applications**: market basket analysis (association rules), fraud detection (the health insurance example — the machine *focuses* the expert's attention rather than replacing them), credit scoring (trees and Bayes; the comprehensibility requirement), customer segmentation (clustering, the centroid = the typical customer), churn and targeted marketing (imbalanced data → ROC/AUC and *lift*), text classification (SVM), time series prediction (a sliding window), web usage mining (sequential patterns).
- **Flagship SNA applications**: identification of key players in criminal and terrorist networks (centrality), friend recommendation (link prediction), optimisation of communication networks, epidemiology (Rockdale County 1996), vaccination targeted at *hubs*, marketing and cascaded financial crashes in scale-free networks, segmentation of an operator's customers by language communities (Louvain, 2 million customers).
- **The key property of SF networks for practice**: resilience to random failures (up to 80 % of nodes) vs. vulnerability to a targeted attack (5–15 % of hubs) — from which both the vaccination strategy and the impossibility of eradicating internet viruses follow.
- **Deployment is not the end**: monitoring of drift, feedback of the model into the data, privacy (even an anonymised graph can be de-anonymised), fairness (a protected attribute can be reconstructed from the others), GDPR Art. 22 and the AI Act.